行，-c 选项用于选定不在"[a-z][A-Z]"字符集内的字符，tr 命令将选定的字符转换成换行符，\012 是换行符的八进制码，*表示将换行符任意扩展，使其等于被替换的字符集个数，例 5-33 的结果显示该 tr 命令确实已将 AREACODE.db 文件中的冒号、数字和空白行删除。

例 5-33 的*符号表示前面字符的任意扩展，事实上，tr 命令可以用[character*n]表示 character 字符的 n 次重复出现，下面给出这一用法的例子：

```
#例 5-34：[character*n]在 tr 命令中的用法
#将 REPEAT 文件连续出现 5 次 o 字符改为*字符
alloy@alloy:~/linuxshell/CH05$ tr "[o*5]" "*" <REPEAT
W*****mennnn
TTTTheyyyyyyy
alloy@alloy:~/linuxshell/CH05$
```

例 5-34 中的 tr 命令通过"[o*5]"匹配连续出现 5 次 o 字符形成的字符串，并将该字符串替换为*字符串。

5.8 tar 命令

tar 命令是 Linux 的归档命令，通俗地说，tar 命令实现了 Linux 系统文件的压缩和解压缩，类似于 Windows 下的 WinRAR 软件。tar 命令是 Linux 系统最常用的命令之一，很多 Linux 安装文件包首先需要用 tar 命令进行解压缩才能使用。本节介绍 tar 命令的基本用法，tar 命令的基本格式如下：

```
tar [选项] 文件名或目录名
```

tar 命令的选项比较多，我们将常用的选项列在表 5-9 中。

表 5-9　tar 命令选项及其意义

选　项	意　义
-c	创建新的包
-r	为包添加新的文件
-t	列出包内容
-u	更新包中的文件，若包中无此文件，则将该文件添加到包中
-x	解压缩文件
-f	使用压缩文件或设备，该选项通常是必选的
-v	详细报告 tar 处理文件的信息
-z	用 gzip 压缩和解压缩文件，若加上此选项创建压缩包， 那么解压缩时也需要加上此选项

下面举几个例子说明 tar 命令的用法，首先给出例 5-35，它演示了 tar 命令创建和显示包的用法：

```
#例 5-35：tar 命令-c 和-f 选项的用法
alloy@alloy:~/linuxshell/CH05$ tar -cf db.all *.db        #将所有.db 结尾的文件放入压缩包
alloy@alloy:~/linuxshell/CH05$ ls db*
db.all                                                    #创建压缩包的名字
```


混混藏书阁 http://book-life.blog.163.com

```
alloy@alloy:~/linuxshell/CH05$ tar -tf db.all                    #查看 db.all 压缩包的内容
AREACODE.db
CARGO2.db
CARGO.db
HOBBY.db
NAME.db
PROFESSOR.db
SORT_CARGO.db
TEACHER1.db
TEACHER.db
TEACHER_HOBBY.db
alloy@alloy:~/linuxshell/CH05$
```

上面的例 5-35 用 tar 命令创建一个新的包，名字是 db.all，内容是当前目录下所有以.db 结尾的文件，创建包时，tar 命令用了-c 和-f 选项，-c 表示创建新的包，-f 通常是必选选项；接着，例 5-35 演示了 tar 命令带上-t 和-f 选项查看包内容的用法，-t 表示列出包内容，再带上必选的-f 选项，可以看到，all.db 确实包含了所有以.db 结尾的文件。

对于已创建的包，tar 命令的-r 选项可将新的文件加入到其中，下面的例 5-36 演示了 tar -r 的用法：

```
#例 5-36：tar 命令-r 选项的用法
[root@jselab shell-book]# tar -rf db.all log*          #将以 log 开头的文件添加到 db.all 中
[root@jselab shell-book]# tar -tf db.all               #查看 db.all 的内容
AREACODE.db
CARGO2.db
CARGO.db
HOBBY.db
NAME.db
PROFESSOR.db
SORT_CARGO.db
TEACHER1.db
TEACHER.db
TEACHER_HOBBY.db
loggg                                                  #新添加到 db.all 的文件
loggg1                                                 #新添加到 db.all 的文件
[root@jselab shell-book]#
```

例 5-36 将当前目录中以 log 开头的文件添加到 db.all 包，-r 选项表示为包添加新的文件，log*表示所有以 log 开头的文件，当再次查看 db.all 包的内容时发现 db.all 多了 loggg 和 loggg1 两个文件。

tar 命令的-u 选项用于更新包中的文件，比如，我们对当前目录的 TEACHER.db 文件进行修改，需要将修改后的 TEACHER.db 文件添加到 db.all 包，那么只需要使用命令 tar -uf db.all TEACHER.db，就可实现 TEACHER.db 文件的更新操作。如果包中不存在需要更新的文件，那么 tar -u 就将该文件添加到包，因此，从该角度来说，-u 选项也可用于为包添加新的文件，-u 选项完全能代替-r 选项。

tar 命令的另一重要功能就是解压缩，Linux 系统下有很多种压缩包格式，如.tar、.gz、.tar.gz、.tgz 和.Z 等结尾的压缩包名，从应用的角度来说，我们没有必要搞清楚这些不同格式压缩包的区别，也无须用不同的命令来对这些不同格式的包解压缩，本节给出两个 Linux 系统下解压的通用命令，格式如下：

```
tar -xvf 压缩包名称                                    #解压非 gzip 格式的压缩包
```

```
tar -zxvf 压缩包名称                          #解压 gzip 格式的压缩包
```

上述两种 tar 命令分别针对非 gzip 格式和 gzip 格式，加上-z 选项可以解压用 gzip 压缩的文件，-x 选项表示解压缩文件， -v 选项能生成解压缩过程的状态信息，-f 选项是必选选项，这些选项的次序可以交换，对结果没有影响。下面我们举一个例子说明 tar 命令的解压用法：

```
#例 5-37：tar 命令解压压缩包
[root@jselab shell-book]# tar -zxvf db.all        #db.all 非 gzip 格式，加上-z 选项解压出错

gzip: stdin: not in gzip format
tar: Child returned status 1
tar: 由于前次错误，将以上次的错误状态退出
[root@jselab shell-book]# tar -xvf db.all         #去掉-z 选项，解压 db.all 成功
AREACODE.db                                       #加上-v 选项才生成这些解压出的文件信息
CARGO2.db
CARGO.db
HOBBY.db
NAME.db
PROFESSOR.db
SORT_CARGO.db
TEACHER1.db
TEACHER.db
TEACHER_HOBBY.db
loggg
loggg1
[root@jselab shell-book]#
```

例 5-37 利用 tar 命令解压 db.all 包，由于 db.all 非 gzip 格式，当加上-z 选项时，解压出错；去掉-z 选项解压成功，而且由于-v 选项的存在，tar 命令自动生成被解压出的文件信息。

tar -cf 命令创建的包实际上是将多个文件放到一起，此时文件并没有被压缩，gzip 命令是 Linux 系统中常用的压缩工具，它可以对 tar 命令创建的包进行压缩，但是，gzip 所生成的压缩包使用 tar -zxvf 命令就可解压缩，下面给出一个介绍 gzip 及其解压缩的例子：

```
#例 5-38：gzip 命令基本用法
[root@jselab shell-book]# gzip db.all             #压缩 db.all 包
[root@jselab shell-book]# ls *.gz
db.all.gz                                         #db.all 变成 db.all.gz
[root@jselab shell-book]# tar -zxvf db.all.gz     #解压缩 db.all.gz 包
AREACODE.db
CARGO2.db
CARGO.db
HOBBY.db
NAME.db
PROFESSOR.db
SORT_CARGO.db
TEACHER1.db
TEACHER.db
TEACHER_HOBBY.db
loggg
loggg1                                            #tar 命令解压 gzip 包成功
[root@jselab shell-book]#
```

例 5-38 中的 gzip 命令压缩 db.all 包，生成名为 db.all.gz 的文件，db.all.gz 和 db.all 的内容是一样的，只是 db.all.gz 文件经过了压缩，占用空间比较小；由于 db.all.gz 是 gzip 格式的压缩文件，利用 tar 命令对其解压缩时，需要加上-z 选项。

混混藏书阁 http://book-life.blog.163.com

当然，gzip 命令也可将 db.all.gz 文件还原到 db.all 文件，只要在 gzip 命令后面加上-d 选项就可实现，下面的例 5-39 给出了 gzip -d 命令的用法：

```
#例 5-39: gzip 命令-d 选项的用法
[root@jselab shell-book]# gzip -d db.all.gz #将 db.all.gz 解压缩
[root@jselab shell-book]# ls db*
db.all                                    #gzip -d 的结果仍然是包，与 tar 命令存在区别
[root@jselab shell-book]#
```

gzip -d 对 db.all.gz 的处理也称为解压缩，它得到的是包，而 tar -zxvf 命令可以直接得到包内的文件。

5.9 本章小结

本章的内容极为丰富，介绍了 Linux 的文本处理命令，sort 命令是一种文本排序工具；uniq 命令可以去除文本的重复记录，而且可以统计重复文本行的数量；join 命令用于实现类似于关系数据库中的连接操作；cut 命令用于从标准输入或文本文件中按域或行提取文本；paste 命令用于将多个文本文件或标准输出中的内容粘贴而形成新的文件；split 命令用于将大文件切割成小文件；tr 命令实现字符转换功能，可以实现文本文件的过滤功能；tar 命令是 Linux 的归档命令，tar 命令和 gzip 命令用于实现 Linux 系统文件的压缩和解压缩。这些功能强大的 Linux 文本处理命令经常出现在各种 Shell 脚本程序中，是 Shell 编程的基础。

5.10 上机提议

1．下面给出一个记录学生信息的文件 student，每条记录包含 5 个域，域分隔符是短横线“-”，5 个域依次是：姓名-专业-出生年份-籍贯-学制。请按下面的要求对此文件进行排序：

```
Q Cai-English-1984-Jiangsu-7
Z Wu-Computer-1982-Jiangsu-9
H Yuan-Transportation-1978-Anhui-9
K Song-Chemistry-1982-Shanghai-4
Y Gao-Physical-1981-Hubei-3
L Li-Architecture-1977-Guangdong-7
N Tang-Computer-1983-Jiangsu-7
```

（1）按照第 2 域（即专业）进行排序，对于专业相同的记录，按照姓名排序；

（2）按照第 3 域（即出生年份）进行排序，出生年份数值大的排在前面，如 1984 的记录应该排在 1982 的记录之前；

（3）按照第 4 域（即籍贯）进行排序，籍贯相同的记录学制小的排在前面，并将此排序结果存储到 province 文件中；

（4）分别对 student 文件和 province 文件测试是否排序；

（5）输出每个专业的学生数。

2．5.1.2 节给出的 PROFESSOR.db 文件以文本块存储教授的信息，现要求对文本块根据

学校名字（每个文本块的第 2 行）进行排序，结果仍然以文本块的格式输出。

3．增加 5.2 节给出的 count_word.sh 脚本的功能，使其能过滤冒号、短横线（-）和@符号三种特殊符号。

4．修改 5.2 节给出的 count_word.sh 脚本，使它能不区分大小写统计文本文件中单词的个数，如 word、WORD、Word 都识别为同一个单词。（提示：结合使用 tr 命令的大小写转换功能来实现。）

5．定义 score 文件用于记录学生的平均成绩，结合第 1 题所给出的 student 文件，输出 student 和 score 文件中共有学生的信息及其平均成绩。（提示：使用 join 命令可以方便地实现，但是，用 join 之前需要对两个文件进行排序。）

```
Q Cai-92
Z Wu-69
Y Zhuang-70
H Yuan-79
L Li-80
G Wang-60
N Tang-75
```

6．将 student 文件中的第 1 域（即姓名）和 score 文件的第 2 域（即平均成绩）提取出来，并粘贴到一个新的文件中。（提示：结合使用 cut 和 paste 命令来实现。）

7．将 student 文件中的第 1 域（即姓名）粘贴到屏幕，每行显示三位学生的姓名。（提示：结合使用 cut 和 paste 命令，并需要注意 paste -的用法。）

8．定义 department 文件用于院系名及其地址的对应关系，结合第 1 题所给出的 student 文件，在 student 文件记录中输出该学生所在院系的地址。

```
English-54 Building
Computer-Li Wen Zheng Building
Transportation-55 Building
Chemistry-Central Building
Physical-Chunhui Building
Architecture-Architecture Building
```

9．按文件大小降序列出当前目录中的所有文件。（提示：ll 命令可以列出文件的详细信息，按照文件大小对 ll 命令结果进行排序。）

10．利用 Shell 命令列出 Linux 系统中标识号大于 99 的用户数量。

11．将 student 文件分别按照行数和大小进行分割，体会 split 命令-b 和-C 选项的区别。

12．将 student 文件中的所有小写字母转换为大写字母。

13．将 student 文件中的非字母字符全部转换为@符号。（提示：注意*符号的用法）

14．将 5.1.2 节所给出的 PROFESSOR.db 文件中的文件块格式转换为单行显示，域之间利用冒号分隔，例如：

```
文件块：
Y Zhang
Victory University
Melbourne, Australia

转换为：Y Zhang: Victory University: Melbourne, Australia
```

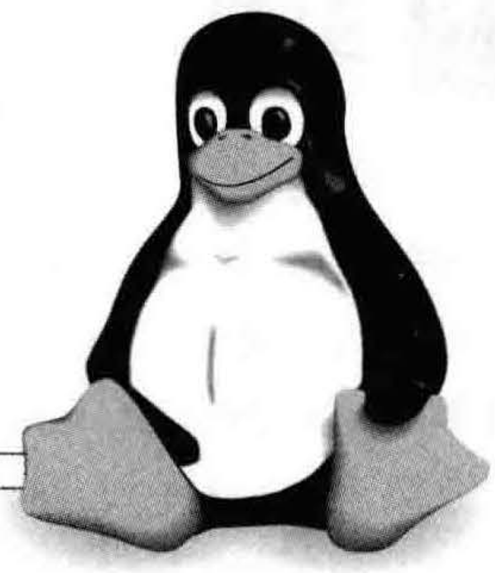
15．下面给出一个脚本，用于将 Windows 系统下的文本文件转换成 Linux 系统下的文本文件，脚本名字是 WintoLin.sh，内容如下：

混混藏书阁 http://book-life.blog.163.com

```
#!/bin/bash
# WintoLin.sh: Window 到 Linux 文本文件的转换.

E_WRONGARGS=65

 if [ -z "$1" ]
  then
  echo "Usage: `basename $0` filename-to-convert"
     exit $E_WRONGARGS
 fi

NEWFILENAME=$1.unx

CR='\015'

tr -d $CR < $1 > $NEWFILENAME                    # 删除回车键，并且写到新文件中

echo "Original Windows text file is \"$1\"."
echo "Converted Linux text file is \"$NEWFILENAME\"."

exit 0
```

请读者利用所学知识读懂 WintoLin.sh 脚本，并执行它；然后编写一个脚本 LintoWin.sh，能将 Linux 下的文本文件转换为 Windows 下的文本文件。

16. 将 Linux 某目录下的所有文件用 tar 命令打包，记录包所占用的字节数，再用 gzip 命令将该包压缩，再次记录包所占用的字节数。



第6章

变量和引用

变量是任何一门编程语言和脚本语言的核心，而 Shell 脚本使用变量就需要引用，因此，引用和变量密切相关。本章首先从变量的替换和赋值的基本操作入手，讨论 Shell 脚本变量的无类型性，并对 Linux Shell 环境变量和位置参数两种特殊的变量做了详尽分析。然后，介绍 Linux Shell 中四种引用符号及其意义和用法，重点讨论转义符的用法及其一些特殊的用法。

混混藏书阁 http://book-life.blog.163.com

6.1 变量

变量用于保存有用信息，如路径名、文件名、数字等，Linux 用户使用变量定制其工作环境，使系统获知用户相关的配置。变量本质上是存储数据的一个或多个计算机内存地址。

变量可分为：本地变量、环境变量和位置参数。本地变量是仅可以在用户当前 Shell 生命期的脚本中使用的变量，本地变量随着 Shell 进程的消亡而无效，在新启动的 Shell 中依旧无效，它类似于 C、C++、Java 等编程语言中局部变量的概念。环境变量则适用于所有由登录进程所产生的子进程，简言之，环境变量在用户登录后到注销之前的所有编辑器、脚本、程序和应用中都有效。位置参数也属于变量，它用于向 Shell 脚本传递参数，是只读的。

6.1.1 变量替换和赋值

本节介绍变量的基本操作：变量替换和赋值。Shell 的三类变量中，位置参数是只读变量，因此没有变量的替换和赋值操作，其他两类变量都存在上述操作。本节不区分变量是本地变量还是环境变量，只对变量替换和赋值的方法进行介绍。

变量是某个值的名称，引用变量值就称为变量替换，$符号是变量替换符号，如 variable 是变量名，那么，$variable 就表示变量的值。将值赋给某个变量名就称为变量赋值，变量赋值有两种格式，如下所示：

```
variable=value
${ variable=value }
```

对于变量赋值，有以下三点说明：

- 等号的两边可以有空格，这不影响赋值操作；
- 如果值（value）中包含空格，则必须用双引号括起来；
- 变量名只能包括大小写字母（a～z 和 A～Z）、数字（0～9）、下画线（_）等符号，并且变量名不能以数字开头，否则视为无效变量名。

下面的例 6-1 演示了变量的赋值和替换，值得注意的是：首先，${variable}和$variable 都表示了变量的值，即进行变量替换；其次，如果值中有空格，需要用双引号引起来，如例 6-1 中的 variable2 变量。

```
#例 6-1：变量的赋值和替换
alloy@alloy:~/linuxshell/CH06$ variable1=33
alloy@alloy:~/linuxshell/CH06$ echo ${variable1}                    #两种方式进行变量替换
33
alloy@alloy:~/linuxshell/CH06$ echo $variable1
33
alloy@alloy:~/linuxshell/CH06$ variable2="hello world"              #值包含空格，用引号引起
alloy@alloy:~/linuxshell/CH06$ echo $variable2
hello world
alloy@alloy:~/linuxshell/CH06$
```

如果例 6-1 中 variable2 的赋值不用双引号引起来，将出现以下错误：

```
alloy@alloy:~/linuxshell/CH06$ variable2=hello world
-bash: world: command not found
```

```
alloy@alloy:~/linuxshell/CH06$
```

由于空格的存在，Shell 将空格后的字符（即 world）解析为新的命令，因此错误信息为 bash Shell 中 world 命令不存在。

当然，变量赋值中也可以使用另一个变量，如例 6-2 所示，variable4 赋值时使用了 variable2 变量，echo 命令输出 variable4 时自动查找 variable2 的值，然后扩展变量，显示整个变量值。

```
#例 6-2：变量赋值中使用另一个变量
alloy@alloy:~/linuxshell/CH06$ variable2="hello world"
alloy@alloy:~/linuxshell/CH06$ variable4="We are saying $variable2" #使用 variable2
alloy@alloy:~/linuxshell/CH06$ echo $variable4
We are saying hello world
alloy@alloy:~/linuxshell/CH06$
```

利用 unset 命令可以清除变量的值，命令格式为：

```
unset 变量名
```

例 6-3 给出一个 unset 命令的例子，将 variable4 变量清除后，echo 命令输出空白行，这表示 variable4 变量未被初始化。

```
#例 6-3：unset 命令的用法
alloy@alloy:~/linuxshell/CH06$ unset variable4          #清除 variable4 变量
alloy@alloy:~/linuxshell/CH06$ echo $variable4
                                                        #空白行表示 variable4 变量未被初始化
alloy@alloy:~/linuxshell/CH06$
```

除了使用等号进行变量赋值以外，变量赋值还有如表 6-1 所示的模式。

表 6-1　变量赋值的模式

模　式	意　义
variable=value	将 value 值赋给变量 variable
variable+value	对已赋值的 variable，重设其值
variable?value 或 variable:?value	对未赋值的 variable，显示系统错误信息
variable:=value	对未赋值的 variable，将 value 值赋给它
variable:-value	对未赋值的 variable，将 value 值赋给它，但 value 值不存储到 variable 对应的地址空间

“:=”和“:-”是两种常用的符号，需要读者理解两种符号的用法及其异同点，首先，请看下面的例 6-4，该例子先将 colour 赋值为 black，然后输出${colour:=blue}和${colour:-blue}的值，两者输出结果相同，都为 black，这是因为 colour 变量已经赋过值，“:=”和“:-”都不对 colour 进行重新赋值。注意，使用以上两种符号时，都需要用花括号将赋值式子括起来，否则，Shell 将 colour:=blue 整个字符串当作变量名进行处理。

```
#例 6-4：“:=”和“:-”的相似之处
alloy@alloy:~/linuxshell/CH06$ colour=black
alloy@alloy:~/linuxshell/CH06$ echo "The Background is ${colour:=blue}"
                                                        #:=符号的用法
The Background is black
alloy@alloy:~/linuxshell/CH06$ echo "The Background is ${colour:-blue}"
                                                        #:-符号的用法
The Background is black
alloy@alloy:~/linuxshell/CH06$
```

其次，再看下面的例 6-5，它说明了“:=”和“:-”符号的区别，该例首先清除 colour 变

混混藏书阁 http://book-life.blog.163.com

量值，再分别输出${colour:=blue}、$colour 和${colour:-blue}、$colour 的值，结果显示：${colour:=blue}和${colour:-blue}的值都为 blue，因为 colour 未赋值，两个符号都将 blue 赋给 colour。但是，调用${colour:-blue}后，colour 值仍然为空，即“:-”符号未将 blue 真正存储到 colour 中，而调用${colour:=blue}后，colour 值却变为 blue，即“:=”符号将 blue 真正存储到 colour 中。

```
#例 6-5：“:=”和“:-”的区别
alloy@alloy:~/linuxshell/CH06$ unset colour
alloy@alloy:~/linuxshell/CH06$ echo "The Background is ${colour:-blue}"
                                                    #用:-符号对 colour 赋值
The Background is blue
alloy@alloy:~/linuxshell/CH06$ echo $colour          #colour 未曾真正赋值

alloy@alloy:~/linuxshell/CH06$ echo "The Background is ${colour:=blue}"
                                                    #用:=符号对 colour 赋值
The Background is blue
alloy@alloy:~/linuxshell/CH06$ echo $colour          #colour 已赋值
blue
alloy@alloy:~/linuxshell/CH06$
```

“:?”和“?”可用于测试变量是否被赋值，若变量未赋值，Shell 提示错误信息，下面给出一个简单的例子。

```
#例 6-6：“:?”和“?”的用法
alloy@alloy:~/linuxshell/CH06$ unset colour
alloy@alloy:~/linuxshell/CH06$ echo "The Background is ${colour:?blue}"
-bash: colour: blue                                 #bash 给出的错误信息
alloy@alloy:~/linuxshell/CH06$ echo "The Background is ${colour?blue}"
-bash: colour: blue                                 #bash 给出的错误信息
alloy@alloy:~/linuxshell/CH06$
```

最后，介绍将变量设置为只读的方法，变量一旦设置为只读，任何用户不能对此变量进行重新赋值，如果有用户对此变量重新赋值，Shell 提示错误信息。设置只读可以使用 readonly 关键字，格式如下：

```
variable=value                                      #先对一个变量进行赋值
readonly variable                                   #将 variable 变量设置为只读
```

例 6-7 演示了利用 readonly 命令将变量设置为只读的用法，例中命令将 colour 设置为只读变量，当再对 colour 进行重新赋值时，Shell 提示错误信息“-bash: colour: readonly variable”，即 colour 是只读的，不能对其进行赋值操作。

```
#例 6-7：设置变量只读
alloy@alloy:~/linuxshell/CH06$ colour=blue
alloy@alloy:~/linuxshell/CH06$ readonly colour      #将 colour 变量设置为只读
alloy@alloy:~/linuxshell/CH06$ colour=grey          #试图改变 colour 变量的值
-bash: colour: readonly variable                    #bash 给出的错误信息
alloy@alloy:~/linuxshell/CH06$
```

如果要查看系统中所有的只读变量，只需执行 readonly 命令即可，例 6-8 给出 readonly 命令的执行结果，可以看到，最后一个结果是前面定义的只读变量 colour。

```
#例 6-8：显示所有的只读变量
alloy@alloy:~/linuxshell/CH06$ readonly
declare -ir BASHPID=""
declare -ar BASH_VERSINFO='([0]="4" [1]="0" [2]="16" [3]="1" [4]="release" [5]="i386-
redhat-linux-gnu")'
```

```
    declare -ir EUID="0"
    declare -ir PPID="2590"
    declare   -r   SHELLOPTS="braceexpand:emacs:hashall:histexpand:history:interactive-
comments:monitor"
    declare -ir UID="0"
    declare -r colour="blue"                                    #该变量是例 6-7 中所设置的
    alloy@alloy:~/linuxshell/CH06$
```

当然，用 readonly 设置变量为只读只是其中的一种方法，我们还可以利用 declare 和 typeset 命令实现同样的功能。

6.1.2 无类型的 Shell 脚本变量

在 C、C++、Java 等编程语言中，定义变量需要声明其类型，如整型、浮点型、字符型等。Shell 脚本变量却是无类型的，这与 awk 变量是一样的。bash Shell 不支持浮点型，只支持整型和字符型，默认情况下，Shell 脚本变量是字符型的，同时，字符型的变量还具有一个整型值，为 0，尽管如此，bash Shell 并不要求在定义一个变量时声明其类型。但是，Shell 会根据上下文判断出数值型的变量，并进行变量的算术运算和比较等数值操作。判断标准是变量中是否只包含数字，如果变量只包含数字，则 Shell 认定该变量是数值型的，反之，Shell 认定该变量是字符串。

下面通过一个例子分析 Shell 对无类型变量的处理方式，新建一个名为 integer.sh 的脚本，内容如下：

```
#例 6-9：integer.sh 脚本分析 Shell 处理无类型变量的方式
#!/bin/bash

a=2009
let "a+=1"                      #让 a 加 1
echo "a=$a"

b=xx09
echo "b=$b"
declare -i b                    #将 b 强制定义为整型
echo "b=$b"

let "b+=1"                      #让 b 加 1
echo "b=$b"

exit 0
```

integer.sh 脚本首先定义 a 变量，值为 2009，然后将 a 的值增加 1，再输出 a 的值。从下面的 integer.sh 脚本执行结果可以看到，Shell 自动将 a 解析为数值变量，a=2010。let 命令用于在变量上执行算术运算，实际上，let "a+=1"等价于 a+=1。然后，integer.sh 脚本定义 b 变量，值为 xx09，b 显然是字符型的，我们利用 declare 命令将 b 强制转化为整型，发现 b 的值并没有改变，即 declare 强制转化并没有起作用。将 b 执行算术操作，增加 1，结果为 b=1，即字符型变量的数值为 0。

```
#例 6-9 integer.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH06$chmod u+x integer.sh
alloy@alloy:~/linuxshell/CH06$ ./integer.sh
a=2010
```


混混藏书阁 http://book-life.blog.163.com

```
b=xx09
b=xx09
b=1
alloy@alloy:~/linuxshell/CH06$
```

declare 命令与 typeset 命令一样，用于定义和限制变量的属性，declare -r 可将变量设置为只读，等价于 readonly 命令。declare 和 typeset 的详细用法可参见 8.4 节。

接下来，我们通过一个例子说明 Shell 对空字符串和未定义变量的处理方式，新建 null-undeclare.sh 脚本，内容如下：

```
#例 6-10：null-undeclare.sh 脚本处理空字符串和未定义变量的方式
#!/bin/bash

c=""                                    #定义 c 为空字符串
echo "c=$c"
let "c+=1"
echo "c=$c"

echo "e=$e"                             #不预先定义 e，直接输出
let "e+=1"
echo "e=$e"

exit 0
```

null-undeclare.sh 脚本定义 c 变量为空字符串，将 c 执行算术操作，增加 1，结果为 c=1，即空字符串变量的数值仍为 0。然后，null-undeclare.sh 脚本直接输出未曾预先定义的变量 e，结果 e 为空值，将 e 加 1，结果为 e=1。null-undeclare.sh 脚本的执行结果如下所示：

```
#例 6-10 null-undeclare.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH06$chmod u+x null-undeclare.sh
alloy@alloy:~/linuxshell/CH06$ ./ null-undeclare.sh
c=
c=1
e=
e=1
alloy@alloy:~/linuxshell/CH06$
```

从以上两个脚本不难总结出：Shell 脚本变量是无类型的，并且 Shell 变量同时有数值型和字符型两种赋值，数值型的初值为 0，字符型的初值为空，而且可以不预先定义变量而直接使用它。

6.1.3 环境变量

环境变量不仅在 Shell 编程方面，而且在 Linux 系统管理方面，都起着非常重要的作用，本节从环境变量的基本操作入手，重点介绍几个 Linux 的重要环境变量，并介绍 Linux 环境变量的几个常用配置文件，最后举一个父子进程的例子进一步说明本地变量和环境变量。

1. 定义和清除环境变量

环境变量的替换和赋值依然遵循 6.1.1 节所述的一般规律，环境变量也是无类型的，环境变量的特殊之处仅在于它的值适用于所有由登录进程所产生的子进程。定义环境变量的格式如下：

```
#定义环境变量的基本格式
ENVIRON-VARIABLE=value              #环境变量赋值
```

```
export ENVIRON-VARIABLE          #声明环境变量
```

在给环境变量赋值后，用 export 命令声明一下，就说明此变量为环境变量，环境变量的名称一般由大写字母组成。

例 6-11 演示了如何定义一个环境变量。首先将 APPSPATH 变量赋值为/usr/local，然后利用 export 命令将 APPSPATH 声明为环境变量。

```
#例 6-11：演示定义环境变量
alloy@alloy:~/linuxshell/CH06$ APPSPATH=/usr/local          #为 APPSPATH 赋值
alloy@alloy:~/linuxshell/CH06$ export APPSPATH              #将 APPSPATH 声明为环境变量
alloy@alloy:~/linuxshell/CH06$ echo $APPSPATH
/usr/local
alloy@alloy:~/linuxshell/CH06$
```

如果要列出系统中所有的环境变量，可使用 env 命令，由于系统中环境变量的数量太多，例 6-12 仅列出了系统中的一些环境变量。

```
#例 6-12：列出系统中的环境变量
alloy@alloy:~/linuxshell/CH06$ env
TERM=xterm
SHELL=/bin/bash
XDG_SESSION_COOKIE=4488e7baed5632cb2ff04a810000000a-1398666301.311874-20165746
SSH_CLIENT=192.168.0.102 58648 22
SSH_TTY=/dev/pts/1
USER=alloy
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;01:cd=40
;33;01:or=40;31;01:su=37;41:sg=30;43:ca=30;41:tw=30;42:ow=34;42:st=37;44:ex=01;32:*.tar
=01;31:*.tgz=01;31:*.arj=01;31:*.taz=01;31:*.lzh=01;31:*.lzma=01;31:*.tlz=01;31:*.txz=
01;31:*.zip=01;31:*.z=01;31:*.Z=01;31:*.dz=01;31:*.gz=01;31:*.lz=01;31:*.xz=01;31:*.bz2=
01;31:*.bz=01;31:*.tbz=01;31:*.tbz2=01;31:*.tz=01;31:*.deb=01;31:*.rpm=01;31:*.jar=01;3
1:*.war=01;31:*.ear=01;31:*.sar=01;31:*.rar=01;31:*.ace=01;31:*.zoo=01;31:*.cpio=01;31:
*.7z=01;31:*.rz=01;31:*.jpg=01;35:*.jpeg=01;35:*.gif=01;35:*.bmp=01;35:*.pbm=01;35:*.pg
m=01;35:*.ppm=01;35:*.tga=01;35:*.xbm=01;35:*.xpm=01;35:*.tif=01;35:*.tiff=01;35:*.png=
01;35:*.svg=01;35:*.svgz=01;35:*.mng=01;35:*.pcx=01;35:*.mov=01;35:*.mpg=01;35:*.mpeg=0
1;35:*.m2v=01;35:*.mkv=01;35:*.webm=01;35:*.ogm=01;35:*.mp4=01;35:*.m4v=01;35:*.mp4v=01
;35:*.vob=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=01;35:*.asf=01;35:*.rm=01;35:*.rmvb=01;35:
*.flc=01;35:*.avi=01;35:*.fli=01;35:*.flv=01;35:*.gl=01;35:*.dl=01;35:*.xcf=01;35:*.xwd
=01;35:*.yuv=01;35:*.cgm=01;35:*.emf=01;35:*.axv=01;35:*.anx=01;35:*.ogv=01;35:*.ogx=01
;35:*.aac=00;36:*.au=00;36:*.flac=00;36:*.mid=00;36:*.midi=00;36:*.mka=00;36:*.mp3=00;3
6:*.mpc=00;36:*.ogg=00;36:*.ra=00;36:*.wav=00;36:*.axa=00;36:*.oga=00;36:*.spx=00;36:*.
xspf=00;36:
MAIL=/var/mail/alloy
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games
PWD=/home/alloy/linuxshell/CH06
LANG=zh_CN.UTF-8
SHLVL=1
HOME=/home/alloy
LANGUAGE=zh_CN:zh
LOGNAME=alloy
SSH_CONNECTION=192.168.0.102 58648 192.168.0.108 22
LESSOPEN=| /usr/bin/lesspipe %s
LESSCLOSE=/usr/bin/lesspipe %s %s
_=/usr/bin/env
OLDPWD=/home/alloy/linuxshell
```

清除环境变量的方法与清除其他变量的方法一样，都是用 unset 命令，例 6-13 将

混混藏书阁 http://book-life.blog.163.com

APPSPATH 变量清除，清除 APPSPATH 变量后，APPSPATH 变量为空，因而 echo $APPSPATH 命令输出空白行。

```
#例 6-13：清除环境变量
alloy@alloy:~/linuxshell/CH06$ unset APPSPATH
alloy@alloy:~/linuxshell/CH06$ echo $APPSPATH
                                        #输出空白行，因为 APPSPATH 已被清除
alloy@alloy:~/linuxshell/CH06$
```

2. 重要的环境变量

环境变量通常用来存储路径信息，Linux 系统及其诸多应用程序的正常运行依赖于某些重要环境变量的正确设置，下面介绍 Linux 系统中极为重要的几个环境变量。

（1）PWD 和 OLDPWD

PWD 记录当前的目录路径，当利用 cd 命令改变当前目录时，系统自动更新 PWD 的值，OLDPWD 记录旧的工作目录，即用户所处的前一个目录。

下面的例 6-14 打印出 PWD 和 OLDPWD 两个变量的值，当前目录是/home/alloy/linuxshell/CH06，旧的工作目录是/home/alloy/linuxshell。

```
#例 6-14：打印 PWD 和 OLDPWD 变量值
alloy@alloy:~/linuxshell/CH06$ echo $PWD
/home/alloy/linuxshell/CH06
alloy@alloy:~/linuxshell/CH06$ echo $OLDPWD
/home/alloy/linuxshell
alloy@alloy:~/linuxshell/CH06$
```

（2）PATH

PATH 是 Linux 中一个极为重要的环境变量，它用于帮助 Shell 找到用户所输入的命令。用户所输入的每个命令实际上是一个源代码文件，计算机执行这个文件里的代码以实现这个命令的功能，这些源代码文件称为可执行文件。可执行文件存在于各种各样的目录下，一些可执行文件放置在 Linux 系统标准目录中，还有一些可执行文件可能是用户写的程序或 Shell 脚本文件，它们放置在用户目录中。PATH 就记录了一系列的目录列表，Shell 为每个输入命令搜索 PATH 中的目录列表。

例 6-15 显示了 PATH 环境变量的内容，可以看到，PATH 中包含了多个目录的路径，它们之间用冒号分隔，都是以 bin 或 sbin 的文件夹结尾，bin 或 sbin 是 Linux 中存放可执行文件的文件夹。任何在 PATH 中的可执行文件都可以在 Linux 系统的任何目录中直接执行。

```
#例 6-15：打印 PATH 变量值
alloy@alloy:~/linuxshell/CH06$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games
alloy@alloy:~/linuxshell/CH06$
```

如果需要在 PATH 中添加新目录，可以使用下面的命令：

```
export PATH="/new directory":$PATH
```

命令中的 new directory 就是新加的目录，后面用冒号加$PATH，表示 new directory 加上旧的 PATH 变量值，得到新的 PATH 变量值。

（3）HOME

HOME 记录当前用户的根目录，由/etc/passwd 的倒数第 2 个域决定，HOME 目录用于保存用户自己的文件。例 6-16 显示了用户 alloy 的 HOME 变量值，一般来说，非根用户的 HOME 目录都存放在/home 目录下，通常以用户名命名。

```
#例 6-16：显示 root 和 editor 两个用户的 HOME 变量值
alloy@alloy:~/linuxshell/CH06$ echo $HOME          #alloy 用户的 HOME 变量值
/home/alloy
alloy@alloy:~/linuxshell/CH06$
```

（4）SHELL

SHELL 变量保存默认的 Shell，默认的值为/bin/bash，表示当前的 Shell 是 bash Shell，如果有必要用另一个 Shell，则需要重置 SHELL 变量值。

（5）USER 和 UID

USER 和 UID 是保存用户信息的环境变量，USER 表示已登录用户的名字，UID 则表示已登录用户的 ID。例 6-17 输出 USER 和 UID 的值，可以看到，当前登录用户为 alloy，其用户 ID 为 1000。

```
#例 6-17：打印 USER 和 UID 变量的值
alloy@alloy:~/linuxshell/CH06$ echo $USER $UID
alloy 1000
alloy@alloy:~/linuxshell/CH06$
```

（6）PPID

PPID 是创建当前进程的进程号，即当前进程的父进程号，关于此环境变量的示例，可参见例 6-24。

（7）PS1 和 PS2

PS1 和 PS2 称为提示符变量，用于设置提示符格式，比如，例 6-17 中的 Shell 提示符指的是 alloy@alloy:~/linuxshell/CH06$这段文字，方括号里包含了当前用户名、主机名和当前目录等信息，这些信息并非固定不变，它可以通过 PS1 和 PS2 的设置而改变。

PS1 是用于设置一级 Shell 提示符的环境变量，也称为主提示符字符串，例 6-18 打印了 PS1 变量的值。

```
#例 6-18：打印 PS1 变量值
alloy@alloy:~/linuxshell/CH06$ echo $PS1
\[\e]0;\u@\h: \w\a\]${debian_chroot:+($debian_chroot)}\u@\h:\w\$
alloy@alloy:~/linuxshell/CH06$
```

表 6-2 列出了部分提示符变量中的特殊符号，因篇幅所限，我们仅举一例说明几个提示符变量的用法，如例 6-19 所示，该例将 PS1 的值改为：[]\u\@\H \w \#]\$。

表 6-2　提示符变量中特殊符号及其意义

模　式	意　义
\d	以“周 月 日”格式显示的日期
\H	主机名和域名
\h	主机名
\s	Shell 的类型名称
\T	以 12 小时制显示时间，格式为：HH:MM:SS
\t	以 24 小时制显示时间，格式为：HH:MM:SS
\@	以 12 小时制显示时间，格式为：am/pm
\u	当前的用户名
\v	bash Shell 的版本号

混混藏书阁 http://book-life.blog.163.com

续表

模　式	意　义
\V	bash Shell 的版本号和补丁号
\w	当前工作目录的完整路径
\W	当前工作目录名字
\#	当前命令的序列号
\$	如果 UID 为 0，打印#；否则，打印$

```
#例 6-19：改变 PS1 变量值
alloy@alloy:~/linuxshell/CH06$  PS1="[\u\@\H \w \#]\$"
[alloy09:43 下午 alloy ~/linuxshell/CH06 114]$ls
```

PS2 是用于设置二级 Shell 提示符的环境变量，例 6-20 首先显示 PS2 的值，为“>”，然后演示当在 Shell 中输入不完全命令时，将出现二级提示符，二级提示符内容即为 PS2 变量的值。

```
#例 6-20：打印 PS2 变量值
alloy@alloy:~/linuxshell/CH06$ echo $PS2                          #打印 PS2 变量值
>
alloy@alloy:~/linuxshell/CH06$ echo "E-Commence
> "                                                               #出现二级提示符
E-Commence
alloy@alloy:~/linuxshell/CH06$
```

我们同样可以使用表 6-2 中的特殊符号来改变 PS2 变量值，例 6-21 给出一个重新设置 PS2 变量值的例子，它将 PS2 设为“\s >”，二级提示符变为“bash >”。

```
#例 6-21：改变 PS2 变量值
alloy@alloy:~/linuxshell/CH06$ PS2="\s >"
alloy@alloy:~/linuxshell/CH06$ $echo "E-Commence"
-bash >
```

（8）IFS

IFS 用于指定 Shell 域分隔符，默认值为空格。例 6-22 重新设置 IFS 变量的值，它将 IFS 设定为冒号，然后显示 PATH 的值，Shell 就以冒号为分隔符，将 PATH 的各目录以空格分隔而显示出来。

```
#例 6-22：改变 IFS 变量值
alloy@alloy:~/linuxshell/CH06$ export IFS=:
alloy@alloy:~/linuxshell/CH06$ echo $PATH
/usr/lib/qt-3.3/bin /usr/kerberos/sbin /usr/kerberos/bin /usr/lib/ccache /usr/local/
sbin /usr/local/bin /sbin /bin /usr/sbin /usr/bin /root/bin       #不同路径之间用空格分隔
alloy@alloy:~/linuxshell/CH06$
```

3. 几个环境变量配置文件

由于 Linux 环境变量的数量较多，因此，系统管理员通常不会利用 export 逐个设置环境变量，而是将 export 命令放置在特殊的配置文件之中，Shell 能够在特定时刻执行这些配置文件，从而自动完成环境变量的配置工作。本节介绍.bash_profile、.bashrc 和.bash_logout 三个配置文件，这三个文件存在于用户根目录下，即/home/用户名目录，Linux 中以“.”开头的文件为隐藏文件。因此，上述三个文件为隐藏文件。

/home/用户名/.bash_profile 是最重要的配置文件，当某 Linux 用户登录时，Shell 会自动

执行.bash_profile 文件，但是在 ubuntu12.04 中常常不存在这个文件，此时则会在登录时自动执行用户的配置文件/home/用户名/profile。

例 6-23 是用户 alloy 的.profile 文件内容及其执行命令。

```
#例 6-23: .profile 文件示例
alloy@alloy:~$ cat .profile
# ~/.profile: executed by the command interpreter for login shells.
# This file is not read by bash(1), if ~/.bash_profile or ~/.bash_login
# exists.
# see /usr/share/doc/bash/examples/startup-files for examples.
# the files are located in the bash-doc package.

# the default umask is set in /etc/profile; for setting the umask
# for ssh logins, install and configure the libpam-umask package.
#umask 022

# if running bash
if [ -n "$BASH_VERSION" ]; then
    # include .bashrc if it exists
    if [ -f "$HOME/.bashrc" ]; then
        . "$HOME/.bashrc"
    fi
fi

# set PATH so it includes user's private bin if it exists
if [ -d "$HOME/bin" ] ; then
    PATH="$HOME/bin:$PATH"
fi
alloy@alloy:~$
```

从例 6-23 可以看出，.profile 文件中定义了该用户的环境变量，我们可以在已有行后面添加新的行以定义新的环境变量，但是，新加入的行只有注销用户，并再次登录后方可生效。如果要使新加入的行立即生效，需要利用 source 命令执行. profile 文件。source 命令也称为“点命令”，即句点符号“.”和 source 命令是等价的，source 命令通常用于重新执行刚修改的初始化文件，使之立即生效，而不必注销并重新登录。因此，使.bash_profile 生效可以使用如下两条等价命令：

```
. . profile                                    #注意：句点符号后面用空格与文件名分隔
source bash_profile
```

利用 source 命令执行脚本和在 Shell 中执行脚本是有区别的，source 在当前 bash 环境下执行命令，而执行 Shell 脚本是启动一个子 Shell 来执行命令。因此，如果在 Shell 中直接执行.bash_profile 文件，新的环境变量只在子 Shell 中生效，而无法在当前的 Shell 中生效。但是，使用 source 命令执行.bash_profile 后，新环境变量在当前 Shell 及其子 Shell 中立即生效。

bash Shell 还支持继承于其他 Shell 的登录配置文件，bash Shell 的.bash_login 文件来源于 C Shell 的.login 文件，bash Shell 的.profile 文件来源于 Bourne Shell 和 Korn Shell 的.profile 文件。当用户登录时，首先查找是否存在.bash_profile 文件，若它不存在，则查找是否存在.bash_login 文件，若它也不存在，则查找是否存在. profile 文件。至于上述几类 Linux Shell 的类型和区别在此不作详细介绍，读者可参见本书第 11 章。

如果用户由当前 Shell 创建一个新的 Shell，称为子 Shell（subshell），子 Shell 尝试读

混混藏书阁 http://book-life.blog.163.com

取.bashrc 中的命令以设置环境变量。.bashrc 文件使得用户登录时的环境变量设置与子 Shell 的环境变量设置相分离，使用户具有更大的灵活度。当然，如果系统不存在.bashrc 文件，子 Shell 启动时将不执行任何命令。

.bash_logout 文件在用户注销时执行，用户可以在该文件中写入清除某些环境变量或记录登录时间等命令，.bash_logout 文件也可以不存在，此时，用户注销时将不再执行任何额外的命令。

4. 一个实例

本节举一个父进程创建子进程的例子，以说明本地变量和环境变量的区别，以及诸多环境变量的结合使用，我们创建两个脚本：father.sh 和 child.sh，father.sh 的脚本内容如下：

```
#例 6-24：father.sh 脚本输出其进程号，分别定义本地变量和环境变量，并创建子进程
#!/bin/bash

#输出父进程号
echo "Father Process ID is $$"

#定义本地变量并输出
localvar="Define a local variable."
echo "localvar=$localvar"

#定义环境变量并输出
ENVVAR="Define a environment variable."
export ENVVAR
echo "ENVVAR=$ENVVAR"

#调用 child.sh 脚本，即创建子进程
$PWD/child.sh

#child.sh 执行完毕，输出相关变量值
echo "Return to father process: $$"
echo "localvar=$localvar"
echo "ENVVAR=$ENVVAR"
```

father.sh 脚本首先打印其进程号，$$表示执行该脚本的进程号，6.1.4 节将详细介绍该符号；然后，father.sh 脚本定义本地变量 localvar 和环境变量 ENVVAR；接着，father.sh 脚本调用 child.sh 脚本，即创建执行 child.sh 脚本的子进程，由于 child.sh 与 father.sh 放置在同一目录，因此，用环境变量$PWD 指明 child.sh 的目录，若直接调用 child.sh，Shell 将返回找不到此脚本的错误信息，child.sh 执行完毕后，father.sh 输出进程号、localvar 和 ENVVAR。child.sh 脚本的内容如下：

```
#child.sh：该脚本输出自身进程号及其父进程号、重新定义本地变量和环境变量
#!/bin/bash

#输出自身进程号及其父进程号
echo "Child Process ID is $$"
echo "My Father Process ID is $PPID"

#输出本地变量和环境变量的当前值
echo "localvar=$localvar"
echo "ENVVAR=$ENVVAR"
```

```
#重新定义本地变量和环境变量
localvar="Redefine this local variable."
ENVVAR="Redefine this environment variable."
echo "localvar=$localvar"
echo "ENVVAR=$ENVVAR"
```

child.sh 脚本输出自身进程号及其父进程号，$PPID 就记录了 father.sh 的进程号。然后，输出本地变量 localvar 和环境变量 ENVVAR 的值；接着，child.sh 脚本对 localvar 和 ENVVAR 两个变量重新赋值。

```
alloy@alloy:~/linuxshell/CH06$ chmod u+x father.sh
alloy@alloy:~/linuxshell/CH06$ chmod u+x child.sh
alloy@alloy:~/linuxshell/CH06$ ./father.sh
Father Process ID is 12770                          #父进程号
localvar=Define a local variable.
ENVVAR=Define a environment variable.
Child Process ID is 12771
My Father Process ID is 12770                       #PPID 变量值和其父进程值相等
localvar=                                           #localvar 变量为空
ENVVAR=Define a environment variable.               # ENVVAR 则为父进程中所赋的值
localvar=Redefine this local variable.
ENVVAR=Redefine this environment variable.
Return to father process: 12770
localvar=Define a local variable.                   #localvar 和 ENVVAR 仍为父进程中所赋的值
ENVVAR=Define a environment variable.
alloy@alloy:~/linuxshell/CH06$
```

father.sh 的进程号为 12770，child.sh 的进程号为 12771，child.sh 中的$PPID 变量值为 12770，确实是 father.sh 的进程号。child.sh 输出 localvar 为空值，这说明本地变量无法传递到子进程，而 ENVVAR 为 father.sh 中所赋的值，这说明环境变量对所有的子进程都有效。child.sh 将 localvar 和 ENVVAR 进行了重新赋值，但是，child.sh 执行完毕，返回 father.sh 后，localvar 和 ENVVAR 仍为 father.sh 中所赋的值，这说明无论是本地变量还是环境变量，都无法向其父进程传递。

6.1.4 位置参数

位置参数（Positional Parameters）是一种特殊的 Shell 变量，用于从命令行向 Shell 脚本传递参数，$1 表示第 1 个参数，$2 表示第 2 个参数等，$0 为脚本的名字，从${10}开始，参数号需要用花括号括起来，如${10}、${11}、${100}等。$*和$@一样，表示从$1 开始的全部参数。

下面举例说明位置参数的用法，新建一个名为 position 的脚本，内容如下：

```
#例 6-25: position 脚本说明$0,$1,$2,…,${10},$*
#!/bin/sh
echo "The script name is: $0"                       #$0 表示脚本本身
echo "Parameter #1: $1"
echo "Parameter #2: $2"
echo "Parameter #3: $3"
echo "Parameter #4: $4"
echo "Parameter #5: $5"
echo "Parameter #6: $6"
echo "Parameter #7: $7"
echo "Parameter #8: $8"
echo "Parameter #9: $9"
```


混混藏书阁 http://book-life.blog.163.com

```
echo "Parameter #10: ${10}"                    #用花括号括起来

echo "-------------------------"
echo "All the command line parameters are: $*"
#$*和$@一样，表示从$1 开始的全部参数
```

position 脚本接收命令行的 10 个参数，然后将$0,$1,$2,…,${10},$*回显出来。下面给出 position 脚本的执行结果，从中可以清晰地看出$0,$1,$2,…,${10},$*等符号的意义。

```
#例 6-25 position 脚本的执行结果
alloy@alloy:~/linuxshell/CH06$ chmod u+x position
alloy@alloy:~/linuxshell/CH06$ ./position a bat cat dive eager fair gate hi ideal java
The script name is:./position
Parameter #1:a
Parameter #2:bat
Parameter #3:cat
Parameter #4:dive
Parameter #5:eager
Parameter #6:fair
Parameter #7:gate
Parameter #8:hi
Parameter #9:ideal
Parameter #10:java
-----------------------------------------
All the command line parameters are: a bat cat dive eager fair gate hi ideal java
```

Shell 还定义了一些特殊的位置参数，表 6-3 列出了这些特殊位置参数及其意义。

表 6-3　特殊位置参数及其意义

特殊位置参数	意　义
$#	传递到脚本的参数数量
$*和$@	传递到脚本的所有参数
$$	脚本运行的进程号
$?	命令的退出状态，0 表示没有错误，非 0 表示有错误

将 position 脚本“echo "-----------------------"”以下的语句改成如下内容，以说明表 6-3 中特殊位置参数的意义。

```
echo "-------------------------"
echo "All the command line parameters are: $@"
echo "The number of command line parameters is: $#"
echo "The prcess ID is: $$"
echo "Did this script have any errors? $?"
```

下面显示 position 脚本的执行结果，请注意标注处的新加脚本的运行结果，可以看到$@与$*一样，是全部位置的内容；$#=10，是位置参数的个数；$$是执行该脚本所启动的进程号，此值并不固定；$?=0，表示脚本执行没有发生错误。

```
#例 6-25 position 新加脚本的执行结果
alloy@alloy:~/linuxshell/CH06$ ./position a bat cat dive eager fair gate hi ideal java
The script name is:./position
Parameter #1:a
Parameter #2:bat
Parameter #3:cat
Parameter #4:dive
```

```
Parameter #5:eager
Parameter #6:fair
Parameter #7:gate
Parameter #8:hi
Parameter #9:ideal
Parameter #10:java
-----------------------------------------
All the command line parameters are: a bat cat dive eager fair gate hi ideal java
The number of command line parameters is: 10          #下面是新加脚本的执行结果
The prcess ID is: 12779
Did this script have any errors? 0
alloy@alloy:~/linuxshell/CH06$
```

6.2 引用

尽管本书前面的内容几乎每章都涉及引用，但是，我们只是提醒读者需要用双引号、单引号等将其引起来，并没有解释其原因，本节详细介绍 Shell 脚本中引用的类型和用法。

引用指将字符串用引用符号引起来，以防止特殊字符被 Shell 脚本重解释为其他意义，特殊字符是指除了字面意思之外还可以解释为其他意思的字符，如$符号的字面意思就是美元符号，但是$还表示正则表达式中行尾，还可进行变量替换。首先请看下面的例 6-26，其中第 1 条命令没有引用，*被解释为特殊意义，因此，列出目录中以 a 开头的文件。第 2 条命令引用了 a*，*被解释为字面意义，由于文件中没有名为 a*的文件，因此，Shell 提示无此文件或目录。由此可以看出：引用是屏蔽特殊字符的特殊意义，而将其解释为字面意义。

```
#例 6-26：无引用和引用的区别
alloy@alloy:~/linuxshell/CH06$ ls a*                    #无引用，*被解释为通配符
append.sed  argv.awk  array.awk
You have new mail in /var/spool/mail/root
alloy@alloy:~/linuxshell/CH06$ ls "a*"                  #引用后，*被解释为字面含义
ls: 无法访问 a*: 没有那个文件或目录
alloy@alloy:~/linuxshell/CH06$
```

Shell 中定义了四种引用符号，如表 6-4 所示，6.2.1 节将讨论前三种引用符，6.2.2 节将单独讨论转义符。

表 6-4　引用符号、名称及其意义

符　号	名　称	意　义
""	双引号	引用除美元符号（$）、反引号（`）和反斜线（\）之外的所有字符
''	单引号	引用所有的字符
``	反引号	Shell 将反引号中的内容解释为系统命令
\	反斜线	转义符，屏蔽下一个字符的特殊意义

6.2.1 全引用和部分引用

双引号引用除美元符号（$）、反引号（`）和反斜线（\）之外的所有字符，即$、`和\在双引号中仍被解释为特殊意义。在双引号中保持$符号的特殊意义可以引用变量，如

混混藏书阁 http://book-life.blog.163.com

"$variable"表示以变量值替换变量名。利用双引号引用变量能够防止字符串分隔，保留变量中的空格。

下面的例 6-27 新建一个名为 double.sh 的脚本，内容如下：

```
#例 6-27：double.sh 脚本演示$variable 和"$variable"的区别
#!/bin/bash
variable1=2010
echo "$variable1"
echo $variable1

variable2="X    Y      Z"                    #字符之间用多个空格分开
echo "$variable2"
echo $variable2
```

double.sh 脚本定义两个变量：variable1 为数字 2010，variable2 为字符串，字符之间用多个空格分开，对这两个变量分别打印$variable 和“$variable”的值。下面给出 double.sh 脚本的执行结果，可以看到对于变量值是数字的情况，$variable1 和“$variable1”的值相同；对于由多个空格分开的字符串的情况，“$variable2”打印出了保留多个空格的字符串，而$variable2 打印出的字符间只用一个空格相分隔。

```
#例 6-27 double.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH06$ chmod u+x double.sh
alloy@alloy:~/linuxshell/CH06$ ./double.sh
2010
2010
X      Y      Z                              #"$variable2"保留了空格
X Y Z                                        #$variable2 字符间只有一个空格
alloy@alloy:~/linuxshell/CH06$
```

6.2.2 节的 weirdvars.sh 脚本和本章上机提议第 4 题将有助于读者更深入地理解双引号引用变量防止字符串分割、保留变量中的空格这一特性。

单引号引用了所有的字符，即单引号中字符除单引号本身之外都解释为字面意义，单引号不再具备引用变量的功能。因此，我们通常将单引号的引用方式称为全引用，将双引号的引用方式称为部分引用。

例 6-28 解释了双引号和单引号之间的区别，其中命令分别用双引号和单引号输出$PWD is the current directory.字符串，可以看到双引号将$PWD 解释为其值，而单引号原样输出$PWD。

```
#例 6-28：双引号和单引号的区别
alloy@alloy:~/linuxshell/CH06$ echo "$PWD is the current directory."
/home/globus is the current directory.                    #$PWD 被替换
alloy@alloy:~/linuxshell/CH06$ echo '$PWD is the current directory.'
$PWD is the current directory.                            #$PWD 保持不变
alloy@alloy:~/linuxshell/CH06$
```

单引号是不屏蔽单引号本身的，请看例 6-29 给出的名为 stephane.sh 的脚本：

```
#例 6-29：stephane.sh 脚本说明了单引号不屏蔽单引号本身
#!/bin/bash
echo "Why can't I write 's between single quotes"

echo 'Why can'"'"'t I write' "'"'s between single quotes'
#   |----------|  |----------|   |--------------------------|    三段单引号
```

stephane.sh 脚本需要输出 Why can't I write 's between single quotes 字符串，该字符串中包

含两个单引号，当然可以用双引号直接引起来将其输出，但是，当用单引号输出此行文本时，需要对此行文本中的两个单引号进行单独处理。因此，echo 需要分三段将此行文本输出，下面给出 stephane.sh 脚本的执行结果。

```
#例 6-29 stephane.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH06$ chmod u+x stephane.sh
alloy@alloy:~/linuxshell/CH06$ ./stephane.sh
Why can't I write 's between single quotes
Why can't I write 's between single quotes
alloy@alloy:~/linuxshell/CH06$
```

双引号和单引号是 Shell 编程中最常用的符号，读者需要清晰地理解双引号和单引号的异同点。

6.2.2 命令替换

命令替换是指将命令的标准输出作为值赋给某个变量，bash Shell 定义了两种语法进行命令替换，一种是使用反引号，另一种是利用$()，两种等价的语法格式如下：

```
`Linux 命令`
$( Linux 命令)
```

比如，pwd 是显示当前工作目录的命令，`pwd`和$(pwd)等价，值都为当前工作目录，与环境变量$PWD 的值一样。下面举几个例子说明命令替换的用法和特性。

首先，例 6-30 演示了反引号的基本用法，例中第 1 条命令 Shell 将 world 解释为系统命令，由于系统没有 world 命令，因此，Shell 返回此命令找不到的错误；第 2、3 条命令反引号中分别为 who 和 date 命令，这两条都是正确的系统命令。因此，输出上述两条系统命令的执行结果。

```
#例 6-30：反引号的基本用法
alloy@alloy:~/linuxshell/CH06$ echo `world`                    #第 1 条命令：调用 world 命令失败
-bash: world: command not found
alloy@alloy:~/linuxshell/CH06$ echo `who`                      #第 2 条命令：``符号中调用 who 命令
editor   tty1         2014-04-16 11 51 ( 0)
editor   pts/0        2014-04-16 11 51 ( 0.0)
root     pts/2        2014-04-23 12 52 (210.28.82.199)
alloy@alloy:~/linuxshell/CH06$ echo `date`                     #第 3 条命令：``符号中调用 date 命令
2014 年 04 月 27 日 星期二 14 18 28 CST
alloy@alloy:~/linuxshell/CH06$
```

其次，我们注意到使用反引号进行命令替换时，字符串分割的特性依然存在。为说明字符串分割，请看下面的例 6-31 给出的四条语句：

```
#例 6-31：反引号和引号的综合使用
(1) command `echo`
(2) command "`echo`"
(3) command `echo x y`
(4) command "`echo x y`"
```

四条语句的 command 泛指 Linux 中的任何命令。第（1）条语句 echo 空字符串，Shell 将 command 命令解析为不带任何参数，第（2）条语句将 echo 空字符串用双引号引起来，双引号有保留空格的作用，因此，Shell 将 command 命令解析为带了一个空字符串作为参数。第（3）条语句 echo 产生 x 和 y 字符，中间用空格分隔，Shell 将 command 命令解析为带了两个参数 x 和 y，此时 echo 产生的一个字符串就被解析为两个字符，这就造成了所谓的字符

混混藏书阁 http://book-life.blog.163.com

串分割。避免字符串分割只要用双引号将 echo 命令引起来，即如第（4）条语句所示，Shell 将 command 命令解析为带了一个参数“x y”。

如果深入分析 bash Shell 产生字符串分割的原因，是因为 bash Shell 的 IFS 值默认为空，即 bash Shell 默认为空格是一个参数的结束符。

当命令替换返回有多行结果时，如果不引用命令替换的结果，换行符也将被删除，下面的例 6-32 中第 1 条命令定义 dirlist 为 ls -l a*命令的执行结果，由多个文件信息行组成。第 2 条命令不引用 dirlist，结果显示 dirlist 中的换行符被删除。第 3 条命令引用 dirlist 时，结果保留了 dirlist 的换行符。

```
#例 6-32：命令替换删除换行符
alloy@alloy:~/linuxshell/CH06$  dirlist=`ls -l a*`    #第 1 条命令：命令替换
alloy@alloy:~/linuxshell/CH06$ echo $dirlist          #第 2 条命令：不引用 dirlist 变量，换行符被删除
-rw-r--r--. 1 root root 81 2012-10-22 append.sed -rw-r--r--. 1 root root 72 2012-10-15 argv.awk -rw-r--r--. 1 root root 79 2012-10-14 array.awk
alloy@alloy:~/linuxshell/CH06$ echo "$dirlist"        #第 3 条命令：引用 dirlist 变量，换行符保留
-rw-r--r--. 1 root root 81 2012-10-22 append.sed
-rw-r--r--. 1 root root 72 2012-10-15 argv.awk
-rw-r--r--. 1 root root 79 2012-10-14 array.awk
alloy@alloy:~/linuxshell/CH06$
```

尽管$()与反引号在命令替换上是等价的，但是，$()形式的命令替换是可以嵌套的。例 6-33 说明了$()的嵌套，例中的命令是计算 input 文件中第 1 行文本的长度，expr length 是计算字符串长度的命令，该命令所计算的字符串嵌套$()格式的命令替换，所替换的命令是一个 sed 命令。

```
#例 6-33：命令替换删除换行符
alloy@alloy:~/linuxshell/CH06$ firstlinelength=$(expr length "$(sed -n '1p' input)")
alloy@alloy:~/linuxshell/CH06$ echo $firstlinelength
35
alloy@alloy:~/linuxshell/CH06$
```

bash Shell 中，反引号与$()在处理双反斜线符号时存在区别，请看下面的例 6-34，反斜线符号是转义符，将在 6.2.3 节中介绍，在此仅说明反引号与$()在处理双反斜线符号时的区别，反引号将反斜线符号处理为空格，而$()符号将其处理为单斜线符。

```
#例 6-34：反引号与$()在处理双反斜线符号的区别
alloy@alloy:~/linuxshell/CH06$ echo \\
\                                          #输出转义符
alloy@alloy:~/linuxshell/CH06$ echo `echo \\`
                                           #输出空白行
alloy@alloy:~/linuxshell/CH06$ echo $(echo \\)
\                                          #输出单斜线符
alloy@alloy:~/linuxshell/CH06$
```

命令替换使 bash Shell 可以与其他编程语言编写的程序结合起来，运行如 C/C++、Java 语言编写的程序同样输出到 stdout 上，我们只要使用命令替换将输出保存到 Shell 变量，Shell 就可以对其他编程语言编写的程序所产生的输出进行任何处理。下面的例 6-35 在命令替换中调用 C 语言程序，例 6-35 首先新建一个 C 语言源代码文件，名字为 simpleC.c，内容如下：

```
/* simpleC.c：C 语言源代码文件，输出一行简单语句*/
#include <stdio.h>
```

```
main ()
{
  printf ("This is the output from C program.");
}
```

例 6-35 中第 1 条命令使用 gcc 编译 simpleC.c 文件，gcc 是 Linux 系统下的一种 C 语言编译器，它的用法不在本书讨论范围之内。在此，读者可做如下简单的理解：gcc 是编译命令，-o 选项后指定所生成可执行文件的名字，例 6-35 中为 simpleC。例 6-35 的第 2 条命令用反引号进行命令替换，将 simpleC 的执行结果赋给 testc 变量，最后，打印 testc 变量值，发现 testc 的值确实为 C 语言程序的输出。

```
#例 6-35: bash Shell 和 C 程序结合
#第 1 条命令：编译 C 语言源文件，形成可执行文件 simpleC
[root@zawu shell-program]# gcc -o simpleC simpleC.c
#第 2 条命令：将 simpleC 的执行结果赋给 testc 变量
[root@zawu shell-program]# testc=`./simpleC`
#第 3 条命令：输出 testc 变量
[root@zawu shell-program]# echo $testc
This is the output from C program.                    #来自 C 语言程序的输出结果
[root@zawu shell-program]#
```

例 6-35 提供了 bash Shell 与其他类型程序结合的模板，只要将其他类型程序封装成 Shell 的可执行文件，然后就可以方便地使用命令替换来调用这个可执行文件，而得到来自于其他类型程序的输出结果。

6.2.3 转义

反斜线符号（\）表示转义，当反斜线后面的一个字符具有特殊意义时，反斜线将屏蔽下一个字符的特殊意义，而以字面意义解析它。特殊字符既包括正则表达式中定义的元字符，也包括 Shell 重定向、管道命令中的一些特殊字符，表 6-5 总结了 Shell 中所有的特殊字符及其意义，即如果用 Shell 解析表 6-5 中的字符，必须使用转义符。

表 6-5 特殊字符及其意义

特殊字符	意　义
&	传递到脚本的参数数量
*	0 个或多个在*字符之前的那个普通字符
+	匹配 1 个或多个在其之前的那个普通字符
^	匹配行首，或后面字符的非
$	命令的退出状态，0 表示没有错误，非 0 表示有错误
`	反引号，Shell 引用符号
“	双引号，Shell 引用符号
|	管道符号或表示“或”意义
?	匹配 0 个或 1 个在其之前的那个普通字符
\	转义符

例 6-36 使用转义符将双引号和美元符号转义，例中第 1 条命令将双引号转义，Shell 按

混混藏书阁 http://book-life.blog.163.com

照字面含义解析双引号，结果中直接输出双引号；例中第 3 条命令在美元符号$前加上转义符，Shell 按字面含义解析$，所以$PWD 不再被解析为环境变量，而被当作普通字符串$PWD 直接输出。

```
#例 6-36：将双引号和美元符号转义
#第 1 条命令：将双引号转义
alloy@alloy:~/linuxshell/CH06$ echo "This is \" The 60th National Day\""
This is " The 60th National Day"
alloy@alloy:~/linuxshell/CH06$ echo "$PWD"
/home/globus
alloy@alloy:~/linuxshell/CH06$ echo "\$PWD"          #第 3 条命令：将$符号转义
$PWD
alloy@alloy:~/linuxshell/CH06$
```

对于*、^、?、+、&、`等符号，不再一一举例，其用法与双引号和$符号完全一致，但是当转义符后跟转义符本身时，用法略显特别，下面举例加以说明。例 6-37 在命令行中将转义符本身转义，例中第 1 条命令 echo 后加两个转义符，第 1 个转义符将第 2 个转义符转义，输出转义符本身；第 2 条命令 echo 后只有一个转义符，此时出现二级提示符。

```
#例 6-37：将转义符本身转义
alloy@alloy:~/linuxshell/CH06$ echo "\\"             #第 1 条命令：将转义符本身转义
\
alloy@alloy:~/linuxshell/CH06$ echo "\"              #第 2 条命令：出现二级提示符
>
```

假如将例 6-37 中的第 2 条命令写入一个脚本文件，结果是否与命令行调用它完全一样呢？新建名为 echoes.sh 的脚本，内容如下：

```
#echoes.sh 脚本：测试 echo "\"
#!/bin/bash
echo "\"
```

echoes.sh 脚本的执行结果如下，可以看到脚本执行发生错误，错误提示第 2 行发现未知结束符（EOF），第 3 行未知文件末端。

```
# echoes.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH06$ chmod u+x echoes.sh
alloy@alloy:~/linuxshell/CH06$ ./echoes.sh
./echoes.sh: line 2: unexpected EOF while looking for matching `"'
./echoes.sh: line 3: syntax error: unexpected end of file
alloy@alloy:~/linuxshell/CH06$
```

例 6-38 演示了将转义符赋给变量，从结果可以看出，仍然是出现二级提示符，变量的值为二级提示符下输入的字符串，读者可以上机测试将多个转义符赋给变量时的结果，如本章上机提议第 10 题。

```
#例 6-38：将转义符赋给变量
alloy@alloy:~/linuxshell/CH06$ variable=\            #转义符赋给 variable 变量
> National
alloy@alloy:~/linuxshell/CH06$ echo $variable
National
alloy@alloy:~/linuxshell/CH06$
```

由以上三例可以看出，转义符可以将其本身转义，但是，当 Shell 命令行或变量赋值遇到单个转义符时，将出现二级提示符，继续接收 Shell 输入，而当脚本中出现单个转义符时，将出错。因此，在 Shell 编程中，程序员需要了解脚本中每个转义符的作用，避免由单个转义符带来的错误。

下面的例 6-39 演示一个较为特别的变量赋值，新建一个名为 weirdvars.sh 的脚本，内容如下：

```
#例 6-39: weirdvars.sh 脚本演示怪异的变量赋值
#!/bin/bash

variable="()\\{}\$\""                    #将一串符号赋给变量
echo $variable
echo "$variable"

IFS='\'                                  #将域分隔符改为转义符
echo $variable
echo "$variable"

exit 0
```

下面给出例 6-39 中 weirdvars.sh 脚本的执行结果。

```
#例 6-39 weirdvars.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH06$ chmod u+x weirdvars.sh
alloy@alloy:~/linuxshell/CH06$ ./weirdvars.sh
()\{}$"
()\{}$"
() {}$"
()\{}$"
alloy@alloy:~/linuxshell/CH06$
```

weirdvars.sh 脚本将一串符号赋给变量 variable，其中包含四个转义符，第 1 个转义符将第 2 个转义符转义，第 3、4 个转义符分别将$和"符号转义，得出结果为()\{}$"，此时 echo "$variable"和 echo $variable 没有区别。然后，我们利用环境变量将 Shell 域分隔符改为转义符，echo $variable 得到的结果变为：() {}$"，\符号变为了空格，这是因为 Shell 根据域分隔符将 $variable 解析为两个域：()和{}$"，中间用空格分隔。而由于双引号具有防止变量分割的作用，因此，echo "$variable"的输出结果仍为为()\{}$"。

转义符除了屏蔽特殊字符的特殊意义之外，在 echo、sed 和 awk 等命令中，转义符加上一些字母能够表达特殊的含义，如\n 表示新的一行，表 6-6 归纳了转义符表示特殊意义的符号。

表 6-6　转义符后跟字母所表示的特殊意义

符　号	意　义
\n	新的一行
\r	返回
\t	表示 Tab 键
\v 或\f	换行但光标仍旧停留在原来的位置
\b	退格键（Backspace）
\a	发出警报声
\0xx	ASCII 码 0xx 所对应的字符

接着举一些例子说明表 6-6 列出的特殊字符的用法，首先看例 6-40，新建名为 escape.sh 的 Shell 脚本，内容如下：

```
#例 6-40: escape.sh 脚本演示 echo -e 和\符号
```


混混藏书阁 http://book-life.blog.163.com

```
#!/bin/bash

#echo 不加 e 选项，按照字面含义解释\t 等特殊符号
echo "\t\n\a\v"

# echo 加上 e 选项后，按照表 6-5 解释这些特殊符号
echo -e "\t\t\t\thello"
echo -e "hello\v\v\f\fhello"
echo -e "\a\a\a\a"
echo -e "\42"
```

下面给出例 6-40 中 escape.sh 脚本的执行结果：

```
#例 6-40 escape.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH06$ chmod u+x escape.sh
alloy@alloy:~/linuxshell/CH06$ ./escape.sh
\t\n\a\v
                                hello
hello




    hello
"
alloy@alloy:~/linuxshell/CH06$
```

escape.sh 脚本首先通过不带 e 选项的 echo 命令输出一些特殊字符，从结果可以看出，此时 Shell 按字面含义解释这些特殊字符，按原样输出\t\n\a\v 等字符。然后 escape.sh 脚本通过带 e 选项的 echo 命令输出这些字符，从结果可以看出，Shell 将这些字符解释为特殊的含义，\t 解释为 Tab 键，\v 和\f 解释为换行，但是光标水平位置保持不变，\a 表示发出警报声，\0xx 被解释为 ASCII 码 0xx 所对应的字符（042 是双引号的 ASCII 码）。

接着看例 6-41，我们用$"符号将特殊字符引起来，然后用 echo 命令打印，结果显示 Shell 将这些字符解释为特殊的含义。因此，如果使用了$"符号，就无须再使用-e 选项。

```
#例 6-41: $''符号与-e 选项等价
alloy@alloy:~/linuxshell/CH06$ echo $'\t\thello'          #$''与-e 选项等价
            hello
You have new mail in /var/spool/mail/root
alloy@alloy:~/linuxshell/CH06$ echo $'hello\b\101'
hellA
alloy@alloy:~/linuxshell/CH06$
```

$"符号还可以用于变量赋值，变量值以$"中符号的特殊意义进行替换，例 6-42 演示了此用法。

```
#例 6-42: 将特殊字符赋给变量的方法
alloy@alloy:~/linuxshell/CH06$ echo $'hello\b\101'        #$''用于变量赋值
hellA
alloy@alloy:~/linuxshell/CH06$ variable1=$'\042'
alloy@alloy:~/linuxshell/CH06$ echo $variable1
"
alloy@alloy:~/linuxshell/CH06$ variable2=$'\101\102\103'
alloy@alloy:~/linuxshell/CH06$ echo $variable2
ABC
alloy@alloy:~/linuxshell/CH06$
```

在此顺便完整介绍一下 echo 命令的用法，echo 命令在 Shell 编程中极为常用，echo 的功

能是在显示器上打印一段文字，起说明和提示作用，echo 命令的语法如下：

```
echo [选项] [字符串]
```

echo 的选项有两个：-e 和-n，-e 选项表示将转义符后跟字符形成的特殊字符解释成特殊意义，escape.sh 脚本足以解释这一点。-n 选项表示输出文字后不换行，如果不带-n 选项，echo 默认是输出文本后自动换行，例 6-43 说明了使用-n 选项前后的区别。

```
#例 6-43：echo 命令的-n 选项
alloy@alloy:~/linuxshell/CH06$ echo -n "The 60th National Day"  #使用-n 选项，不换行
The 60th National Dayalloy@alloy:~/linuxshell/CH06$ echo "The 60th National Day"
The 60th National Day                    #不使用-n 选项，换行
alloy@alloy:~/linuxshell/CH06$
```

6.3 本章小结

本章深入探讨了 Linux Shell 的变量和引用，首先从变量的替换和赋值的基本操作入手，讨论 Shell 脚本变量的无类型性，并对 Linux Shell 环境变量和位置参数两种特殊的变量进行了详尽分析。然后，介绍了 Linux Shell 中的四种引用符号及其意义和用法，重点讨论转义符的用法及其一些特殊的用法。由于变量和引用无处不在，因此，本章是 Shell 编程的基础章节，扎实地掌握本章内容是学习本书后续章节的基础。

6.4 上机提议

1．指出下列变量名是否有效，并上机测试：

```
FRUIT_BASKET
_APPLE_FIVE
FRUIT-BASKET
5_APPLE
APPLE_5
```

2．利用位置参数编写名为 test-arguments-num 的脚本，测试如下情况的参数个数和参数值：

```
variable=""          #空值
test-arguments-num $variable $variable $variable
test-arguments-num "$variable" "$variable" "$variable"
test-arguments-num "$variable $variable $variable"
```

3．6.1.4 节中 position 脚本$0 的结果为./position，选择正确的引用方式，改写 position 脚本，使执行结果中$0 的值为 position。（提示：Linux 系统命令 basename path 能够从路径中分离出文件名。）

4．上机运行 6.1.3 节中的 father.sh 和 child.sh 脚本，体会$$、$PPID 的意义，以及本地变量和环境变量的区别。

5．执行以下脚本，验证反引号的作用，以及$variable 和"$variable"的区别。

```
#!/bin/bash
variable1=`ls -l`
```


混混藏书阁 http://book-life.blog.163.com

```
echo $variable1
echo "$variable1"

variable2= "()\\{}\$\""
echo $variable2
echo "$variable2"

IFS='\'
echo $variable2
echo "$variable2"
```

6．改变 PS1 和 PS2 的值，验证表 6-2 所列出的特殊符号，理解这些特殊符号的意义。

7．利用.bash_profile 新建环境变量 CATALINA_HOME=/usr/local/ jakarta-tomcat-5.0.28，分别用./命令和 source 命令执行.bash_profile，然后在该 Shell 中 echo $CATALINA_HOME，查看是否有区别。

8．将$CATALINA_HOME/bin 目录添加到 PATH 环境变量中。

9．set 命令可以列出当前 Shell 的所有已设置的环境变量，请读者上机测试该变量，并注意观察结果中数组变量和普通变量有何区别。

10．编写脚本求 Fibonacci 数列的前 10 项，Fibonacci 数列有如下特点：第 1、2 个数为 1，从第 3 个数开始，该数是前面两个数之和。即

```
F1=1   (n=1)
F2=1   (n=1)
Fn= Fn-1+ Fn-2   (n≥3)
```

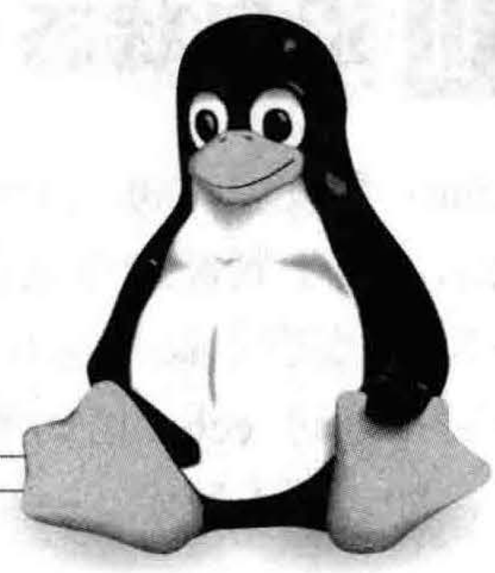
11．编写脚本打印包含*、^、?、+、&、`、$等符号的语句，使用转义符和单引号两种方法实现。

12．将例 6-33 命令中的双引号去掉，运行该命令，观察所出现的结果，并分析其原因。

13．上机验证将多个转义符赋给变量时，变量的值，如：variable=\，variable=\\，variable=\\\，variable=\\\\等。

14．上机验证如下命令的输出结果，并结合单引号、双引号、反引号和转义符四种引用符分析其原因。

```
echo \w
echo \\w
echo '\w'
echo '\\w'
echo "\w"
echo "\\w"

echo `echo \w`
echo `echo \\w`
echo `echo \\\w`
echo `echo \\\\w`
echo `echo \\\\\w`
echo `echo \\\\\\w`
echo `echo "\w"`
echo `echo "\\w"`
```



第7章

退出、测试、判断及操作符

在 Linux Shell 编程中，每个命令或脚本完成后都会有一个退出状态，而Shell中有一个内部命令test经常用于判断语句测试一种或几种状态的条件是否成立。因此，退出状态、测试和判断密切相关。本章从退出状态和测试入手，然后在此基础上重点讨论几种条件判断结构，最后讲解运算符和数字常量。其中的判断和测试是 Linux Shell 编程的基础部分，因此，熟练掌握判断和测试语句的使用是 Shell 编程的最基本要求。

混混藏书阁 http://book-life.blog.163.com

7.1 退出状态

在 Linux 系统中，每当命令执行完成后，系统都会返回一个退出状态。该退出状态用一整数值表示，用于判断命令运行正确与否。若退出状态值为 0，表示命令运行成功；若退出状态值不为 0，则表示命令运行失败。最后一次执行命令的退出状态值被保存在内置变量“$?”中，所以可以通过 echo 语句测试命令是否运行成功。POSIX 规定了几种退出状态和退出状态的含义，如表 7-1 所示。

表 7-1　退出状态及其含义

状　态　值	含　义
0	表示运行成功，程序执行未遇到任何问题
1～125	表示运行失败，脚本命令、系统命令错误或参数传递错误
126	找到了该命令但无法执行
127	未找到要运行的命令
>128	命令被系统强行结束

以下的例 7-1 演示了退出状态命令的用法，该例用 touch 命令创建一个文件，然后通过 echo 命令显示该命令执行后的退出状态，测试文件是否创建成功。

```
#例 7-1：用退出状态显示文件是否创建成功
alloy@alloy:~/linuxshell/CH07$ ls
exmaple1
alloy@alloy:~/linuxshell/CH07$ touch exit_exam1 #创建文件 exit_exam1
alloy@alloy:~/linuxshell/CH07$ ls
exit_exam1  exmaple1                            #使用 ls 命令查询文件 exit_exam1 创建成功
alloy@alloy:~/linuxshell/CH07$ echo $?
0                                               #退出状态为 0，表示文件创建成功
alloy@alloy:~/linuxshell/CH07$
```

在例 7-1 中，ls 命令用于显示当前目录下的所有文件，可以看到，使用 touch 命令前，文件 exit_exam1 还不存在，而使用 touch 命令后，文件 exit_exam1 创建成功，然后通过 echo 命令可以看出退出状态为 0，表示 touch 命令创建文件成功。

当然也有退出状态为非 0 的情况，下面的例 7-2 是一个退出状态为 2 和 127 的例子。要特别注意，虽然有些命令“失败”的退出状态在 1～125 之间有可能因为 Linux 的版本不同而有差异，但其他退出状态都有其特定的含义。

```
#例 7-2：退出状态为 2 和 127 的例子
alloy@alloy:~/linuxshell/CH07$ ls tt          #查询一个当前目录不存在的目录或文件
#由于当前目录下无 tt 这个文件或目录，所以 ls 无法访问 tt
alloy@alloy:~/linuxshell/CH07$ echo $?
2                                             #退出状态为 2，说明要查询的文件或目录不存在
alloy@alloy:~/linuxshell/CH07$ ddde  #输入一个 Shell 命令中不存在的命令
-bash: ddde: command not found
alloy@alloy:~/linuxshell/CH07$ echo $?
127                                           #退出状态为 127，说明未找到要运行的命令 ddde
```

```
alloy@alloy:~/linuxshell/CH07$
```

在例 7-2 中，首先使用 ls 查询一个当前目录不存在的文件或目录，显示的退出状态为 2，表示查询不到文件或目录；接着输入一个不存在的命令，显示的退出状态为 127，表示该命令未找到。

7.2 测试

Linux 的 Shell 命令中存在一组测试命令，该组命令用于测试某种条件或某几种条件是否真实存在。测试命令是判断语句和循环语句中条件测试的工具，所以该命令对编写 Shell 脚本是非常重要的。

7.2.1 测试结构

测试命令可用于测试表达式的条件的真假。如果测试的条件为真，则返回一个 0 值；如果测试的条件为假，将返回一个非 0 整数值。这一点和 C 语言的条件判断语句是有区别的，在 C 语言中，条件为真时返回的是一个非 0 正整数值，条件为假时返回一个 0 值，所以，若以前学习过 C 语言，要注意其区别，以免混淆。

测试命令有两种方式，一种是使用 test 命令进行测试，该命令的格式为：

```
test expression
```

其中，条件 expression 是一个表达式，该表达式可由数字、字符串、文本和文件属性的比较，同时可加入各种算术、字符串、文本等运算符。由于在 Linux Shell 编程中 test 命令使用很多，所以为了提高命令的可读性，可以使用另一种命令格式：

```
[ expression ]
```

其中，“[”是启动测试的命令，但要求在 expression 后有一个“]”与其配对。使用该命令时要特别注意“[”后和“]”前的空格是必不可少的。

第二种方式经常与 if、case、while 语句联用，作为流程控制语句的判断条件。

7.2.2 整数比较运算符

整数比较运算符是算术运算中很简单的一种，用于两个值的比较，测试其比较结果是否符合给定的条件。例如：a –eq b 用于整数比较运算，等于（-eq）是一个整数比较运算符，如果满足 a 等于 b，则该测试的结果为真（测试条件用 0 表示）；如果 a 不大于 b，则测试结果为假（测试条件用非 0 表示）。

test 测试数值时有一整套的整数比较运算符，其格式为：

```
test "num1" numeric_operator  "num2"
```

或者为：

```
["num1" numeric _operator "num2" ]
```

其中，numeric_operator 为整数比较运算符，用于比较数值的大小，但这些整数比较运算符不可用于字符串、文件操作，同样的字符串比较运算符和文件操作符也不可用于其他操作，如果误用，将产生不必要的错误，这一点在 Linux Shell 编程时要特别注意。表 7-2 是对各整

混混藏书阁 http://book-life.blog.163.com

数比较运算符的一个概括。

表 7-2　整数比较运算符

整数比较运算符	描　　述
num1 -eq num2	如果 num1 等于 num2，测试结果为 0
num1 -ge num2	如果 num1 大于或等于 num2，测试结果为 0
num1 -gt num2	如果 num1 大于 num2，测试结果为 0
num1 -le num2	如果 num1 小于或等于 num2，测试结果为 0
num1 -lt num2	如果 num1 小于 num2，测试结果为 0
num1 -ne num2	如果 num1 不等于 num2，测试结果为 0

下面的例 7-3 是一个整数变量与一个整数常量比较的例子，首先设置了变量 num1 值为 15，然后将变量 num1 使用运算符-eq 和-le 分别与 15、20 进行比较，最后通过 echo 命令返回的退出状态表示其测试结果。

```
#例 7-3：一个整数变量与一个整数常量比较的例子
alloy@alloy:~/linuxshell/CH07$ num1=15
alloy@alloy:~/linuxshell/CH07$ [ "$num1" -eq 15 ]    #测试 num1 是否等于 15
alloy@alloy:~/linuxshell/CH07$ echo $?
0                                #退出状态为 0，表示 num1 等于 15
alloy@alloy:~/linuxshell/CH07$ [ "$num1" -eq 20 ]    #测试 num1 是否等于 20
alloy@alloy:~/linuxshell/CH07$ echo $?
1                                #退出状态为 1，表示 num1 不等于 20
alloy@alloy:~/linuxshell/CH07$ [ "$num1" -lt 15 ]    #测试 num1 是否小于 15
alloy@alloy:~/linuxshell/CH07$ echo $?
1                                #退出状态为 1，表示 num1 不小于 15
alloy@alloy:~/linuxshell/CH07$ [ "$num1" -gt 15 ]    #测试 num1 是否大于 15
alloy@alloy:~/linuxshell/CH07$ echo $?
1                                #退出状态为 1，表示 num1 不大于 15
alloy@alloy:~/linuxshell/CH07$
```

上面的例子是一个变量与常量进行比较的例子，还可以同时对两个整数变量进行测试，下面的例 7-4 是两个整数变量比较的例子。

```
#例 7-4：两个整数变量比较的例子
alloy@alloy:~/linuxshell/CH07$first_num=99           #设置第一个变量为 99
alloy@alloy:~/linuxshell/CH07$ second_num=100        #设置第二个变量为 100
alloy@alloy:~/linuxshell/CH07$ [ "$first_num" -gt "$second_num" ]
alloy@alloy:~/linuxshell/CH07$ echo $?
1                                #退出状态为 1，说明变量 first_num 值不大于变量 second_num 值
alloy@alloy:~/linuxshell/CH07$ [ "$first_num" -eq "$second_num" ]
alloy@alloy:~/linuxshell/CH07$ echo $?
1                                #退出状态为 1，说明变量 first_num 值不等于变量 second_num 值
alloy@alloy:~/linuxshell/CH07$ [ "$first_num" -lt "$second_num" ]
alloy@alloy:~/linuxshell/CH07$ echo $?
0                                #退出状态为 0，说明变量 first_num 值小于变量 second_num 值
alloy@alloy:~/linuxshell/CH07$
```

在例 7-4 中，首先设置了两个变量的值分别为 99 和 100，然后通过大于、等于、小于运算符分别比较这两个变量的值的大小。

整数比较运算符不适用于浮点型数值的比较，这一点和 C 语言是有区别的。下面的例 7-5

列出了一个浮点型数值比较的例子，可以看出，在 bash 编程中只能对整数使用整数比较运算符，若想对浮点型数值进行比较，需使用特定的函数。

```
#例 7-5：将整数运算符用于浮点数运算
alloy@alloy:~/linuxshell/CH07$ [ 1.5 -lt 2.2 ]
-bash: [: 1.5: integer expression expected
alloy@alloy:~/linuxshell/CH07$
```

7.2.3 字符串运算符

同整数比较运算符一样，Linux Shell 中也存在一组字符串运算符，该组运算符可以用来测试字符串是否为空、两个字符串是否相等或者是否不相等。字符串运算符经常用于测试用户输入是否为空或比较字符串变量。一共有 5 种字符串运算符，表 7-3 列出了字符串运算符的各种测试方式和描述。

表 7-3 字符串运算符

字符串运算符	描　　述
string	测试字符串 string 是否不为空
-n string	测试字符串 string 是否不为空
-z string	测试字符串 string 是否为空
string1 = string2	测试字符串 string1 是否与字符串 string2 相同
string1 != string2	测试字符串 string1 是否与字符串 string2 不相同

对于第一种方式，用 string 进行测试时要特别注意该方式仅有一种格式：

```
test string
```

而不存在用“[”和“]”括起来的命令格式。进行字符串的比较时，建议字符串变量要使用双引号，即使变量为空，同样也要使用双引号。

下面的例 7-6 是一个运用字符串运算符的例子，在该例子中设置了一个字符串 str1 为空，然后分别用三种命令格式测试字符串 str1 是否为空。

```
#例 7-6：字符串比较运算的例子
alloy@alloy:~/linuxshell/CH07$ str1=""
alloy@alloy:~/linuxshell/CH07$ test "$str1"
alloy@alloy:~/linuxshell/CH07$ echo $?
1                                          #退出状态为 1，表示该字符串为空
alloy@alloy:~/linuxshell/CH07$ test -n "$str1"
alloy@alloy:~/linuxshell/CH07$ echo $?
1                                          #退出状态为 1，表示该字符串为空
alloy@alloy:~/linuxshell/CH07$ test -z "$str1"
alloy@alloy:~/linuxshell/CH07$ echo $?
0                                          #退出状态为 0，表示该字符串为空
alloy@alloy:~/linuxshell/CH07$
```

下面的例 7-7 是一个比较两个字符串是否相等的一个例子，其中设置两个字符串变量 type1 和 type2，然后比较两个变量的值是否相等或是否不相等。

```
#例 7-7：比较两字符串是否相等
alloy@alloy:~/linuxshell/CH07$ type1="vi"
alloy@alloy:~/linuxshell/CH07$ type2="vim"
```


混混藏书阁 http://book-life.blog.163.com

```
    alloy@alloy:~/linuxshell/CH07$ [ "$type1"  =  "$type2" ] #测试变量 type1 是否等于变量
type2
    alloy@alloy:~/linuxshell/CH07$ echo $?
    1                    #退出状态为 1，说明变量 type1 不等于变量 type2
    alloy@alloy:~/linuxshell/CH07$ [ "$type1" !=  "$type2" ] #测试 type1 是否不等于 type2
    alloy@alloy:~/linuxshell/CH07$ echo $?
    0                    #退出状态为 0，表示变量 type1 不等于变量 type2
    alloy@alloy:~/linuxshell/CH07$
```

在编写程序的过程中，要注意空格问题，下面的例 7-8 是多了一个空格后造成测试结果不相等的例子。

```
    #例 7-8：空格造成测试结果不相等的例子
    alloy@alloy:~/linuxshell/CH07$ str2="hello "          #赋值字符串变量 str2 为“hello ”
    alloy@alloy:~/linuxshell/CH07$ [ "$str2" = "hello" ] #测试变量 str2 是否与“hello”相等
    alloy@alloy:~/linuxshell/CH07$ echo $?
    1                    #退出状态为 1，说明变量 str2 不等于“hello”
    alloy@alloy:~/linuxshell/CH07$
```

在例 7-8 中，在给变量赋值的过程中加入了一个空格，通过退出状态的测试显示测试结果不相等，所以在以后的编程中要注意不要添加不必要的空格。在 Linux Shell 编程中是严格区分大小写的，下面的例 7-9 就说明了字符串“Hello”不等于字符串“hello”。

```
    #例 7-9：大小写区分的例子
    alloy@alloy:~/linuxshell/CH07$ str3="Hello"          #赋值字符串变量 str3 为“Hello”
    alloy@alloy:~/linuxshell/CH07$ [ "$str3" = "hello" ] #测试变量 str3 是否与“hello”相等
    alloy@alloy:~/linuxshell/CH07$ echo $?
    1                    #退出状态为 1，说明变量 str2 不等于“hello”
    alloy@alloy:~/linuxshell/CH07$
```

在 Linux Shell 中，由于字符串赋值和整数赋值没有区别，所以在比较整数时要注意不要把字符串比较运算符当作字符串使用。下面的例 7-10 说明了使用的运算符不同，赋值变量也不同，这是 Linux Shell 对变量的弱化造成的。

```
    #例 7-10：变量弱化造成的赋值结果的不同
    alloy@alloy:~/linuxshell/CH07$ var1="007"          #给变量赋值，可以当作整数，也可当作字符串
    alloy@alloy:~/linuxshell/CH07$ [ "$var1" = "7" ]     #测试变量 var1 的值是否等于字符串 7
    alloy@alloy:~/linuxshell/CH07$ echo $?
    1                    #退出状态为 1，表示变量 var1 的值不等于字符串 7
    alloy@alloy:~/linuxshell/CH07$ [ "$int1" -eq "7" ]   #测试变量 var1 的值是否等于整数 7
    alloy@alloy:~/linuxshell/CH07$ echo $?
    0                    #退出状态为 0，表示变量 var1 的值等于整数 7
    alloy@alloy:~/linuxshell/CH07$
```

7.2.4 文件操作符

同其他的编程语言一样，Linux Shell 提供了大量的文件操作符，这样可以完成测试文件的各种操作。其格式为：

```
    test  file_operator  File
```

或者为：

```
    [ file_ operator file ]
```

其中，file_operator 为文件操作符，file 为文件名、目录名或文件路径等。表 7-4 是几个比较常用的文件操作符。

表 7-4　文件操作符

文件运算符	描　　述
-d file	测试 file 是否为目录
-e file	测试 file 是否存在
-f file	测试 file 是否为普通文件
-r file	测试 file 是否是进程可读文件
-s file	测试 file 的长度是否不为 0
-w file	测试 file 是否是进程可写文件
-x file	测试 file 是否是进程可执行文件
-L file	测试 file 是否符号化链接

下面的例 7-11 是一个判断文件是目录还是文件的例子，该例中首先显示了当前目录下的所有文件，然后选取其中的一个判断其是文件还是目录，判断结果是一个文件。

```
#例 7-11：判断输入的文件是文件还是目录
alloy@alloy:~/linuxshell/CH07$ ls    #显示当前目录下的所有文件
exit_exam1  exmaple1  file_exam
alloy@alloy:~/linuxshell/CH07$ [ -d file_exam ]  #测试 file_exam 是否是目录
alloy@alloy:~/linuxshell/CH07$ echo $?
1                        #退出状态为 1，说明 file_exam 不是目录
alloy@alloy:~/linuxshell/CH07$ [ -f file_exam ]  #测试 file_exam 是否是文件
alloy@alloy:~/linuxshell/CH07$ echo $?
0                        #测试结果为 0，说明 file_exam 是文件
alloy@alloy:~/linuxshell/CH07$
```

下面再举一个判断文件是否存在的例子，例 7-12 首先使用 ls 命令列出当前目录下的所有文件，然后选取一个存在的文件和一个不存在的文件，分别进行测试。

```
#例 7-12：测试文件是否存在
alloy@alloy:~/linuxshell/CH07$ ls
exit_exam1  exmaple1  file_exam
alloy@alloy:~/linuxshell/CH07$ [ -e file_exam ]  #测试 file_exam 是否存在
alloy@alloy:~/linuxshell/CH07$ echo $?
0                        #退出状态为 0，表示文件 file-exam 存在
alloy@alloy:~/linuxshell/CH07$ [ -e file_exam1 ]  #测试文件 file_exam1 是否存在
alloy@alloy:~/linuxshell/CH07$ echo $?
1                        #退出状态为 1，表示文件 file_exam1 不存在
alloy@alloy:~/linuxshell/CH07$
```

在以后的 Linux Shell 编程过程中会用到很多这样的例子，如创建文件后测试一下该文件是否创建成功，当删除一个文件后，判断该文件是否删除成功，当然还有其他方面的应用，这里不一一列举。

下面的例 7-13 是一个测试文件权限的例子。这个例子首先列出了当前目录下的所有文件以及权限，然后用文件操作符中的权限测试符分别测试文件 file_exam2 是否可读、可写、可执行，通过测试可以看出，文件 file_exam2 可读、可写，但不能执行。

```
#例 7-13：用于测试文件权限的例子
alloy@alloy:~/linuxshell/CH07$ ls -l
总计 0
-rw-r--r--. 1 root root 0 12-23 12:25 exit_exam1
```


混混藏书阁 http://book-life.blog.163.com

```
-rw-r--r--. 1 root root 0 12-23 12:24 exmaple1
-rw-r--r--. 1 root root 0 12-23 15:06 file_exam
-rw-r--r--. 1 root root 0 12-23 16:08 file_exam1
-rw-r--r--. 1 root root 0 12-23 16:08 file_exam2
alloy@alloy:~/linuxshell/CH07$ [ -r file_exam2 ]      #测试文件是否可读
alloy@alloy:~/linuxshell/CH07$ echo $?
0
alloy@alloy:~/linuxshell/CH07$ [ -w file_exam2 ]      #测试文件是否可写
alloy@alloy:~/linuxshell/CH07$ echo $?
0
alloy@alloy:~/linuxshell/CH07$ [ -x file_exam2 ]      #测试文件是否执行
alloy@alloy:~/linuxshell/CH07$ echo $?
1
alloy@alloy:~/linuxshell/CH07$
```

文件操作符中的可读、可写、可执行的权限判断经常和 chmod 命令联用，该命令可用于对文件进行增加或减少权限。下面的例 7-14 是一个文件操作符与 chmod 命令联用的例子。通过 ls 命令可以看出 file_exam2 文件可读可写但不可执行，通过 chmod 命令给文件 file_exam2 增加可执行权限，然后通过测试可以看出文件可执行了。

```
#例 7-14：显示通过 chmod 命令增减文件权限的测试结果
alloy@alloy:~/linuxshell/CH07$ ls -l
总计 0
-rw-r--r--. 1 root root 0 12-23 12:25 exit_exam1
-rw-r--r--. 1 root root 0 12-23 12:24 exmaple1
-rw-r--r--. 1 root root 0 12-23 15:06 file_exam
-rw-r--r--. 1 root root 0 12-23 16:08 file_exam1
-rw-r--r--. 1 root root 0 12-23 16:08 file_exam2
alloy@alloy:~/linuxshell/CH07$ [ -x file_exam2 ]      #测试文件 file_exam2 是否可执行
alloy@alloy:~/linuxshell/CH07$ echo $?
1                     #退出状态为 1，说明文件 file_exam2 不可执行
alloy@alloy:~/linuxshell/CH07$ chmod a+x file_exam2   #给文件 file_exam2 增加可执行权限
alloy@alloy:~/linuxshell/CH07$ [ -x file_exam2 ]      #再次测试文件 file_exam2 是否可执行
alloy@alloy:~/linuxshell/CH07$ echo $?
0                     #退出状态为 0，表示文件 file_exam2 现在可执行了
alloy@alloy:~/linuxshell/CH07$
```

Linux Shell 编程中还有其他的测试操作符，和上面提到的整数比较运算符、字符串运算符和文件操作符组合使用，进而组合成复杂的测试用于判断或循环语句中。

7.2.5 逻辑运算符

逻辑运算符用于测试多个条件是否为真或为假，或使用逻辑非测试单个表达式，这些运算符在 Shell 编程中经常用到，这些条件一般与测试命令结合使用。这些运算符主要包括逻辑非、逻辑与、逻辑或运算符，具体描述如表 7-5 所示。

表 7-5 逻辑操作符

逻辑操作符	描　述
！expression	如果 expression 为假，则测试结果为真
expression1 –a expression2	如果 expression1 和 expression 同时为真，则测试结果为真
expression1 –o expression2	如果 expression1 和 expression2 中有一个为真，则测试条件为真

其中，expression 为一个表达式，该表达式描述了一个测试条件。在逻辑表达式的运算过程中并不是所有的运算符都会被执行，如：

```
expr1 -a expr2 -a expr3
```

只有当表达式 expr1 为真时，才会接着测试 expr2 的值为真，同样，只有在 expr1 和 expr2 为真时，才会接着测试 expr3 的值是否为真。对于逻辑或的运算过程中也不是所有的语句都会被执行，如：

```
expr1 -o expr2 -o expr3
```

只要 expr1 为真，就不用去测试表达式 expr2 和 expr3，只有 expr1 为假时，才去判断表达式 expr2 和 expr3，同样，只有 expr1 和 expr2 同时为假时，才去测试 expr3。

表 7-6 为逻辑运算的“真假表”，用它表示 expr1 和 expr2 的不同组合时，各逻辑运算所得到的值。

表 7-6　逻辑运算的真假表

expr1	expr2	! expr1	! expr2	expr1 –a expr2	expr1 –o expr2
真	真	假	假	真	真
真	假	假	真	假	真
假	真	真	假	假	真
假	假	真	真	假	假

下面的例 7-15 是一个逻辑非的例子，该例用到的是文件操作符中测试的文件是否存在的例子，该例子和文件操作符中测试文件是否存在的例子对比，可以看出，测试加上逻辑非操作符后的测试结果是反的。

```
#例 7-15：一个逻辑非的例子
alloy@alloy:~/linuxshell/CH07$ ls
exit_exam1  exmaple1  file_exam  file_exam2
alloy@alloy:~/linuxshell/CH07$ [ ! -e file_exam ]    #使用逻辑非测试一个存在的文件
alloy@alloy:~/linuxshell/CH07$ echo $?
1                    #退出状态为 1，说明文件 file_exam 不存在是假的
alloy@alloy:~/linuxshell/CH07$ [ ! -e file_exam1 ]   #使用逻辑非测试一个不存在的文件
alloy@alloy:~/linuxshell/CH07$ echo $?
0                    #退出状态为 0，说明文件 file_exam1 存在是假的
alloy@alloy:~/linuxshell/CH07$
```

再举一个逻辑与的例子，例 7-16 中同样用文件操作符中的文件是否存在的操作符和文件的权限操作符。本例首先测试文件 file_exam2 是否存在和是否可执行，由 ls 命令可以看出 file_exam2 存在而且可执行，所以，该例返回的命令为真，然后使用 chmod 命令将文件的可执行权限去除，通过测试可以看到退出状态为非 0，最后本例测试一个不存在的文件 file_exam3 是否存在和是否可读，返回结果为假。可以看出，对于逻辑与操作符，如果条件中有一个为假，则最终的结果就为假。

```
#例 7-16：一个逻辑与的例子
alloy@alloy:~/linuxshell/CH07$ ls -l
总计 0
-rw-r--r--. 1 root root 0 12-23 12:25 exit_exam1
-rw-r--r--. 1 root root 0 12-23 12:24 exmaple1
-rw-r--r--. 1 root root 0 12-23 15:06 file_exam
```


混混藏书阁 http://book-life.blog.163.com

```
-rwxr-xr-x. 1 root root 0 12-23 16:08 file_exam2  #可以看出文件 file_exam2 存在且可执行

#测试文件 file_exam2 是否存在且可执行
alloy@alloy:~/linuxshell/CH07$ [ -e file_exam2 -a -x file_exam2 ]
alloy@alloy:~/linuxshell/CH07$ echo $?
0                                    #退出状态为 0，说明文件 file_exam2 存在且可读
alloy@alloy:~/linuxshell/CH07$ chmod a-x file_exam2
                                     #将文件 file_exam2 的可执行权限去除
alloy@alloy:~/linuxshell/CH07$ ls -l
总计 0
-rw-r--r--. 1 root root 0 12-23 12:25 exit_exam1
-rw-r--r--. 1 root root 0 12-23 12:24 exmaple1
-rw-r--r--. 1 root root 0 12-23 15:06 file_exam
-rw-r--r--. 1 root root 0 12-23 16:08 file_exam2
                                   #可以看出现在文件 file_exam2 存在但不可执行

#再次测试文件 file_exam2 是否存在且可执行
alloy@alloy:~/linuxshell/CH07$ [ -e file_exam2 -a -x file_exam2 ]
alloy@alloy:~/linuxshell/CH07$ echo $?
1                                    #退出状态为 1，说明文件 file_exam2 不存在或不可执行
alloy@alloy:~/linuxshell/CH07$
```

下面的例 7-17 是一个逻辑或的例子，本例用于整数比较大小。首先初始化一个整数变量值为 15，然后测试该整数是否小于 20 或大于 30，测试结果表示该测试符合条件；然后测试该整数是否小于 5 或大于 10，测试结果表示该测试符合条件；最后测试该整数是否小于 10 或大于 20，测试结果表示该测试不符合条件。可以看出，对于逻辑与操作，只要有一个符合条件的表达式，整个测试条件就为真。

```
#例 7-17：一个逻辑或的例子
alloy@alloy:~/linuxshell/CH07$ integer1=15

#测试整数变量 integer1 是否小于 20 或大于 30
alloy@alloy:~/linuxshell/CH07$ [ "$integer1" -lt 20 -o "$integer1" -gt 30 ]
alloy@alloy:~/linuxshell/CH07$ echo $?
0                    #退出状态为 0，表示整数变量 integer 小于 20 或大于 30

#测试整数变量 integer1 是否小于 5 或大于 10
alloy@alloy:~/linuxshell/CH07$ [ "$integer1" -lt 5 -o "$integer1" -gt 10 ]
alloy@alloy:~/linuxshell/CH07$ echo $?
0                    #退出状态为 0，表示整数变量 integer1 小于 5 或大于 10

#测试整数变量 integer1 是否小于 10 或大于 20
alloy@alloy:~/linuxshell/CH07$ [ "$integer1" -lt 10 -o "$integer1" -gt 20 ]
alloy@alloy:~/linuxshell/CH07$ echo $?
1                    #测试状态为 1，表示整数变量 integer1 大于 10 且小于 20
alloy@alloy:~/linuxshell/CH07$
```

7.3 判断

所谓判断，就是对语句中不同的条件进行测试，进而根据不同的条件执行不同的语句。if、then、else 语句用于判断给定的条件是否满足，可用于判断整数比较大小、字符串操作、

文件操作以及逻辑运算等方面，并根据测试条件的真假来选择相应的操作。

if/else 结构仅仅用于两分支判断，但在实际问题中常常需要多分支的选择，所以需用到 if/else 语句嵌套、if/elif/else 和 case 多分支选择判断结构。

7.3.1 简单 if 结构

简单的 if 语句是条件语句的一种形式，它针对某种条件做出相应的处理。最简单的 if 结构如下：

```
if  expression
then
    command
    command
    …
fi
```

在使用这种简单的 if 结构时，要特别注意测试条件后如果没有“;”，则 then 语句要换行，否则会产生不必要的错误。如果 if 和 then 可以处于同一行，则必须用“;”来终止 if 语句，其格式为：

```
if expression; then
    command
    command
    …
fi
```

下面通过一个简单的例子来说明 if 的用法，例 7-18 中使用了整数比较运算符，新建一个 if_exam1.sh 的脚本，内容如下：

```
#例 7-18: if_exam1.sh 脚本判断输入的数是否小于 15
#!/bin/bash

#提示用户键盘输入一个整数,
echo "Please input a integer: "
read integer1

#判断输入的是否小于 15，小于 15 时将执行 then 语句
if [ "$integer1" -lt 15 ]
then echo "The integer which you input is lower than 15."
fi
```

例 7-18 中脚本 if_exam1.sh 的执行结果如下：

```
#例 7-18 if_exam1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH07$ ./if_exam1.sh
Please input a integer:
10
The integer which you input is lower than 15.
alloy@alloy:~/linuxshell/CH07$ ./if_exam1.sh
Please input a integer:
20
alloy@alloy:~/linuxshell/CH07$
```

脚本 if_exam1.sh 在执行时，首先输入一个小于 15 的值，显示 echo 语句“The integer which you input is lower than 15.”；当输入的整数值大于或等于 15 时，将不执行任何操作，并退出 if 结构。

混混藏书阁 http://book-life.blog.163.com

上面的例子用到了一个 if 语句，在实际编程的过程中可能用很多判断条件对同一个整数、字符串、文件等进行测试，接下来再举一个这样的例子，例 7-19 中用到了文件操作符，新建一个 if_exam2.sh 脚本，其脚本内容如下：

```
#例 7-19：if_exam2.sh 脚本用于创建一个文件，然后测试是否创建成功，并测试文件权限
#!/bin/bash

#创建一个文件 if_file1
touch if_file1

#判断文件 if_file1 是否创建成功
if [ -e if_file1 ]
then echo "The file if_file1 is created successfully!"
fi

#判断文件 if_file1 是否可读
if [ -r if_file1 ]
then echo "The file can read."
fi

#判断文件 if_file1 是否可写
if [ -w if_file1 ]
then echo "The file can write."
fi

#判断文件 if_file1 是否可执行
if [ -x if_file ]
then echo "The file can execute."
fi
```

例 7-19 中脚本 if_exam2.sh 的执行结果为：

```
#例 7-19 if_exam2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH07$ ./if_exam2.sh
The file if_file1 is created successfully!
The file can read.
The file can write.
alloy@alloy:~/linuxshell/CH07$
```

脚本 if_exam2.sh 首先用 touch 命令创建了一个文件 if_file1，然后使用简单的 if 结构判断文件 if_file1 是否创建成功，如果创建成功，将显示“The file if_file1 is created successfully!”，然后测试创建的文件的读、写、执行的权限。由脚本的执行结果可以看出，用 touch 命令创建文件成功，同时创建的文件 if_file1 可读写但不可执行。

7.3.2 exit 命令

由于判断语句和循环语句都涉及 exit 命令来控制程序和表达式的流程，所以，在本节中将介绍 exit 命令作为补充。现在有的编程语言中一般都会有一个 exit 的函数，在 Linux Shell 中也存在这样的内建命令。命令格式为：

```
exit status
```

其中，status 用 0～255 之间的数字表示，一般返回该状态值的同时伴随着脚本的退出，同退出状态一样，参数被保存在 Shell 变量$?中，可通过 echo 命令进行查询。要注意的是，不要在终端运行 exit 命令，否则将会导致系统重启。我们通过例 7-20 来说明 exit 命令的用法，

新建一个 exit_exam.sh 的脚本，该脚本的内容如下：

```
#例 7-20：exit_exam.sh 脚本用于判断用户输入的字符串是否为空
 #!/bin/bash

#提示用户输入一个字符串
echo "Please input a string: "
read str1

#判断输入的字符串是否为空，为空时执行 then 后面的命令
if [ -z "$str1" ]
then
    echo "What you input is null!"
    exit 1
fi
```

例 7-20 中脚本 exit_exam.sh 用于判断从键盘输入的字符串是否为空。下面是这个脚本的执行结果：

```
#例 7-20 exit_exam.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH07$ ./exit_exam.sh
Please input a string:
Hello World!
alloy@alloy:~/linuxshell/CH07$ echo $?
0
alloy@alloy:~/linuxshell/CH07$ ./exit_exam.sh
Please input a string:

What you input is null!
alloy@alloy:~/linuxshell/CH07$ echo $?
1
alloy@alloy:~/linuxshell/CH07$
```

脚本 exit_exam.sh 首先输入了一个字符串，可以看到退出状态为 0，表示脚本输入的字符串不为空，没有执行 then 中的语句，然后输入一个空字符串，返回“What you input is null!”，同时返回一个退出状态为 1，该退出状态值是在脚本中设置的，不是系统默认的退出状态值。所以在编写脚本时可以自己设置退出状态值，不过特定的值一般都有其特定的退出状态含义，不要乱用，以免执行脚本时产生误解。

7.3.3 if/else 结构

7.3.1 节讲解了简单的 if 结构，可以看出简单的 if 结构仅仅只能满足 if 后的表达式时才会执行 then 后面的命令，如果不满足 if 后的表达式，则任何命令都不执行，这种方式使得脚本的交互性很差，所以需要复杂一点的判断结构来满足脚本的交互性。

if/else 命令是双向选择语句，当用户执行脚本时，如果不满足 if 后的表达式，也会执行 else 后的命令，所以，有很好的交互性。该结果首先判断 if 后面的表达式，如果表达式的退出状态为 0，则执行 then 和 else 中间的语句，如果表达式的退出状态为非 0，则执行 else 和 fi 中的语句，then 和 else、else 和 fi 中间的语句可以是单个命令，也可是多个命令。其结构为：

```
if expression1
then
    command
    …
```


混混藏书阁 http://book-life.blog.163.com

```
    command
else
    command
    …
    command
fi
```

下面将通过例 7-21 说明 if/else 结构的用法，新建一个 ifelse_exam1 脚本，该脚本的内容如下：

```
#例 7-21：ifelse_exam1 脚本用于判断输入的文件名是否有对应的文件存在
#!/bin/bash

#if 语句用于判断输入的文件是否不存在，不存在时执行 then 和 else 间的命令
if [ ! -e "$1" ]
then
    echo "file $1 do not exist."
    exit 1

#输入的文件存在时，执行 else 和 fi 间的命令
else
    echo "file $1 exits."
fi
```

例 7-21 中脚本 ifelse_exam1.sh 的执行结果为：

```
#例 7-21 ifelse_exam1.sh 脚本的执行结果
#判断一个不存在的文件是否存在的情况
alloy@alloy:~/linuxshell/CH07$ ./ifelse_exam1.sh not_exit_file.sh
file not_exit_file.sh do not exist.
alloy@alloy:~/linuxshell/CH07$ echo $?
1
#判断一个存在的文件是否存在的情况
alloy@alloy:~/linuxshell/CH07$ ./ifelse_exam1.sh exit_exam.sh
file exit_exam.sh exits.
alloy@alloy:~/linuxshell/CH07$ echo $?
0
alloy@alloy:~/linuxshell/CH07$
```

脚本 ifelse_exam1.sh 首先用变量$1 来存储用户输入的字符串，该字符串用于表示文件名，然后通过文件操作符判断输入的文件名对应的文件是否不存在于当前的目录中，如果不存在，则输出当前文件不存在且返回一个退出状态值 1。如果当前文件中存在该文件，则显示该文件存在的提示信息。在运行该脚本时首先是输入一个当前目录下存在的文件，其执行的是 else 和 fi 之间的命令，然后再次执行该脚本，输入一个不存在的目录，然后进行判断，其执行的是 then 和 else 之间的语句，并返回一个退出状态值 1。

下面的例 7-22 是一个文件删除的例子，新建一个 ifelse_exam2.sh 脚本，该脚本的内容如下：

```
#例 7-22：ifelse_exam2.sh 脚本用于删除一个文件，并判断是否执行了该操作
#!/bin/bash

#提示用户输入要删除的文件
echo "Please input the file which you want to delete:"
read file

通过 if/else 结构判断文件是否被删除
```

```
if rm -f "$file"
then
   echo "Delete the file $file  sucessfully!"
else
   echo "Delete the file $file failed!"
fi
```

例 7-22 中脚本 ifelse_exam2.sh 的执行结果为：

```
#例 7-22 ifelse_exam2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH07$ touch test.txt          #创建文件 test.txt
alloy@alloy:~/linuxshell/CH07$ ls
exit_exam.sh    ifelse_exam1.sh     if_file1
ifelse_exam2.sh     if_exam1.sh   if_exam2.sh
test.txt
alloy@alloy:~/linuxshell/CH07$ chmod u+x ifelse_exam2.sh
alloy@alloy:~/linuxshell/CH07$ ./ifelse_exam2.sh
Please input the file which you want to delete:      #提示用户要删除的文件
test.txt
Delete the file test.txt  sucessfully!
alloy@alloy:~/linuxshell/CH07$ ls                    #文件删除成功
exit_exam.sh     ifelse_exam1.sh     if_file1
ifelse_exam2.sh    if_exam1.sh  if_exam2.sh
alloy@alloy:~/linuxshell/CH07$
```

ifelse_exam2.sh 脚本在执行前创建了一个测试文件 test.txt，在执行该脚本时，首先使用 ls 查询当前目录下的所有文件，然后执行脚本，输入要删除的当前目录中的文件 test.txt，可以看到文件已经成功删除的提示，然后再次使用 ls 命令查询当前的目录，文件已经成功地删除。

7.3.4 if/else 语句嵌套

if/else 结构只能判断两个条件，我们在实际的 Shell 编程过程中常常需要很多判断条件，如果需要同时判断三个或三个以上的条件，可以使用 if/else 语句嵌套，还可使用 if/elif/else 结构和 case 命令，本节主要讲解 if/else 语句嵌套，下面的两节将详细介绍 if/elif/else 结构和 case 命令。

if/else 语句嵌套可同时判断三个或三个以上的条件，其命令标准结构如下：

```
if expression1
then
    if expression2
    then
       command
       command
       …
    else
       command
       command
       …
    fi
else
    if  expression3
    then
       command
```


混混藏书阁 http://book-life.blog.163.com

```
        command
        …
    else
        command
        command
        …
    fi
fi
```

在写 if/else 结构嵌套时要注意 if 与 else 的配对关系，else 语句总是与它上面最近的未配对的 if 配对。在实际的 Shell 编程过程中，有时不需要在 if 和 else 后的条件语句中都嵌套一个 if/else 结构，应随着我们的编程思路来定。下面的例 7-23 是一个 if/else 结构的嵌套的例子，新建一个 ifelse_exam3.sh 脚本，该脚本的内容如下：

```
#例 7-23：ifelse_exam3.sh 脚本用于检查输入的字符串是否是一个当前目录的文件名
#!/bin/bash

 #测试用户输入是否为空，然后判断当前目录是否存在该文件
if [ "$1" ]
then
    echo "What you input is not null!"
    if [ -e "$1" ]
    then
        echo "The file $1 is existence !"
    else
        echo " The file $1 is not existence !"
    fi
else
    echo "what you input is null!"
fi
```

例 7-23 中脚本 ifelse_exam3.sh 首先提示用户输入一个文件名，然后判断输入的文件是否为空，不为空时执行 if 后面的 if/else 嵌套，否则将提示用户输入的文件名为空，该脚本的执行结果为：

```
#例 7-23 ifelse_exam3.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH07$ ls
exit_exam.sh    ifelse_exam1.sh  if_exam1.sh  if_exam2.sh
ifelse_exam2.sh  ifelse_exam3.sh  if_exam3.sh
alloy@alloy:~/linuxshell/CH07$ chmod u+x ifelse_exam3.sh
alloy@alloy:~/linuxshell/CH07$ ./ifelse_exam3.sh ifelse_exam2
What you input is not null!
The file ifelse_exam2 is existence !
alloy@alloy:~/linuxshell/CH07$ ./ifelse_exam3.sh
what you input is null!
alloy@alloy:~/linuxshell/CH07$
```

脚本 ifelse_exam3.sh 首先用 ls 命令显示了当前目录下所有的文件，然后首次执行脚本，并输入一个当前目录存在的文件名，可以看到提示用户输入不为空，而且找到相应的文件，接着再次执行该脚本，输入一个不存在的文件名或目录名，可以看到提示用户输入不为空，但输入的文件名对应的文件在当前目录查询不到；最后再次执行该脚本，但不输入任何字符串，可以看到提示用户输入为空。

当 if/else 结构超过两路判断时，需要在 else 后不断地嵌套 if/else 语句，这样可以实现多

路选择，这里举一个这样的例子，例 7-24 用于学生成绩分类，新建一个 ifelse_exam4.sh 脚本，其脚本内容如下：

```
#例 7-24：ifelse_exam4 脚本用于对学生成绩进行分类
#!/bin/bash

#提示用户输入分数（0～100）
echo "Please Input a integer(0-100): "
read score

#判断学生的分数类别
if [ "$score" -lt 0 -o "$score" -gt 100 ]
then
  echo "The score what you input is not integer or the score is not in (0-100)."
else
    if [ "$score" -ge 90 ]
    then
       echo "The grade is A!"
    else
        if [ "$score" -ge 80 ]
        then
           echo "The grade is B!"
        else
            if [ "$score" -ge 70 ]
           then
               echo "The grade is C!"
           else
               if [ "$score" -ge 60 ]
               then
                   echo "The grade is D!"
               else
                   echo "The grade is E!"
               fi
           fi
        fi
    fi
fi
```

例 7-24 中脚本 ifelse_exam4.sh 用于学生成绩分类，90～100 分为 A；80～89 分为 B；70～79 分为 C；60～69 分为 D，小于 60 分为 E。下面是该脚本的执行结果：

```
#例 7-24 ifelse_exam4 脚本的执行结果
alloy@alloy:~/linuxshell/CH07$ chmod u+x ifelse_exam4.sh
alloy@alloy:~/linuxshell/CH07$ ./ifelse_exam4.sh
Please Input a integer(0-100):
92
The grade is A!
alloy@alloy:~/linuxshell/CH07$ ./ifelse_exam4.sh
Please Input a integer(0-100):
78
The grade is C!
alloy@alloy:~/linuxshell/CH07$ ./ifelse_exam4.sh
Please Input a integer(0-100):
1000
The score what you input is not integer or the score is not in (0-100).
alloy@alloy:~/linuxshell/CH07$
```


混混藏书阁 http://book-life.blog.163.com

脚本 ifelse_exam4.sh 首先提示用户输入一个 0～100 之间的整数，当用户输入一个整数后将执行下面的操作，在执行该脚本时输入 92 分，可以看到学生成绩为 A，接着在输入了一个分数为 78 分，可以看到该学生的成绩为 C，然后输入了一个大于 100 的数，可以看到提示信息“The score what you input is not integer or the score is not in (0-100).”。

7.3.5 if/elif/else 结构

由脚本 ifelse_exam4.sh 可以看出，使用 if/else 嵌套在 Shell 编程的过程中很容易漏掉 then 或 fi 而产生错误，而且可读性很差，可以用 if/elif/else 结构或 case 结果代替。if/elif/else 结构针对某一事件的多种情况进行处理。通常表现为“如果满足某种条件，则进行某种处理，否则接着判断另一个条件，直到找到满足的条件，然后执行相应的处理”。其语法格式为：

```
if expression1
then
command
command
…
elif expression2
then
    command
    command
    …
elif expressionN
then
    command
    …
    command
else
   command
    …
   command
fi
```

可以看出，if/elif/else 的结构比 if/else 嵌套结构简单很多，且 fi 只出现一次，这样的结构可读性提高了很多。在该结构中，要特别注意表达式 expression1 到 expressionN 都必须为测试条件，且返回一个退出状态。在执行过程中，如果第一个表达式测试的退出状态为非 0，则尝试执行第二条测试条件；同样，第二条如果测试的退出状态为非 0，则尝试执行第三条测试条件，直到满足退出状态为 0，执行其 then 后的命令；如果所有的测试条件都失败，则执行 else 中的命令。如果 fi 命令后仍有其他命令，则执行 fi 后的命令。

例 7-25 是用这种结构对 ifelse_exam4.sh 脚本进行改写，新建一个名为 ifelifelse_exam1.sh 的脚本，其内容如下：

```
#例 7-25：ifelifelse_exam1.sh 脚本使用 if/elif/else 结构对脚本 ifelse_exam4 进行改写
#!/bin/bash

#提示用户输入分数（0～100）
echo "Please Input a integer(0-100): "
read score

#判断学生的分数类别
if [ "$score" -lt 0 -o "$score" -gt 100 ]
```

```
then
   echo "The score what you input is not integer or the score is not in (0-100)."
elif [ "$score" -ge 90 ]
then
   echo "The grade is A!"
elif [ "$score" -ge 80 ]
then
   echo "The grade is B!"
elif [ "$score" -ge 70 ]
then
   echo "The grade is C!"
elif [ "$score" -ge 60 ]
then
   echo "The grade is D!"
else
   echo "The grade is E!"
fi
```

下面是脚本 ifelifelse_exam1.sh 的执行结果，该脚本同样是首先提示用户输入一个 0～100 之间的整数，用户输入一个整数后将执行下面的判断，当用户输入的成绩为 73 分时，可以看到学生的成绩为 C，接着再输入一个分数为 56 分，可以看到该学生的成绩为 E，当输入一个大于 100 的整数时，可以看到提示信息“The score what you input is not integer or the score is not in (0-100).”。由此可以看出，该脚本的执行结果符合脚本 ifelse_exam4.sh 的执行结果。由于脚本中少了很多 if/else 嵌套和 fi 命令，所以使用 if/elif/else 结构能够使程序更简洁，在编写脚本的过程中出错的可能性降低了很多。

```
#例 7-25 ifelifelse_exam1.s h 脚本的执行结果
alloy@alloy:~/linuxshell/CH07$ ./ifelifelse_exam1.sh
Please Input a integer(0-100):
73
The grade is C!
alloy@alloy:~/linuxshell/CH07$ ./ifelifelse_exam1.sh
Please Input a integer(0-100):
56
The grade is E!
alloy@alloy:~/linuxshell/CH07$ ./ifelifelse_exam1.sh
Please Input a integer(0-100):
999
The score what you input is not integer or the score is not in (0-100).
alloy@alloy:~/linuxshell/CH07$
```

为了更好地理解 if/elif/else 结构，下面的例 7-26 是一个关于此结构的应用，用于判断用户输入的年份是否是闰年，闰年的判断条件是：

能被 4 整除，但不能被 100 整除的年份都是闰年。

能被 100 整除，但又能被 400 整除的年份是闰年。

新建一个脚本 ifelifelse_exam2.sh，其内容如下：

```
#例 7-26: ifelifelse_exam2.sh 脚本用于判断输入的年份是否是闰年
#!/bin/bash

#提示输入年份
echo "Please Input a year: "
read year
```


混混藏书阁 http://book-life.blog.163.com

```
#设置取余参数
let "n1=$year % 4"
let "n2=$year % 100"
let "n3=$year %400"

#判断输入的年份是否是闰年
if [ ! "$n1" -eq 0 ]
then
    leap=0
elif [ ! "$n2" -eq 0 ]
then
    leap=1
elif [ ! "$n3" -eq 0 ]
then
    leap=0
else
    leap=1
fi

#输出用户输入的年份是否是闰年
if [ "$leap" -eq 1 ]
then
    echo "$year is a leap year!"
else
    echo "$year is not a leap year!"
fi
```

例 7-26 中脚本 ifelifelse_exam2.sh 首先提示用户输入一个年份，然后通过 let 命令将年份与 4、100 和 400 的取余结果分别保存在 n1、n2 和 n3 中，其中 let 命令和取余操作符将在 7.4.1 节中讲解，这里不再详加说明。在判定用户输入的年份是否为闰年的过程中，首先判断用户输入的年份能否被 4 整除，不能被 4 整除时，即可马上判断该年份不是闰年；接着在能被 4 整除的条件下判断是否能被 100 整除，如果不能被 100 整除，可以判定该年份是闰年；最后，如果该年份能被 100 整除，则接着判断该年份能否被 400 整除，如果能被 400 整除，说明该年份是闰年，否则不是闰年。

在编写这个脚本的过程中用了一个标志位 leap，用于标记该年份是否是闰年，leap 为 1 表示该年份是闰年，leap 为 0 表示该年份不是闰年，脚本在最后用 if/then 结构判断该标志位的值，然后给出相应的提示信息。

下面是脚本 ifelifelse_exam2.sh 的执行结果，首先输入 1996，由于 1996 能被 4 整除，但不能被 100 整除，所以该年份是闰年；接着输入 2000，由于 2000 能被 100 整除且能被 400 整除，所以该年份是闰年；再输入 1900 年，由于 1900 能被 100 整除，但不能被 400 整除，所以该年份不是闰年；最后输入 1998 年，由于 1998 不能被 4 整除，可以立即判定该年份不是闰年。

```
#例 7-26 ifelifelse_exam2.sh 脚本的执行结果
#输入一个能被 4 整除，但不能被 100 整除的闰年年份
alloy@alloy:~/linuxshell/CH07$ ./ifelifelse_exam2.sh
Please Input a year:
1996
1996 is a leap year!
```

```
#输入一个能被 100 整除，但不能被 400 整除的非闰年年份
alloy@alloy:~/linuxshell/CH07$ ./ifelifelse_exam2.sh
Please Input a year:
1900
1900 is not a leap year!
#输入一个不能被 4 整除的年份
alloy@alloy:~/linuxshell/CH07$ ./ifelifelse_exam2.sh
Please Input a year:
1998
1998 is not a leap year!
alloy@alloy:~/linuxshell/CH07$
```

7.3.6 case 结构

和 if/elif/else 结构一样，case 结构同样可用于多分支选择语句，常用来根据表达式的值选择要执行的语句，该命令的一般格式为：

```
case variable  in
value1)
    command
    …
    command;;
value2)
    command
    …
    command;;
…
valueN)
     command
     …
     command;;
*)
      command
      …
      command;;
esac
```

case 结构的变量值 variable 与 value1、value2 等进行逐一比较，直到找到匹配的值，如果与其匹配，将执行其后的命令，遇到双分号则跳到 esac 后的语句执行，如果没有找到与变量值 variable 匹配的值，脚本将执行默认值“*)”后的命令，直至“;;”为止。

要注意的是，value1 与 value2 等的值必须是常量或正则表达式，所以，使用 case 命令有时无法对 if/else 语句嵌套和 if/elif/else 结构进行改写，只有当两个结构的判断条件匹配某常量或正则表达式时，才能使用 case 命令。在 Shell 编程过程中，对判断条件很少的一般可以使用 if 条件语句，但在多个条件判断且判断条件为非等于某个具体常量或正则表达式时，用 if/elif/else 结构；对于多个判断条件且判断条件是某变量匹配某常量或正则表达式时，可用 case 结构。

下面通过例 7-27 说明 case 命令的用法，新建一个脚本 case_exam1.sh，其脚本内容如下：

```
#例 7-27：case_exam1.sh 脚本提示输入一个数字（1～12），然后显示其对应月份的英文
#!/bin/bash

#提示用户输入月份（0～12）
```

```
echo "Please Input a month(0-12): "
read month

#判断数字对应的月份
case "$month" in
1)
    echo "The month is January!";;
2)
    echo "The month is February!";;
3)
    echo "The month is March!";;
4)
    echo "The month is April!";;
5)
    echo "The month is May!";;
6)
    echo "The month is June!";;
7)
    echo "The month is July!";;
8)
    echo "The month is August!";;
9)
    echo "The month is September!";;
10)
    echo "The month is October!";;
11)
    echo "The month is November!";;
12)
    echo "The month is December!";;
*)
    echo "The month is not in (0-12).";;
esac

#显示 case 脚本执行完成，开始执行 esac 后面的命令
echo "The 'case' command ends!"
```

case_exam1.sh 脚本用于将用户输入的数字月份转化为对应的英文月份。该脚本首先请求用户输入，并将输入结果保存到变量 month 中，case 命令对表达式$month 求值，然后通过变量的值对 1～12 进行匹配，如果匹配到 1～12 的值，将显示其结果对应的月份，对于 1～12 不能匹配的 case 表达式，将执行“*)”后面的命令，然后将执行 esac 后面的命令。

下面是脚本 case_exam1.sh 的执行结果，首先输入一个 1～12 之间的月份，结果显示 12 对应的英文，接着脚本执行了 esac 后的命令；然后输入一个不在 1～12 之间的数 15，显示的输出为“The month is not in (0-12). ”，接着执行 esac 后的命令。

```
#例 7-27 case_exam1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH07$ chmod a+x case_exam1.sh
alloy@alloy:~/linuxshell/CH07$ ./case_exam1.sh
Please Input a mouth(0-12):
12
The month is December!
The 'case' command ends
alloy@alloy:~/linuxshell/CH07$ ./case_exam1.sh
Please Input a mouth(0-12):
15
```

```
The month is not in (0-12).
The 'case' command ends
alloy@alloy:~/linuxshell/CH07$
```

下面的例 7-28 是一个通过 case 命令对 ifelse_exam4 脚本进行反过来改写的例子，当用户输入成绩分类 A～E 时，可以显示学生的分数段，如用户输入成绩“A”后，将显示其分数段在 90～100 分之间，输入 B、C、D、E 将显示其对应的分数段。新建一个脚本 case_exam2，其脚本内容如下：

```
#例 7-28: case_exam2.sh 脚本用于对输入一个分数类别（A～E）显示其对应的分数段
#!/bin/bash

#提示用户输入分数类别（A～E）
echo "Please Input a score_type(A-E): "
read score_type

#判断用户输入类别，并输出对应的分数段
case "$score_type" in
A)
    echo "The range of score is from 90 to 100 !";;
B)
    echo "The range of score is from 80 to 89 !";;
C)
    echo "The range of score is from 70 to 79 !";;
D)
    echo "The range of score is from 60 to 69 !";;
E)
    echo "The range of score is from 0 to 59 !";;
*)
    echo "What you input is wrong !";;
esac
```

下面是例 7-28 中脚本 case_exam2.sh 的执行结果，该脚本同样是首先提示用户输入一个 A～E 之间的大写字母，用户输入一个大写字母后将执行下面的操作，然后用户输入 E，可以看到该学生成绩分数段为 0～59，接着输入一个不在 A～E 之间的大写字母，显示出错信息“What you input is wrong !”。

```
#例 7-28 case_exam2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH07$ ./case_exam2.sh
Please Input a score_type(A-E):
E
The range of score is from 0 to 59 !
alloy@alloy:~/linuxshell/CH07$ ./case_exam2.sh
Please Input a score_type(A-E):
H
What you input is wrong !
alloy@alloy:~/linuxshell/CH07$
```

7.4 运算符

在 7.2 节中提到了整数比较运算符、字符串运算符、文件操作符和逻辑运算符，这里不再罗列这些运算符或操作符，本节主要详细讲解算术运算符、位运算符，以及自增自减运算

符和数字常量。

7.4.1 算术运算符

在 Linux Shell 中，算术运算符包括：+（加运算）、-（减运算）、*（乘运算）、/（除运算）、%（取余运算）、**（幂运算），这些算术运算符的举例及其结果如表 7-7 所示。

表 7-7 算术运算符

运 算 符	举 例	结 果
+（加运算）	3+5	8
-（减运算）	5-3	2
*（乘运算）	5*3	15
/（除运算）	8/3	2
%（取余运算）	15%4	3
**（幂运算）	5**3	125

在运算过程中，要特别注意在进行整数的除法运算时，其结果将舍弃整数部分，并且忽略四舍五入，最终结果为商的整数部分。下面的例 7-29 说明了这一点，通过 let 命令分别计算“8/5*5”与“8*5/5”的结果是不同的，前者的结果为 5，后者的结果为 8。

```
#例 7-29：一个关于整数运算符的例子
#先除后乘运算
alloy@alloy:~/linuxshell/CH07$ let "z=8/5*5"
alloy@alloy:~/linuxshell/CH07$ echo "z=$z"
z=5
#先乘后除运算
alloy@alloy:~/linuxshell/CH07$ let "z=8*5/5"
alloy@alloy:~/linuxshell/CH07$ echo "z=$z"
z=8
alloy@alloy:~/linuxshell/CH07$
```

下面再举一个取余和幂运算的例子，例 7-30 同样使用 let 命令来执行算术运算，然后通过 echo 命令将结果显示出来。在取余的操作中计算“19%5”的结果，在幂运算的例子中计算“5**3”的结果，前者的结果为 4，后者为 125。

```
#例 7-30：一个取余和幂运算的例子
#取余运算
alloy@alloy:~/linuxshell/CH07$ let "v=19%5"
alloy@alloy:~/linuxshell/CH07$ echo "v=$v"
v=4
#幂运算
alloy@alloy:~/linuxshell/CH07$ let "v=5**3"
alloy@alloy:~/linuxshell/CH07$ echo "v=$v"
v=125
alloy@alloy:~/linuxshell/CH07$
```

在除法和取余运算的过程中要注意，除数不能为 0，否则将显示如例 7-31 的除 0 的错误。

```
#例 7-31：除 0 错误
alloy@alloy:~/linuxshell/CH07$ let "v=5/0"
-bash: let: v=5/0: division by 0 (error token is "0")
alloy@alloy:~/linuxshell/CH07$ let "v=5%0"
```

```
-bash: let: v=5%0: division by 0 (error token is "0")
alloy@alloy:~/linuxshell/CH07$
```

使用算术运算符无法对字符串、文件的浮点型数进行计算，对于浮点型操作，需要用到专门的函数，这一点和 C 语言是不相同的，要特别注意。

算术运算符可与赋值运算符“=”结合使用，称为算术复合赋值运算符。表 7-8 列出了这些常用的复合赋值运算符及其用法。

表 7-8　算术复合赋值运算符

运　算　符	举　　例	等价表达式
+=	v+=5	v=v+5
-=	v-=10	v=v-10
*=	v*=5	v=v*5
/=	v/=3	v=v/3
%=	v%=5	v=v%5

下面是一个算术复合赋值运算符的例子，例 7-32 中首先设置变量 value 值为 5，然后通过赋值加运算加 6，结果为 11，最后在使用赋值除运算符将 value 的值除以 5，最终的 value 结果为 2。

```
#例 7-32：复合运算符的例子
alloy@alloy:~/linuxshell/CH07$ value=5
alloy@alloy:~/linuxshell/CH07$ let "value+=6"
alloy@alloy:~/linuxshell/CH07$ echo "value=$value"
value=11
alloy@alloy:~/linuxshell/CH07$ let "value/=5"
alloy@alloy:~/linuxshell/CH07$ echo "value=$value"
value=2
alloy@alloy:~/linuxshell/CH07$
```

7.4.2　位运算符

位运算符在 Shell 编程中很少使用，通常用于整数间的运算，位运算符是针对整数在内存中存储的二进制数据流中的位进行的操作。例如，表达式“2<<1”，表示将整数 2 在内存中的二进制数据流向左移动一位，相当于算术运算“乘以 2”的操作。表 7-9 列出了位运算符及其用法。

表 7-9　位运算符

运　算　符	举　　例	解释和 value 值
<<（左移）	value=4<<2	4 左移 2 位，value 值为 16
>>（右移）	value=8>>2	8 右移 2 位，value 值为 2
&（按位与）	value=8&4	8 按位与 4，value 值为 0
|（按位或）	value=8|4	8 按位或 4，value 值为 12
~（按位非）	value=~8	按位非 8，value 值为-9
^（按位异或）	Value=10^3	10 按位异或 3，value 值为 9

按位与运算，只有两个二进制数都为 1 时，结果才为 1；按位或运算，只要有一个二进

制位为 1，则结果为 1；按位异或运算，两个二进制位数相同（同时为 0 或 1）时，结果为 0，否则为 1；按位取反运算符是将二进制中的 0 修改为 1，1 修改为 0。位运算符在 C、C++或 Java 中会列出详细运算过程，这里不再列出其详细的计算过程。例 7-33 是一个位运算的例子，使用到了表 7-9 中列出的位运算符，可以看出，左移 2 位相当于乘以 4，而右移 2 位相当于除以 4，所以在算术运算过程中用到乘以或除以 2 的幂时，可用位运算来代替，因为位运算的效率要高于算术运算。

```
#例 7-33：位运算的例子
#左移运算
alloy@alloy:~/linuxshell/CH07$ let "value=10<<2"
alloy@alloy:~/linuxshell/CH07$ echo "value=$value"
value=40
#右移运算
alloy@alloy:~/linuxshell/CH07$ let "value=10>>2"
alloy@alloy:~/linuxshell/CH07$ echo "value=$value"
value=2
alloy@alloy:~/linuxshell/CH07$ let "value=10&2"
alloy@alloy:~/linuxshell/CH07$ echo "value=$value"
value=2
#按位或运算
alloy@alloy:~/linuxshell/CH07$ let "value=10|2"
alloy@alloy:~/linuxshell/CH07$ echo "value=$value"
value=10
#按位取反运算
alloy@alloy:~/linuxshell/CH07$ let "value=~10"
alloy@alloy:~/linuxshell/CH07$ echo "value=$value"
value=-11
#按位异或运算
alloy@alloy:~/linuxshell/CH07$ let "value=10^2"
alloy@alloy:~/linuxshell/CH07$ echo "value=$value"
value=8
alloy@alloy:~/linuxshell/CH07$
```

同算术运算符一样，位运算符同样可以与赋值运算符“=”联用，组成复合赋值运算符。表 7-10 列出了这些复合运算符及其用法。

表 7-10　复合运算符

运　算　符	举　　例	等价表达式
<<=	value<<=2	value=value<<2
>>=	value>>=2	value=value>>2
&=	value&=4	value=value&4
|=	value|=4	value=value|4
^=	Value^=3	value=value^3

要注意的是，按位非运算符是一元运算符，无法和赋值运算符组成复合运算符。下面的例 7-34 是一个使用位运算的符号赋值运算符的例子，首先设置一个 value 值，然后对其进行复合赋值运算。

```
#例 7-34：复合位运算的例子
alloy@alloy:~/linuxshell/CH07$ value=10
```

```
alloy@alloy:~/linuxshell/CH07$ let "value<<=2"
alloy@alloy:~/linuxshell/CH07$ echo "value=$value"
value=40
alloy@alloy:~/linuxshell/CH07$ let "value>>=2"
alloy@alloy:~/linuxshell/CH07$ echo "value=$value"
value=10
alloy@alloy:~/linuxshell/CH07$ let "value&=4"
alloy@alloy:~/linuxshell/CH07$ echo "value=$value"
value=0
alloy@alloy:~/linuxshell/CH07$
```

7.4.3 自增自减运算符

与其他编程语言一样，Linux Shell 中也提供了自增自减运算符，其作用是自动将变量加 1 或减 1。自增自减操作符主要包括前置自增（++variable）、前置自减（--variable）、后置自增（variable++）和后置自减（variable--）。前置操作首先改变变量的值（++用于给变量加 1，--用于给变量减 1），然后将改变的变量值交给表达式使用；后置变量则是在表达式使用后再改变变量的值。

要特别注意：自增自减操作符的操作元只能是变量，不能是常数或表达式，且该变量值必须为整数型，例如：++1、（num+2）++都是不合法的。下面的例 7-35 是一个自增自减的例子，新建脚本 increment_and_decrement_exam1，脚本内容如下：

```
#例 7-35: increment_and_decrement_exam1.sh 脚本实现变量的自增自减
#!/bin/bash

#变量赋初值
num1=5

#使用前置自增操作
let "a=5+(++num1) "
echo "a=$a"

#变量赋初值
num2=5

#使用后置自增操作
let  "b=5+(num2++) "
echo "b=$b"
```

在脚本 increment_and_decrement_exam1.sh 中使用了前置自增和后置自增运算符，用于对这两种自增方式进行比较。下面是该脚本的执行结果，可以看到前置自增运算的结果为 11，后置自增运算的结果为 10。

```
#例 7-35 increment_and_decrement_exam1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH07$ ./increament_and_decrement_exam1.sh
a=11
b=10
alloy@alloy:~/linuxshell/CH07$
```

7.4.4 数字常量

Linux Shell 脚本或命令默认将数字以十进制的方式进行处理，如果要使用其他进制的方

式进行处理，则需对这个数字进行特定的标记或加前缀。当使用 0 作为前缀时，表示八进制；当使用 0x 进行标记时，表示十六进制，同时还可使用 num#这种形式标记进制数。

下面的例 7-36 是一个使用数字常量的例子，新建一个脚本 constant_exam1.sh，其脚本内容如下：

```
#例 7-36：constant_exam1.sh 脚本实现数字常量
#!/bin/bash

#默认进制表示方式：十进制
let "num1=40"
echo "num1=$num1"

#八进制表示方式：以“0”作为前缀
let "num2=040"
echo "num2=$num2"

#十六进制表示方式：以“0x”作为前缀
let "num3=0x40"
echo "num3=$num3"
```

在脚本 constant_exam1.sh 中，分别列出了十进制数、八进制数和十六进制数的表示方式，同时设置了 40 这个数字常量，可以看出这三种进制最后产生的十进制结果是不同的。下面给出了该脚本的执行结果。

```
#例 7-36 constant_exam1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH07$ ./constant_exam1.sh
num1=40
num2=32
num3=64
alloy@alloy:~/linuxshell/CH07$
```

在 Linux Shell 编程过程中，还可使用 num#表示不同的进制。下面的例 7-37 就是一个使用 num#的例子，新建一个脚本 constant_exam2.sh，其脚本内容如下：

```
#例 7-37：constant_exam1.sh 脚本使用 num#实现进制表示
#!/bin/bash

#二进制表示方式：以“2#”表示
let "num1=2#1101100110001101"
echo "num1=$num1"

#八进制表示方式：以“8#”表示
let "num2=8#50"
echo "num2=$num2"

#十六进制表示方式：以“16#”表示
let "num3=16#50"
echo "num3=$num3"
```

在脚本 constant_exam1.sh 中，分别列出了二进制数、八进制数和十六进制数的表示方式，下面给出了该脚本的执行结果。

```
#例 7-37 constant_exam1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH07$ ./constant_exam2.sh
num1=55693
num2=40
num3=80
```

```
alloy@alloy:~/linuxshell/CH07$
```

对于二进制和八进制来说，要注意在设置数的时候二进制都是由 0 和 1 表示的，八进制由 0～7 表示，如果超出这个数值范围，就会显示出错信息，示例如下：

```
#二进制超出数值范围的错误
alloy@alloy:~/linuxshell/CH07$ let "2#23"
-bash: let: 2#23: value too great for base (error token is "2#23")
alloy@alloy:~/linuxshell/CH07$
```

7.5 本章小结

本章深入探讨了 Linux Shell 的退出状态、测试、判断和运算符。首先从退出状态入手，讨论退出状态的含义，接着介绍了测试的结构，并对整数比较运算符、字符串运算符、文件操作符和逻辑运算符做了详尽分析，然后介绍了 Linux Shell 中的判断结构，重点讨论各种判断结构的用法，最后介绍了算术运算符、位运算符、自增自减运算符和数字常量的用法。操作符或运算符、测试和判断是 Shell 编程的入门部分，因此，本章是 Shell 编程的基础章节，扎实地掌握本章内容是学习本书后续章节的基础。

7.6 上机提议

1. 首先使用一个正确的 Shell 命令，用变量$?测试其退出状态，然后使用一个错误的 Shell 命令，用变量$?测试其退出状态。

2. 初始化两个变量值，然后利用整数比较运算符比较这两个变量的大小，并用 echo 变量测试其退出状态。

3. 初始化一个字符串，然后测试文件是否为空，接着再初始化一个字符串，比较这两个字符串是否相等。

4. 创建一个文件或目录，然后测试其是文件还是目录，如果创建的是文件，则测试该文件是否可读、可写、可执行，并测试其退出状态。（提示：使用 touch 创建文件，使用 mkdir 创建目录。）

5. 上机运行 7.2.5 节中关于逻辑非、逻辑与和逻辑或的例子，体会一下逻辑与和逻辑或的区别。

6. 初始化一个字符串，然后通过简单的 if 结构测试该字符串是否为空，如果为空，则用 echo 命令输出“What you input is null!”；如果不为空，则用 echo 命令输出“What you input is not null!”，同时分析一下两种测试字符串为空的命令的区别与联系。

7. 在习题 6 的基础上，如果字符串为空，使用 exit 命令设置一个退出状态，然后在脚本执行的过程中使用 echo 命令测试程序中设置的退出状态是否为预先设置的退出状态。

8. 创建一个目录，然后用 if/else 测试文件是否创建成功，如果创建成功，使用 echo 命令返回“The directory is created Successfully!”；如果不成功，则使用 echo 命令返回“The

directory is not created successfully!”。

9．设置一个 0～6 的数字，然后通过 if/else 嵌套测试输入的数字，最后显示其对应的星期数：

如果输入的数字为 0，则用 echo 命令返回“Sunday”；
如果输入的数字是 1，则用 echo 命令返回“Monday”；
如果输入的数字是 2，则用 echo 命令返回“Tuesday”；
如果输入的数字是 3，则用 echo 命令返回“Wednesday”；
如果输入的数字是 4，则用 echo 命令返回“Thursday”；
如果输入的数字是 5，则用 echo 命令返回“Friday”；
如果输入的数字是 6，则用 echo 命令返回“Saturday”。

10．使用 if/elif/else 结构改写习题 9。

11．使用 case 结构改写习题 9，并比较 if/else 嵌套结构、if/elif/else 结构和 case 结构的区别与联系。

12．使用 let 命令计算 12*5/5 与 12/5*5 的值，然后查看结果是否相同，如果不相同，请说明结果不相同的原因。

13．通过 Shell 编程计算下列位运算的值：

（1）20<<2
（2）20>>2
（3）20&2
（4）20|2
（5）~20
（6）20^2

14．上机运行 7.4.3 节中的脚本 increment_and_decrement_exam1，比较前置自增和后置自增的区别。

第 8 章

循环与结构化命令

在 Linux Shell 的编程过程中，有时需要反复执行某一条命令或某一组命令，这时要用到循环结构化命令。循环命令用于特定条件下决定某些语句重复执行的控制方式，它具有封闭型的单入/单出性质，也就是说，进入循环结构后，只要循环条件未达到结束状态，就始终执行循环体内的操作。在 Shell 中提供了三种常用的循环语句，分别是 for 循环语句、while 循环语句和 until 循环语句。本章将详细介绍这三种循环结构，并在此基础上讨论循环控制符 break 和 continue 的用法。

8.1 for 循环

for 循环是 Linux Shell 中最常用的结构，for 循环有三种结构：第一种结构是列表 for 循环；第二种结构是不带列表 for 循环；第三种结构是类 C 风格的 for 循环。这三种结构将在下面进行详细介绍。

8.1.1 列表 for 循环

列表 for 循环语句用于将一组命令执行已知的次数，下面给出了 for 循环语句的基本格式：

```
for variable in {list}
do
   command
   command
   …
done
```

其中，do 和 done 之间的命令称为循环体，执行次数和 list 列表中常数或字符串的个数相同。当执行 for 循环时，首先将 in 后 list 列表的第一个常数或字符串赋值给循环变量，然后执行循环体；接着将 list 列表中的第二个常数或字符串赋值给循环变量，再次执行循环体。这个过程将一直持续到 list 列表中无其他的常数或字符串，然后执行 done 命令后的命令序列。

下面的例 8-1 演示了列表 for 循环中 list 列表是常数的情况，新建一个 for_exam1.sh 脚本，该脚本的内容如下：

```
#例 8-1: for_exam1.sh 脚本演示利用 for 循环显示 5 次欢迎操作
#!/bin/bash

#使用列表 for 循环显示 5 次欢迎操作
for varible1 in 1 2 3 4 5
do
    echo "Hello, Welcome $varible1 times "
done
```

脚本 for_exam1.sh 这类的循环经常用于计数，范围被限定在 1～5 之间。下面是这个脚本的执行结果，由于 in 后面的列表列出了 5 个参数，可以看出该脚本执行了 5 次欢迎操作。

```
#例 8-1 for_exam1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./for_exam1.sh
Hello, Welcome 1 times
Hello, Welcome 2 times
Hello, Welcome 3 times
Hello, Welcome 4 times
Hello, Welcome 5 times
alloy@alloy:~/linuxshell/CH08$
```

Linux Shell 中支持列表 for 循环中使用略写的计数方式，脚本 for_exam2.sh 就是一个这样的例子，该脚本将 1～5 的范围用{1..5}表示。

```
#例 8-2：for_exam2.sh 脚本利用 for 循环显示 5 次欢迎操作
#!/bin/bash
```

```
#使用列表 for 循环略写方式显示显示 5 次欢迎操作
for varible1 in {1..5}
do
    echo "Hello, Welcome $varible1 times "
done
```

下面是脚本 for_exam2.sh 的执行结果，可以看出，和脚本 for_exam1.sh 执行结果相同。

```
#例 8-2 for_exam2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./for_exam2.sh
Hello, Welcome 1 times
Hello, Welcome 2 times
Hello, Welcome 3 times
Hello, Welcome 4 times
Hello, Welcome 5 times
alloy@alloy:~/linuxshell/CH08$
```

Linux Shell 中还支持按规定的步数进行跳跃的方式实现列表 for 循环，例 8-3 中的脚本 for_exam3.sh 用于计算 1～100 内所有奇数之和，下面是其脚本内容：

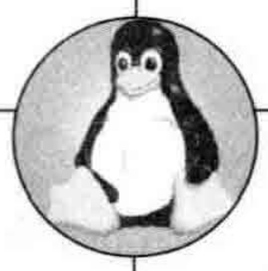
```
#例 8-3: for_exam3.sh 脚本使用列表 for 循环计算 1～100 内所有的奇数之和
#!/bin/bash

#对 sum 赋初值
sum=0

#使用列表 for 循环计算 1～100 内所有的奇数之和，并将值保存在 sum 中
for i in {1..100..2}
do
    let "sum+=i"
done

#输出 sum 值
echo "sum=$sum"
```

脚本 for_exam3.sh 首先给 sum 赋初值，通过 i 的按步数 2 不断递增最终计算出 sum 的值，脚本的执行结果如下：

```
#例 8-3 for_exam3.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./for_exam3.sh
sum=2500
alloy@alloy:~/linuxshell/CH08$
```

脚本 for_exam3.sh 中通过{1..100..2}实现 1～100 内的整数按步数 2 进行跳跃，同样可以通过 seq 命令实现按 2 递增来计算 1～100 内的所有奇数之和，新建脚本 for_exam4.sh，脚本内容如下：

```
#例 8-4: for_exam4.sh 脚本使用列表 for 循环和 seq 命令计算 1～100 内所有的奇数之和
#!/bin/bash

#对 sum 赋初值
sum=0

#将 seq 命令用于 for 循环中
for i in $( seq 1 2 100 )
do
    let "sum+=i"
done
```




```
#输出 sum 值
echo "sum=$sum"
```

seq 命令是 Linux 预设的外部命令，一般用作一堆数字的简化写法。脚本 for_exam4.sh 中的命令：

```
( seq 1 2 100 )
```

有三个常数，其中，1 表示起始数，2 表示跳跃的步数，100 表示结束条件值。脚本执行结果如下：

```
#例 8-4 for_exam4.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./for_exam4.sh
sum=2500
alloy@alloy:~/linuxshell/CH08$
```

上面的例子是对数字进行的操作，同样，列表 for 循环可以对字符串进行操作。下面的例 8-5 中脚本 for_exam5.sh 实现周一到周日的英文显示。

```
#例 8-5: for_exam5.sh 脚本显示周一到周日对应的英文翻译
#!/bin/bash

#使用列表 for 循环显示周一到周日对应的英文
for day in Monday Tuesday Wednesday Thursday Friday Saturday Sunday
do
    echo "$day"
done
```

脚本 for_exam5.sh 在 list 列表中列出了周一到周日对应的英语翻译，然后通过列表 for 循环将其显示给用户，脚本执行结果如下：

```
#例 8-5 for_exam5.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./for_exam5.sh
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday
alloy@alloy:~/linuxshell/CH08$
```

可以通过 for 循环显示当前目录下所有的文件，例 8-6 的脚本 for_exam6.sh 通过 ls 命令和列表 for 循环实现当前目录下所有文件的显示。

```
#例 8-6: for_exam6.sh 脚本通过 ls 命令和列表 for 循环显示当前目录下的文件
#!/bin/bash

#ls 显示当前目录的所有文件，以这些文件作为 list 列表，然后显示 ls 罗列出的文件
for file in $( ls )
do
   echo "file: $i"
done
```

脚本 for_exam6.sh 中，通过命令 ls 显示当前目录下的所有文件，然后通过不断地循环赋值给 file 将其对应的文件名显示给用户，脚本执行结果如下：

```
#例 8-6 for_exam6.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./for_exam6.sh
file: for_exam1
file: for_exam2
```

```
file: for_exam3
file: for_exam4
file: for_exam5
file: for_exam6
alloy@alloy:~/linuxshell/CH08$
```

上面的脚本使用了 ls 来显示当前目录的所有文件，同样可以通过通配符（*）产生文件名扩展，其中通配符（*）可以匹配当前目录下的所有文件，脚本 for_exam7.sh 就是一个这样的例子，其脚本内容如下：

```
#例 8-7：for_exam7.sh 脚本通过通配符（*）和列表 for 循环显示当前目录下的文件
#!/bin/bash

#通配符（*）显示当前目录下的所有文件，以这些文件作为 list 列表
 for file in $( * )
do
   echo "file: $i"
done
```

同脚本 for_exam6.sh 的执行结果一样，脚本 for_exam7.sh 的执行结果如下：

```
#例 8-7 for_exam7.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./for_exam7.sh
file: for_exam1
file: for_exam2
file: for_exam3
file: for_exam4
file: for_exam5
file: for_exam6
file: for_exam7
alloy@alloy:~/linuxshell/CH08$
```

列表 for 循环可以实现通过命令行来传递脚本中的 for 循环列表参数，脚本 for_exam8.sh 就是这样的例子，新建脚本 for_exam8.sh，内容如下：

```
#例 8-8：for_exam8.sh 脚本演示列表 for 循环实现通过命令行来传递脚本中 for 循环列表参数
#!/bin/bash

#提示用户输入参数个数
echo "number of arguments is $#"

#提示用户输入内容
echo "What you input is: "

#通过命令行传递脚本 for 循环列表参数
for argument in "$*"
do
   echo "$argument"
done
```

脚本 for_exam8 在执行过程中，首先将显示用户输入参数的个数，然后输出用户输入的命令行参数内容，下面是脚本的执行结果。

```
#例 8-8 for_exam8.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./for_exam8.sh  1 2 3
number of arguments is 3
What you input is:
1 2 3
alloy@alloy:~/linuxshell/CH08$ ./for_exam8.sh  Hello World !
```

```
number of arguments is 3
What you input is:
Hello World !
alloy@alloy:~/linuxshell/CH08$ ./for_exam8.sh  Hello World!
number of arguments is 2
What you input is:
Hello World!
alloy@alloy:~/linuxshell/CH08$
```

可以看出，参数列表可以是数字，也可以是字符串，但输入是以空格进行分隔的，如果存在空格，脚本执行时会认为存在另一个参数。脚本第一次执行时，输入的数字是 1、2 和 3，输出结果为 3 个参数，并输出了这 3 个数字；脚本第二次执行时，在“world”和“!”之间加了一个空格，可以看到输出时的参数个数为 3 个；第三次执行该脚本时，在“world”和“!”之间无空格，可以看到提示输入的参数个数为 2 个。所以，列表 for 循环在传递参数时要注意参数间的空格。

8.1.2 不带列表 for 循环

不带列表的 for 循环执行时，由用户指定参数和参数的个数。下面给出了不带列表的 for 循环的基本格式：

```
for variable
do
   command
   command
   …
done
```

其中，do 和 done 之间的命令称为循环体，Shell 会自动将命令行键入的所有参数依次组织成列表，每次将一个命令行键入的参数显示给用户，直至所有的命令行中的参数都显示给用户。这种结构的 for 循环和下面带列表的 for 循环的结构功能完全一致。

```
for variable in "$@"
do
   command
   command
   …
done
```

不带列表的 for 循环同样通过命令行来传递参数列表，脚本 for_exam9.sh 利用不带列表 for 循环改写脚本 for_exam8.sh，下面是脚本内容。

```
#例 8-9：for_exam9.sh 脚本演示一个不带参数列表 for 循环的例子
#!/bin/bash

#提示用户输入参数个数
echo "number of arguments is $#"

#提示用户输入内容
echo "What you input is: "

#通过命令行来传递脚本 for 循环列表参数
for argument
do
   echo "$argument"
```

```
done
```

脚本 for_exam9.sh 的执行结果与脚本 for_exam8.sh 的执行结果相同，下面是脚本的执行结果。

```
#例 8-9 for_exam9.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./for_exam9.sh 1 2 3
number of arguments is 3
What you input is:
1
2
3
alloy@alloy:~/linuxshell/CH08$ ./for_exam9.sh Hello World !
number of arguments is 3
What you input is:
Hello
World
!
alloy@alloy:~/linuxshell/CH08$ ./for_exam9.sh Hello World!
number of arguments is 2
What you input is:
Hello
World!
alloy@alloy:~/linuxshell/CH08$
```

可以看出，不带列表的 for 循环结构比列表 for 循环结构简洁易读，但其只可从命令行来传递参数，所以这种方式在 Linux Shell 编程中使用相对较少，一般只限于命令行传递参数。

8.1.3 类 C 风格的 for 循环

类 C 风格的 for 循环也可被称为计次循环，一般用于循环次数已知的情况。下面给出了类 C 风格的 for 循环的语法格式：

```
for(( expr1; expr2; expr3 ))
do
    command
    command
    …
done
```

其中，表达式 expr1 为循环变量赋初值的语句；表达式 expr2 决定是否进行循环的表达式，当判断 expr2 退出状态为 0 时，执行 do 和 done 之间的循环体，当退出状态为非 0 时，将退出 for 循环执行 done 后的命令；表达式 expr3 用于改变循环变量的语句。类 C 风格的 for 循环结构中，循环体也是一个块语句，要么是单条命令，要么是多条命令，但必须包裹在 do 和 done 之间。

下面的类 C 风格的 for 循环语句用于输出前 5 个正整数，新建脚本 for_exam10.sh，下面是脚本内容。

```
#例 8-10: for_exam10.sh 脚本使用类 C 风格的 for 循环实现输出前 5 个正整数
#!/bin/bash

#使用类 C 风格的 for 循环输出 1～5
for(( integer = 1; integer <= 5; integer++ ))
do
    echo "$integer"
```

```
done
```

脚本 for_exam10.sh 的执行结果如下：

```
#例 8-10 for_exam10.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./for_exam10.sh
1
2
3
4
5
alloy@alloy:~/linuxshell/CH08$
```

例 8-10 中的脚本 for_exam10.sh 在 for 循环中首先声明了循环变量 integer，并赋值为 1，之所以称 integer 为循环变量，是因为 integer 用于控制循环执行的次数和结束条件，接着判断 integer 是否小于或等于 5，若 integer 小于或等于 5 成立，则执行循环体 do 和 done 之间的命令，而后执行修正表达式“integer++”，将 integer 的值加 1，再次判断 integer 小于或等于 5 是否成立，以此类推，直至循环结束。

使用类 C 风格的 for 循环要注意以下事项。

- 如果循环条件最初的退出状态为非 0，则不会执行循环体。
- 当执行更新语句时，如果循环条件的退出状态永远为 0，则 for 循环将永远执行下去，从而产生所谓的死循环。
- Linux Shell 中不运行使用非整数类型的数作为循环变量。
- 如果循环体中的循环条件被忽略，则默认的退出状态为 0。
- 在类 C 风格的 for 循环中，可以将三个语句全部忽略，下面是合法的 for 循环：

```
for(( ; ;))
do
   echo "Hello World! "
done
```

可以使用类 C 风格的 for 循环对脚本 for_exam3.sh 进行改写，新建脚本 for_exam11.sh，下面是脚本内容。

```
#例 8-11：for_exam11.sh 脚本使用类 C 风格的 for 循环计算 1～100 内所有的奇数之和
#!/bin/bash

#对 sum 赋初值
sum=0

#使用类 C 风格的 for 循环计算 1～100 内所有的奇数之和，并将值保存在变量 sum 中
for(( i = 1; i <= 100; i = i + 2 ))
do
   let "sum += i"
done

#输出 sum 值
echo "sum=$sum"
```

脚本 for_exam11.sh 同样是首先给 sum 赋初值，然后通过 for 循环不断地对 sum 与循环变量 i 进行叠加，最终计算出 sum 的值，脚本的运行结果如下：

```
#例 8-11 for_exam11.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./for_exam11.sh
sum=2500
```

```
alloy@alloy:~/linuxshell/CH08$
```

脚本 for_exam11.sh 的循环条件是 i<=100，初始条件 i=1，因此，i=i+2 将步长设定为 2，每执行一次循环体，就将 i 的值增加 2，最终产生的结果 sum 为 2500。

我们可以使用逗号运算符同时对两个变量进行操作，新建脚本 for_exam12.sh，下面是脚本内容。

```
#例 8-12：for_exam12.sh 脚本使用逗号运算符对两个变量进行操作
#!/bin/bash

#对循环条件设置条件限制变量
LIMIT=5

#初始化两变量，并操作 for 循环执行两个变量相减
for(( a=1, b=5; a <= LIMIT; a++, b-- ))
do
    let "temp=a-b"
    echo "$a-$b=$temp"
done
```

脚本 for_exam12.sh 的执行结果如下：

```
#例 8-12 for_exam12.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./for_exam12.sh
1-5=-4
2-4=-2
3-3=0
4-2=2
5-1=4
alloy@alloy:~/linuxshell/CH08$
```

在脚本中同时初始化 a 和 b 的变量值，通过 a<=LIMIT 来设置循环条件，然后通过 a++ 设置 a 加 1 操作、b--设置 b 减 1 操作，每执行一次 for 循环，都判断 a 是否小于或等于 LIMIT，该循环将不断地执行，直至 a>5。

在使用类 C 风格的 for 循环时，要保证 for 循环可以正常结束，也就是必须保证循环条件的结果存在退出状态为非 0 的情况，否则将无休止地执行下去，从而产生死循环。比如，例 8-13 中脚本 for_exam13.sh 就会产生死循环。

```
#例 8-13：for_exam13.sh 脚本演示产生死循环的例子
#!/bin/bash

#i 初始值为 1，但设置循环条件为 i>=1，会产生死循环
for(( i=1; i>=1; i++))
do
    echo "Hello World! "
done
```

脚本 for_exam13.sh 的执行结果如下：

```
#例 8-13 for_exam13.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./for_exam13.sh
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
```

```
./for_exam13: line 6: echo: write error: 被中断的系统调用
alloy@alloy:~/linuxshell/CH08$
```

可以看出，“Hello World！”将一直执行下去，需要按“Ctrl+C”组合键将该脚本强行终止，否则将无限循环下去，造成死循环的原因是 i 将永远大于或等于 1。

类 C 风格的 for 循环中设置了三个表达式，尤其是修正表达式，它不断提醒开发人员对循环变量进行修正，这样可以更好地设计循环，但该种循环也有其缺陷，就是对字符串或者文件操作时存在困难。

8.2 while 循环

while 循环语句也称前测试循环语句，它的循环重复执行次数，是利用一个条件来控制是否继续重复执行这个语句。while 语句与 for 循环语句相比，无论是语法还是执行的流程，都比较简明易懂。while 循环格式如下：

```
while expression
do
    command
    command
    …
done
```

while 循环语句之所以命名为前测试循环，是因为它要先判断此循环的条件是否成立，然后才进行重复执行的操作。也就是说，while 循环语句执行的过程是：先判断 expression 的退出状态，如果退出状态为 0，则执行循环体，并且在执行完循环体后，进行下一次循环，否则退出循环执行 done 后的命令。为了避免死循环，必须保证在循环体中包含循环出口条件，即存在 expression 的退出状态为非 0 的情况。

while 循环可以分为四种情形，下面分别加以讨论。

8.2.1 计数器控制的 while 循环

假定该种情形是已经准确知道要输入的数据或字符串的数目，在这种情况下可采用计数器控制的 while 循环结构来处理。这种情形和类 C 风格的 for 循环类似，由于指定了循环的次数 *N*，可初始化计数器值，通过计数器与 *N* 进行比较，如果小于或等于 *N*，则执行循环体，该循环体将反复执行，直到计数器中的值大于 *N* 为止。这种情形下，while 循环的格式如下：

```
counter = 1
while expression
do
    command
    …
    let command to operate counter
    command
    …
done
```

下面是一个名为 while_exam1.sh 的脚本，该脚本使用计数器控制的 while 循环，用于显示 1～5。

```
#例 8-14: while_exam1.sh 脚本演示一个 while 的例子，用于显示 1～5
#!/bin/bash

#初始化循环变量 int
int=1

#while 设置循环条件，在循环体中设置命令和循环增量
while(( "$int" <= 5 ))
do
    echo "$int"
    let "int++"
done
```

脚本 while_exam1 的执行结果如下：

```
#例 8-14 while_exam1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./while_exam1.sh
1
2
3
4
5
alloy@alloy:~/linuxshell/CH08$
```

脚本 while_exam1.sh 首先初始化变量 int 为 1，循环条件是测试 int 的值是否小于或等于 5，如果小于或等于 5（即测试条件为真），将进行循环体，在循环体中设置了 int 值加 1 操作，然后再次判断 int 值是否小于或等于 5，这样不断地循环下去，直至 int 大于 5。

可以使用计数器控制的 while 循环对脚本 for_exam3.sh 进行改写，新建脚本 while_exam2.sh，下面是其脚本内容。

```
#例 8-15: while_exam2.sh 脚本使用计数器控制的 while 循环计算 1～100 内所有的奇数之和
#!/bin/bash

#对 sum 赋初值
sum=0

#对计数器 i 赋初值
i=1

#使用计数器控制的 while 循环计算 1～100 内所有的奇数之和，并将值保存在 sum 中
while(( i <= 100 ))
do
    let "sum+=i"
    let "i += 2"   #设置循环计数器使 i 加 2
done

#输出 sum 值
echo "sum=$sum"
```

脚本 while_exam2 的执行结果如下：

```
#例 8-15 while_exam2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./while_exam2.sh
sum=2500
alloy@alloy:~/linuxshell/CH08$
```

可以看出，while_exam2.sh 实现了计算 1～100 内的所有奇数之和，本脚本首先初始化 sum 值为 0，然后初始化计数器 i 值为 1，接着通过不断测试循环条件 i 是否小于或等于 100，如

果小于或等于 100，将执行循环体，在循环条件中设置了计数器加 2 操作，这样通过不断地测试循环条件得到最终的执行结果为 2500。

8.2.2 结束标记控制的 while 循环

在 Linux Shell 编程中，很多时候不知道读入数据的个数，但是可以设置一个特殊的数据值来结束 while 循环，该特殊的数据值称为结束标记，其通过提示用户输入特殊字符或数字来操作。当用户输入该标记后结束 while 循环，执行 done 后的命令。在该种情形下，while 循环的形式如下：

```
read variable
while [[ "$variable" ! = sentinel ]]
do
   read variable
done
```

举一个例子来说明结束标记控制的 while 循环的用法，新建脚本 while_exam3.sh，脚本内容如下：

```
#例 8-16: while_exam3.sh 脚本演示使用结束标记控制的 while 循环实现猜 1～10 内的数
#!/bin/bash

#提示用户输入 1～10 内的整数
echo "Please input the num(1-10) "
read num

#while 循环实现猜数游戏
while [[ "$num" != 4 ]]
do
   if [ "$num" -lt 4 ]
   then
       echo "Too small. Try again!"
       read num
   elif [ "$num" -gt 4 ]
   then
        echo "To high. Try again"
        read num
   else
      exit 0
    fi
done

#提示用户猜对了
echo "Congratulation, you are right! "
```

脚本 while_exam3.sh 的执行结果如下：

```
#例 8-16 while_exam3.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./while_exam3.sh
Please input the num(1-10)
5
Too high. Try again!
3
Too small. Try again!
4
congratulation, you are right!
```

```
alloy@alloy:~/linuxshell/CH08$
```

脚本 while_exam3.sh 是一个猜数游戏的程序，一般是设定一个范围，设定了一个结束标记，然后通过不断地提示用户输入的数是大于还是小于要猜的数，让用户不断调整输入的数值，最终猜到正确的数。该脚本中是一个 1～10 范围内的猜数游戏，在脚本执行的过程中，首先提示用户输入 1～10 内的数，而结束标记是 4。如果用户输入的数字是 4，则结束 while 循环，执行 done 后的命令，提示用户猜正确了，显示语句“Congratulation, you are right!”；如果用户输入的数小于 4，则提示用户输入的数小了，显示语句“Too small. Try again!”，然后让用户重新输入一个数；如果用户输入的数大于 4，则提示用户输入的数大了，显示语句“Too high. Try again!”，然后让用户重新输入一个数，通过这样不断地循环，用户就能猜到正确的答案。当然，这种猜数游戏有一定的技巧，如果使用穷举法，最好的情况是 1 次猜中，最差的情况要猜 10 次，如果设置的数值很大，这种猜数方法有时会使用户失去猜数的兴趣。可以通过折半查询法进行猜数，通过不断地折半查找可以很快猜到正确的数，在 1～10 之间猜数使用折半查找时，最好的情况为 1 次猜中，最差的情况是 4 次猜中，效率明显提高了很多。

可以通过用户指定循环变量的值，并设定好结束标记来实现 while 循环，新建脚本 while_exam4.sh，下面是脚本的内容。

```
#例 8-17：while_exam4.sh 演示使用结束标记控制的 while 循环实现阶乘的操作
#!/bin/bash

#提示用户输入所要实现阶乘的数值，并将其值赋给变量 num
echo "Please input the num "
read num

#初始化阶乘结束结果值为 1
factorial=1

#通过结束标记控制的 while 循环实现 num 的阶乘
while [ "$num" -gt 0 ]
do
    let "factorial= factorial*num"
    num--
done

#显示 num 的阶乘的值
echo "The factorial is $factorial"
```

脚本 while_exam4.sh 的执行结果为：

```
#例 8-17 while_exam4.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./while_exam4.sh
Please input the num
5
The factorial is 120
alloy@alloy:~/linuxshell/CH08$
```

脚本 while_exam4.sh 首先提示用户输入所要求阶乘的数，然后设置保存阶乘结果的变量 factorial 初始值为 1，由于结束标记为循环变量等于 0，所以在循环体中可通过不断地对循环变量进行自减操作，将循环变量与 factorial 相乘，并将结果重新赋值给 factorial，从而得到最终结果，当循环变量为 0 时退出循环，然后执行 done 后的命令，将最终结果输出。该脚本执

行时，设置了求阶乘的数为 5，可以看到，5 的阶乘为 120。

8.2.3 标志控制的 while 循环

标志控制的 while 循环使用用户输入的标志值来控制循环的结束，这样避免了用户不知道循环结束标记的麻烦。在该情形下，while 循环的形式如下：

```
signal=0

while (( signal != 1 ))
do
    …
    if expression
    then
       signal=1
    fi
    …
done
```

使用标志控制的 while 循环中的 signal 和 variable 可以是数字，也可以是字符串，脚本设计时由脚本编写人员指定。下面的脚本 while_exam5.sh 是一个使用标志控制的 while 循环的例子，该脚本是对脚本 while_exam3.sh 的改写，脚本内容如下：

```
#例 8-18：while_exam5.sh 脚本演示使用标志控制的 while 循环实现猜 1～10 内的数
#!/bin/bash

#提示用户输入所要实现阶乘的数值，并将其值赋给变量 num
echo "Please input the num:"
read num

#初始化标志的值
signal=0

#标志控制的 while 循环实现猜数游戏
while [[ "$signal" != 1 ]]
do
   if [ "$num" -lt 4 ]
   then
       echo "Too small. Try again!"
       read num
   else [ "$num" -gt 4 ]
   then
        echo "To high. Try again"
        read num
    else
        signal=1
        echo " Congratulation, you are right! "
   fi
done
```

脚本 while_exam5.sh 的执行结果为：

```
#例 8-18 while_exam5.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./while_exam5.sh
Please input the num(1-10)
```

```
5
Too high. Try again!
3
Too small. Try again!
4
Congratulation, you are right!
alloy@alloy:~/linuxshell/CH08$
```

通过脚本 while_exam5.sh 可以看出，使用标志控制的 while 循环时，当用户输入 4 时，在循环体内执行一次，但使用结束标记控制的 while 循环在用户输入 4 时无法在循环体内执行其语句，这是因为结束标记是用户输入的结果为 4 时退出循环，这是结束标记控制的 while 循环和标志控制的 while 循环的区别。

再举一个标志控制的 while 循环的例子，新建脚本 while_exam6.sh，内容如下：

```
#例 8-19：while_exam6.sh 脚本演示标志控制的 while 循环求累加和（1+2+…+n）
#!/bin/bash

#提示用户输入所要实现累加的数值，并将其值赋给变量 num
echo "Please input the num "
read num

#初始化循环累加和 sum 和循环变量 i
sum=0
i=1

#初始化标志值
signal=0

#执行累加操作
while [[ "$signal" != 1 ]]
do
    if [ "$i" -eq "$num" ]
    then
       let "signal=1"
       let "sum+=i"
       echo "1+2+…+$num=$sum"
    else
       let "sum=sum+i"
      let "i++"
    fi
done
```

脚本 while_exam6.sh 的执行结果为：

```
#例 8-19 while_exam6.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./while_exam6.sh
Please input the num:
100
1+2+...+100=5050
alloy@alloy:~/linuxshell/CH08$
```

脚本 while_exam6.sh 是一个实现累加的程序，在该脚本中，通过控制标志值来达到控制循环的目的，该脚本通过 if/else 命令控制标志值，如果循环变量的值小于 100，则执行 else 与 fi 之间的命令，这些命令实现 sum 的累加，同时对循环变量进行加 1 操作，如果循环变量的值等于 100，则执行 then 和 else 之间的命令，使标志值为 1，同时将累加结果输出。

8.2.4 命令行控制的 while 循环

有时需要使用命令行来指定输出参数和参数个数，这时用其他三种形式的 while 循环是无法实现的，所以需要使用命令行控制的 while 循环。该形式下，while 循环通常与 shift 结合起来使用，其中，shift 命令使位置变量下移一位（即$2 代替$1，$3 代替$2），并且使$#变量递减，当最后一个参数也显示给用户后，$#就会等于 0，同时$*也等于空。下面是该情形下，while 循环的形式：

```
while [[ "$*" != "" ]]
do
   echo "$1"
   shift
done
```

举一个例子来说明命令行控制的 while 循环的用法，新建脚本 while_exam7.sh，脚本内容如下：

```
#例 8-20: while_exam7.sh 脚本命令行控制的 while 循环
#!/bin/bash

#提示用户输入参数个数
echo "number of arguments is $#"

#提示用户输入内容
echo "What you input is: "

#通过命令行来传递脚本 for 循环列表参数
while [[ "$*" != "" ]]
do
   echo "$1"
   shift
done
```

脚本 while_exam7.sh 的执行结果为：

```
#例 8-20 while_exam7.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./while_exam7.sh 1 2 3
number of arguments is 3
What you input is:
1
2
3
alloy@alloy:~/linuxshell/CH08$ ./while_exam7.sh Hello World !
number of arguments is 3
What you input is:
Hello
World
!
alloy@alloy:~/linuxshell/CH08$
```

可以看到，脚本 while_exam8.sh 可以实现命令行输入字符串和数字，并将结果显示给用户，如果用户不输入任何参数，就根本不会执行 echo 和 shift 命令，因为$#为 0。

其中的循环条件可以改写成：

```
while [[ "$#" -ne 0 ]]
```

该语句通过判断参数格式$#不等于 0 时执行 while 循环，等于 0 时，则退出 while 循环。

8.3 until 循环

在执行 while 循环时，只要 expression 的退出状态为 0，将一直执行循环体。until 命令和 while 命令类似，但区别是 until 循环中 expression 的退出状态不为 0 时，循环体将一直执行下去，直到退出状态为 0。下面给出了 until 循环的结构：

```
until expression
do
    command
    command
    …
done
```

和 while 循环类似，当首次测试 expression 的退出状态为 0 时，将一次也不执行循环体。下面举一个例子来说明 until 循环的用法，新建脚本 until_exam1.sh，该脚本用于计算 1～5 的平方，脚本内容如下：

```
#例 8-21：until_exam1.sh 脚本通过 until 循环计算 1～5 的平方
#!/bin/bash

#初始化循环变量
i=0

#使用 until 循环计算 1～5 的平方
until [[ "$i" -gt 5 ]]
do
    let "square=i*i"
    echo "$i * $i = $square"
    let "i++"
done
```

脚本 until_exam1.sh 的执行结果为：

```
#例 8-21 until_exam1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./until_exam1.sh
1 * 1 = 1
2 * 2 = 4
3 * 3 = 9
4 * 4 = 16
5 * 5 = 25
alloy@alloy:~/linuxshell/CH08$
```

脚本 until_exam1.sh 通过 let 命令将每次 i 的平方保存到变量 square 中，然后通过 echo 命令输出，其控制命令为判断 i 是否大于 5，如果 i 不大于 5，将执行循环体，如果大于 5，则结束循环。

对于 while 循环能够实现的脚本，until 循环同样也可实现。下面的脚本是对脚本 while_exam3.sh 进行的改写，新建脚本 until_exam2.sh，内容如下：

```
#例 8-22：until_exam2.sh 脚本演示使用 until 循环实现猜 1～10 内的数
#!/bin/bash

#提示用户输入 1～10
```

```
echo "Please input the num(1-10) "
read num

#until 循环实现猜数游戏
until [[ "$num" = 4 ]]
do
   if [ "$num" -lt 4 ]
   then
       echo "Too small. Try again!"
       read num
   else [ "$num" -gt 4 ]
   then
        echo "To high. Try again"
        read num
    else
       exit 0
   fi
done

#提示用户猜对了
echo "Congratulation, you are right! "
```

脚本 until_exam2.sh 的执行结果为：

```
#例 8-22 until_exam2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./until_exam2.sh
Please input the num(1-10)
5
To high. Try again
3
Too small. Try again!
4
Congratulation, you are right!
alloy@alloy:~/linuxshell/CH08$
```

可以看出，while 循环和 until 循环非常相似，区别仅在于：while 循环在条件为真时继续执行循环，而 until 则在条件为假时执行循环。

8.4 嵌套循环

如果一个循环体内又包含另一个完整的循环结构，则称为循环的嵌套。在外部循环的每次执行过程中都会触发内部循环，直至内部完成一次循环，才接着执行下一次的外部循环。for 循环、while 循环和 until 循环可以相互嵌套。

为了读者更好地理解循环嵌套，下面将以一个具体的实例结束两个 for 语句嵌套的应用。本实例用于输出九九乘法表，新建脚本 nested_loop_exam1.sh，内容如下：

```
#例 8-23: nested_loop_exam1.sh 脚本演示使用 for 循环嵌套实现九九乘法表
#!/bin/bash

#通过 for 循环嵌套实现九九乘法表，其中 temp 用于保存 i*j 的值
for (( i = 1; i <=9; i++ ))
do
```

```
#内层循环
    for (( j=1; j <= i; j++ ))
    do
       let "temp = i * j"      #暂时存储 i*j 的值
       echo -n "$i*$j=$temp  "
     done

     echo ""
done
```

脚本 nested_loop_exam1.sh 的执行结果如下：

```
#例 8-23 nested_loop_exam1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./nested_loop_exam1.sh
1*1=1
2*1=2  2*2=4
3*1=3  3*2=6  3*3=9
4*1=4  4*2=8  4*3=12  4*4=16
5*1=5  5*2=10  5*3=15  5*4=20  5*5=25
6*1=6  6*2=12  6*3=18  6*4=24  6*5=30  6*6=36
7*1=7  7*2=14  7*3=21  7*4=28  7*5=35  7*6=42  7*7=49
8*1=8  8*2=16  8*3=24  8*4=32  8*5=40  8*6=48  8*7=56  8*8=64
9*1=9  9*2=18  9*3=27  9*4=36  9*5=45  9*6=54  9*7=63  9*8=72  9*9=81
alloy@alloy:~/linuxshell/CH08$
```

脚本 nested_loop_exam1.sh 在内循环中使用了 temp 暂时存储 i*j 的结果，然后通过 echo 命令加-n，表示不换行输出。而命令：

```
echo ""
```

实现换行操作。可以看出，类 C 风格的 for 循环使用很少的代码就可实现九九乘法表。

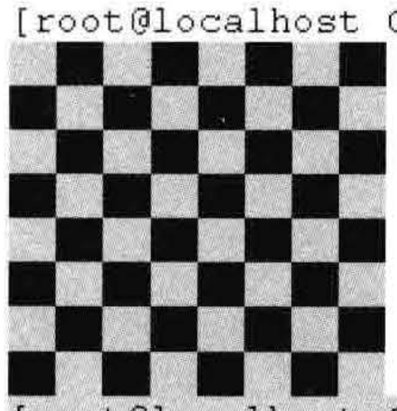
下面再举一个使用 for 循环嵌套的例子，新建脚本 nested_loop_exam2.sh，该脚本用于创建一个棋盘，脚本内容如下：

```
#例 8-24：nested_loop_exam2.sh 脚本演示 for 循环嵌套实现一个 8×8 格的棋盘
#!/bin/bash

#外层循环
for (( i = 1; i <= 8; i++ ))
do

   #内层循环
   for (( j = 1; j <= 8; j++ ))
   do

      #下面两行用于不重叠显示黑白格
      total=$(( $i + $j ))
      tmp=$(( $total % 2 ))

      if [ $tmp -eq 0 ];
      then
         echo -e -n "\033[47m  "  #显示白格
      else
         echo -e -n "\033[40m  "  #显示黑格
      fi
done
echo ""  #换行
```

```
done
```

脚本 nested_loop_exam2.sh 的执行结果如图 8-1 所示。

```
[root@localhost Chapter8]# ./nested_loop_exam2.sh
```



```
[root@localhost Chapter8]#
```

图 8-1　脚本 nested_loop_exam2.sh 的执行结果

脚本 nested_loop_exam2.sh 实现 8×8 的棋盘，使用类 C 风格的 for 循环实现了双层循环。由于前盘的上下左右都不相同，所以需要使用下列命令来判断 i 和 j 的和，然后取余，下面两行的运算过程就是实现该操作：

```
total=$(( $i + $j ))
tmp=$(( $total % 2 ))
```

然后通过 tmp 的 0 或 1 对其赋予不同的颜色，其中，“\033[47m”表示白色，而“\033[40m”后面的空格表示每个棋格的长宽，通过这种方式就实现了一个棋盘。

两个或多个 while 循环同样可以实现循环嵌套，新建脚本 nested_loop_exam3.sh，该脚本用于打印“*”号，脚本的内容如下：

```
#例 8-25：nested_loop_exam3.sh 脚本演示 for 循环嵌套实现“*”图案排列
#!/bin/bash

#初始化外层循环变量
i=1

#外层循环
while (( "$i" <= 9 ))
do

    #初始化内层循环变量
    j=1

    #内层循环
    while (( "$j" <= "$i" ))
    do
        echo -n "*"
        let "j++"
    done

    let "i++"

    echo ""
done
```

脚本 nested_loop_exam3.sh 的执行结果为：

```
#例 8-25 nested_loop_exam3.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./nested_loop_exam3.sh
*
```

```
**
***
****
*****
******
*******
********
*********
alloy@alloy:~/linuxshell/CH08$
```

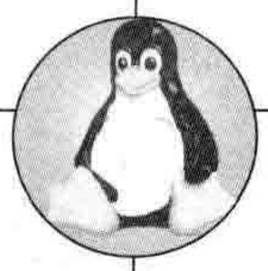
脚本 nested_loop_exam3.sh 用于打印“*”，其中第一行打印一个“*”号，第二行打印两个“*”，以此类推。可以看出，while 循环嵌套显得十分冗余，而且在程序设计时容易忘记初始化循环变量或者忘记对循环变量进行增量或减量处理。

同时可以将 for 循环和 While 循环结合在一起形成循环嵌套，新建脚本 nested_loop_exam4.sh，内容如下：

```
#例 8-26: nested_loop_exam4.sh 脚本演示 for 和 while 循环结合的循环嵌套反序的“*”图案排列
#!/bin/bash

#外层循环使用 for 循环
for (( i=1; i <= 9; i++))
do

    #初始化内层循环变量 j
    j=9

    #内层 while 循环实现打印空格
    while (( j > i ))
    do
        echo -n " "
        let "j--"
     done

    #初始化内层循环变量 k
    k=1

    #内层 while 循环实现打印“*”
    while (( k <= i ))
    do
          echo -n "*"
          let "k++"
      done
done
```

脚本 nested_loop_exam4.sh 的执行结果为：

```
#例 8-26 nested_loop_exam4.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./nested_loop_exam4.sh
        *
       **
      ***
     ****
    *****
   ******
  *******
 ********
```




```
**********
alloy@alloy:~/linuxshell/CH08$
```

脚本中，nested_loop_exam4.sh 实现了第一行打印 8 个空格和 1 个“*”，然后第二行打印 7 个空格和 2 个“*”，以此类推，最后实现打印 9 个“*”。该脚本在执行过程中，外层 for 循环嵌套了两个内层 while 循环，其中，第一个 while 循环第一行打印 8 个空格，第二行打印 7 个空格，以此类推，最后一行打印 0 个空格。而第二个 while 循环用于打印“*”，其中第 1 行打印 1 个“*”，第 2 行打印 2 个“*”，以此类推，第 9 行打印 9 个“*”，从而形成靠右显示的“*”图案排列。

8.5 循环控制符

在 Linux Shell 编程中，有时需要立即从循环中退出，如果是退出循环，则可使用 break 循环控制符；如果是退出本次循环执行后继循环，则可使用 continue 循环控制符。

8.5.1 break 循环控制符

break 语句可以应用在 for、while 和 until 循环语句中，用于强行退出循环，也就是忽略循环体中其他任何语句和循环条件的限制。下面通过例子来说明 break 循环控制符的作用，不使用 break 循环控制符计算 1～100 内连续整数的和，新建脚本 no_break_exam.sh，脚本内容如下：

```
#例 8-27：no _break_exam.sh 脚本演示不使用 break 的循环计算 1～100 内整数的和
#!/bin/bash

#初始化和 sum
sum=0

#使用 for 循环计算 1～100 内整数的和
for (( i=1; i <= 100; i++ ))
do
   let "sum+=i"
done

#显示计算结果
echo "1+2++…+100=$sum"
```

脚本 no_break_exam.sh 的执行结果为：

```
#例 8-27 no _break_exam.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ chmod u+x no_break_exam.sh
alloy@alloy:~/linuxshell/CH08$ ./no_break_exam.sh
1+2+...+100=5050
alloy@alloy:~/linuxshell/CH08$
```

在脚本 no_break_exam.sh 中添加上通过 if 语句控制的 break 语句，则会显示不同的结果，新建脚本 break_exam1.sh，内容如下：

```
#例 8-28：break_exam1.sh 演示使用 break 的循环计算 1～100 内整数的和
#!/bin/bash
```

```
#初始化和 sum
sum=0

#使用 for 循环计算 1～100 内整数的和
for (( i=1; i <= 100; i++ ))
do
    let "sum+=i"

    #在 if 语句中添加 break 语句
    if [ "$sum" -gt 1000 ]
    then
        echo "1+2+…+$i=$sum"
        break
    fi

done
```

脚本的执行结果为：

```
#例 8-28 break_exam1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./break_exam1.sh
1+2+...+45=1035
alloy@alloy:~/linuxshell/CH08$
```

从上面两个脚本可以看出，虽然 for 循环被设计为计算 1～100 内所有整数的和，但是由于当累加结果大于 1000 时，使用了 break 循环控制符终止了 for 循环语句，所以当循环结束时，i 的值并不等于 100，而是等于 45。需要注意的是，break 语句仅能退出当前的循环，如果是两层循环嵌套，且 break 循环控制符在内层循环中，则 break 仅跳出内层循环，如果想跳出整个循环，则需要在外层循环中使用 break 循环控制符，例如：下面的脚本 break_exam2.sh 当语句执行到 break 循环控制符时，将退出内部 for 循环，脚本内容如下：

```
#例 8-29：break_exam2.sh 演示使用 break 的循环跳出内层循环
#!/bin/bash

#使用 break 循环跳出 i=7 的那一行
for (( i=1; i <= 9; i++))
do

        #内层循环执行 break 循环控制符
        for (( j=1; j<= i; j++ ))
        do

            #将 i*j 的临时结果保存在 temp 中
            let "temp=i*j"

            #break 语句跳出内层循环
            if [ "$temp" -eq 7 ]
            then
                break
            fi

            echo -n "$i*$j=$temp  "
        done
        echo ""
done
```

脚本 break_exam2.sh 的执行结果为：

```
#例 8-29 break_exam2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./break_exam2.sh
1*1=1
2*1=2  2*2=4
3*1=3  3*2=6  3*3=9
4*1=4  4*2=8  4*3=12  4*4=16
5*1=5  5*2=10  5*3=15  5*4=20  5*5=25
6*1=6  6*2=12  6*3=18  6*4=24  6*5=30  6*6=36

8*1=8  8*2=16  8*3=24  8*4=32  8*5=40  8*6=48  8*7=56  8*8=64
9*1=9  9*2=18  9*3=27  9*4=36  9*5=45  9*6=54  9*7=63  9*8=72  9*9=81
alloy@alloy:~/linuxshell/CH08$
```

可以看出，将 break 循环控制符放在内部 for 循环时，结果 i=7 的那一行未显示，但 i=8 和 i=9 的结果还在显示。下面的例子是跳出整个循环嵌套的脚本，新建脚本 break_exam3.sh，内容如下：

```
#例 8-30: break_exam3.s 脚本演示使用 break 控制符跳出整个循环
#!/bin/bash

#将 break 循环控制符放在最外层循环
for (( i=1; i <= 9; i++))
do

    #break 语句跳出外层循环
    if [ "$i" -eq 7 ]
      then
          break
    fi

     #内层循环执行输出
     for (( j=1; j<= i; j++ ))
     do
         #将 i*j 的临时结果保存在 temp 中
         let "temp=i*j"

         echo -n "$i*$j=$temp  "
     done

     echo ""
done
```

脚本 break_exam3.sh 的执行结果为：

```
#例 8-30 break_exam3.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./break_exam3.sh
1*1=1
2*1=2  2*2=4
3*1=3  3*2=6  3*3=9
4*1=4  4*2=8  4*3=12  4*4=16
5*1=5  5*2=10  5*3=15  5*4=20  5*5=25
6*1=6  6*2=12  6*3=18  6*4=24  6*5=30  6*6=36
alloy@alloy:~/linuxshell/CH08$
```

可以看出，将 break 循环控制符放在外层循环时，i=7，i=8 和 i=9 都未执行，所以将 break 循环控制符放在外层循环时，则跳出整个循环。

8.5.2 continue 循环控制符

continue 循环控制符应用在 for、while 和 until 语句中，用于让脚本跳过其后面的语句，执行下一次循环，例如，实现输出 100 以内能被 7 整除的所有数，新建脚本 continue_exam1.sh，内容如下：

```
#例 8-31：continue_exam1.sh 脚本演示使用 continue 循环显示 100 以内能被 7 整除的数
#!/bin/bash

#初始化一个中间变量 m
m=1

#使用 for 循环和 continue 循环控制符实现结果输出
for (( i=1; i < 100; i++ ))
do

    #设置中间变量 temp1 的值
    let "temp1=i%7"

    #在 if 结构中使用 continue 循环控制符
    if [ "$temp1" -ne 0 ]
    then
        continue
    fi

    #输出结果
    echo -n "$i"

    #设置中间变量 temp2 的值，该变量用于换行
    let "temp2=m%7"

    #每行显示 7 个数
    if  [ "$temp2" -eq 0 ]
    then
        echo ""
    fi

    let "m++"
done
```

脚本 continue_exam1.sh 的执行结果为：

```
#例 8-31 continue_exam1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./continue_exam1.sh
7  14  21  28  35  42  49
56  63  70  77  84  91  98
alloy@alloy:~/linuxshell/CH08$
```

在脚本 continue_exam1.sh 中，当 i 不能被 7 整除时，将跳过下面的输出命令，直接进入下一次循环，如果 i 能被 7 整除，将执行输出命令，当计数变量 m 取余结果为 7 时，执行换行命令。

下面的脚本是一个 for 循环嵌套中使用 continue 循环控制符的例子。为了形成对比，脚本 continue_exam2.sh 同样用于实现九九乘法表，将 break_exam2.sh 中 if 语句中的 break 修改为 continue，同样将 continue 循环控制符放在内层循环，脚本内容如下：

```
#例 8-32: continue_exam2.sh 脚本演示使用 continue 循环跳出内层循环的九九乘法表
#!/bin/bash

#外层循环
for (( i=1; i <= 9; i++))
do

      #将 continue 循环控制符放在内层循环
      for (( j=1; j<= i; j++ ))
      do

        #将 i*j 的临时结果保存在 temp 中
        let "temp=i*j"

        #break 语句跳出循环
        if [ "$temp" -eq 7 ]
        then
            continue
        fi

        #输出结果
        echo -n "$i*$j=$temp  "
done

      echo ""
 done
```

脚本 continue_exam2.sh 的执行结果为：

```
#例 8-32 continue_exam2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./continue_exam2.sh
1*1=1
2*1=2  2*2=4
3*1=3  3*2=6  3*3=9
4*1=4  4*2=8  4*3=12  4*4=16
5*1=5  5*2=10  5*3=15  5*4=20  5*5=25
6*1=6  6*2=12  6*3=18  6*4=24  6*5=30  6*6=36
7*2=14  7*3=21  7*4=28  7*5=35  7*6=42  7*7=49
8*1=8  8*2=16  8*3=24  8*4=32  8*5=40  8*6=48  8*7=56  8*8=64
9*1=9  9*2=18  9*3=27  9*4=36  9*5=45  9*6=54  9*7=63  9*8=72  9*9=81
alloy@alloy:~/linuxshell/CH08$
```

通过脚本 continue_exam2.sh 的执行结果可以看出，“7*1=7”未显示，所以，将 continue 放在最内层循环体中时，仅仅是结束了本次循环执行下一次循环。下面的脚本是对 break_exam3.sh 的改写，同样只是将 break 改为 continue，将 continue 放在最外层循环，脚本内容如下：

```
#例 8-33: continue_exam3.sh 脚本演示使用 continue 控制符跳出外层循环
#!/bin/bash

#将循环控制符放在最外层循环
for (( i=1; i <= 9; i++))
do

    #continue 语句跳出本次外层循环，执行下一次外层循环
    if [ "$i" -eq 7 ]
```

```
then
    continue
fi

for (( j=1; j<= i; j++ ))
do

    #将 i*j 的临时结果保存在 temp 中
    let "temp=i*j"

      echo -n "$i*$j=$temp  "
   done

   echo ""
 done
```

脚本 continue_exam3.sh 的执行结果为：

```
#例 8-33 continue_exam3.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./continue_exam3.sh
1*1=1
2*1=2  2*2=4
3*1=3  3*2=6  3*3=9
4*1=4  4*2=8  4*3=12  4*4=16
5*1=5  5*2=10  5*3=15  5*4=20  5*5=25
6*1=6  6*2=12  6*3=18  6*4=24  6*5=30  6*6=36
8*1=8  8*2=16  8*3=24  8*4=32  8*5=40  8*6=48  8*7=56  8*8=64
9*1=9  9*2=18  9*3=27  9*4=36  9*5=45  9*6=54  9*7=63  9*8=72  9*9=81
alloy@alloy:~/linuxshell/CH08$
```

通过脚本 continue_exam3.sh 的执行结果可以看出，将 continue 放到最外层循环时，i=7 这一行都未执行，其结束了整个 i=7 的外部循环，所以可以通过将 continue 放在外层循环或内层循环来实现不同的循环控制。

8.6 select 结构

select 结构从技术角度来看不能算是循环结构，因为其不对可执行结果的代码块进行循环操作，但其与循环结构有相似之处，它们也依靠在代码块的顶部或底部的条件判断来决定程序的分支。Select 是 bash 的扩展结构，用于交互式菜单显示，用户可以从一组不同的值中进行选择，功能类似于 case 结构，但其交互性要比 case 好得多，其基本结构为：

```
select variable in {list}
do
    command
    …
break
done
```

下面通过一个例子来说明 select 结构的用法，新建脚本 select_exam1.sh，内容如下：

```
#例 8-34: select_exam1.sh 脚本演示交互式显示用户喜欢的颜色，并让用户选择
#!/bin/bash

#设置提示符
```

```
echo "What is your favorite color? "

#通过 select 结构实现用户选择
select color in "red" "blue" "green" "white" "black"
do
   break
done

#提示用户输入的结果
echo "You have selected $color"
```

脚本 select_exam1.sh 的执行结果为：

```
#例 8-34 select_exam1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./select_exam1.sh
What is your favourite color?
1) red
2) blue
3) green
4) white
5) black
#? 3
You have selected green
alloy@alloy:~/linuxshell/CH08$
```

由脚本 select_exam1.sh 的执行结果可以看出，使用 select 可以提供选项供用户选择，这样提高了系统和用户的交互。select 还有一种格式，该格式是不带参数列表的 select 结构，该结构通过命令行来传递参数列表，由用户自己设定参数列表，其格式为：

```
select variable
do
   command
   …
   break
done
```

下面再举一例子说明该格式的 select 结构的用法，新建脚本 select_exam2.sh，内容如下：

```
#例 8-35：select_exam2.sh 脚本演示命令行输入形式的 select 结构
#!/bin/bash

#设置提示符
echo "What is your favorite color? "

#通过命令行传递 select 结构的 list 参数列表
select color
do
   break
done

#提示用户输入的结果
echo "You have selected $color"
```

脚本 select_exam2.sh 的执行结果为：

```
#例 8-35 select_exam2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH08$ ./select_exam2.sh red green blue white black
What is your favourite color?
1) red
2) green
```

```
    3) blue
    4) white
    5) black
    #? 2
    You have selected green
    alloy@alloy:~/linuxshell/CH08$
```

脚本 select_exam2.sh 执行时通过命令行传递了列表参数，然后罗列出命令行参数，最后通过用户回答显示最终结果。

8.7 本章小结

本章讨论了循环结构的详细用法，结合实例详细讲解了 for 循环、while 循环和 until 循环的基本格式和语义，这三种循环中，while 循环和 for 循环属于“当型循环”，而 until 属于“直到型循环”。

循环结构可以相互嵌套组成更复杂的流程，为了更好地控制循环，Linux Shell 中还提供了和循环密切相关的流程转向控制符 break 和 continue，其中，break 用于跳出其所在的循环体，而 continue 则是结束本次循环执行下一次循环。本章最后介绍了 select 结构的用法。

8.8 上机提议

1．使用 for 循环计算 100 以内所有偶数的和，然后用 while 循环和 until 循环来实现这个计算，比较哪种结构更简单。

2．使用 while 循环或者 until 循环实现从命令行读入字符串，直到输入的字符串为句号为止。

3．将 for_exam9.sh 中输入的命令行参数改为'1 2'、3，查看输出结果是否与原来相同。

4．使用 for 循环实现“2，4，8，16，32，64”的结果显示，然后使用 while 循环和 until 循环实现。

5．通过循环实现从 1 开始叠加，直到和的结果大于 2000 为止。（提示：可以通过两种方式实现，一种方式是在循环条件中设置和大于 2000 时结束，第二种方式是使用 break 循环控制符实现。）

6．找出 100 以内所有能被 3 整除的数，每行显示 8 个数，然后换行显示。

7．打印如下的“*”组成的图案：

```
      *
     ***
    *****
   *******
  *********
```

8．如果一个整数各位数字之和可以被 9 整除，那么该整数就可被 9 整除，编写一个脚本提示用户输入一个整数，然后输出该整数，并告之该整数是否可被 9 整除。

9. 编写一个脚本，该脚本提示用户输入一些整数，然后通过程序控制分别计算出这些整数中奇数之和与偶数之和，并将其输出。

10. 编写一个脚本提示用户输入一个正整数，程序将输出信息提示该正整数是否为质数。（提示：偶数只有 2 是质数，如果为奇数，且不能被任何一个小于或等于它平方根的奇数整除，那么该奇数就是质数。）

11. 一个数恰好等于它的因子之和，这样的数称为“完数”。例如，6 的因子为 1、2、3，而 6=1+2+3，因此 6 是“完数”。编写程序写出 1000 以内所有的完数，并按下面的格式输出其因子：

```
6 its factors are 1 2 3
```

12. 编写一个脚本读入一些整数，并分别输出这些整数中奇数的个数和偶数的个数，然后输出 0 的个数。

13. 编写一个脚本提示用户输入一个整数，程序将分别输出该整数每位上的数字，并输出这些数字的和。例如，输出整数 2345 每个位上的数字 2 3 4 5；输出整数-3456 每个位上的数字是 3 4 5 6。

14. 使用 while 循环编写脚本，使其完成下面的功能。

1）提示用户输入两个整数：firstNum 和 secondNum（firstNum 的值一定要小于 secondNum）。

2）输出所有介于 firstNum 和 secondNum 之间的奇数。

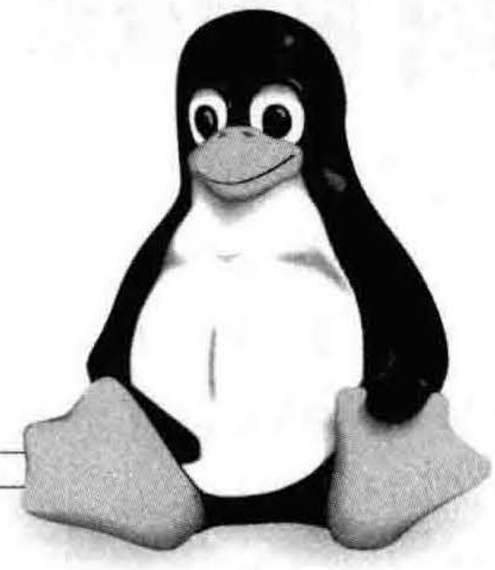
3）输出所有介于 firstNum 和 secondNum 之间的偶数之和。

15. 分别使用 until 循环和 for 循环改写第 14 题。

16. 将第 7 题的图案的脚本中加入 break 或 continue 循环控制符，查询显示的结果。

17. 上机运行下面的脚本，查看脚本的输出结果是什么？

```
#!/bin/bash

for (( i = 11; i <= 16; i++ ))
do
   let "temp = i%6 "

   case "$temp" in
   0)
      echo -n "to  "
   1)
      echo -n "Linux  "
   2)
      echo -n "Shell  "
   3)
      echo -n "world  "
   4)
      echo -n "!  "
   5)
      echo -n "Welcome  "
   *)
      echo "Bad Number. "
done
```



第 9 章

变量的高级用法

第 6 章已经介绍了变量的基本用法，包括变量的替换和赋值、无类型性、环境变量，以及四种引用符号及其意义和用法。本章扩展变量的基本用法，介绍 Shell 编程中变量的一些高级用法，由于本章很多例子使用了流程控制相关的内容。因此，本书将第 9 章放在了第 8 章之后。

本章首先系统介绍 bash Shell 中的内部变量，其次介绍 Shell 处理字符串的命令和方法，再次介绍如何将原本无类型的 Shell 变量声明为特定类型，然后介绍间接变量引用，最后介绍 bash 数学运算，包含 expr 命令和 bc 运算器。本章与第 6 章一起阐述清楚了 Shell 变量的全部内容。

9.1 内部变量

内部变量是指能够对 bash Shell 脚本行为产生影响的变量，它们对 Shell 及其子 Shell 都有效。因此，内部变量属于环境变量的范畴。6.1.3 节已经介绍了几个重要的环境变量，其中的 PWD、SHELL、USER、UID、PPID、PS1、PS2 和 IFS 变量都为内部变量，本节在上述几个内部变量的基础上，完整地介绍 Linux bash Shell 的内部变量。

1. BASH

BASH 记录了 bash Shell 的路径，通常为/bin/bash，内部变量 SHELL 就是通过 BASH 的值确定当前 Shell 的类型。例 9-1 显示了 BASH 和 SHELL 变量的值，由于 Ubuntu 12.04 使用的是 bash Shell，因而两个变量的值都是/bin/bash。

```
#例 9-1：BASH 和 SHELL 变量的值
alloy@alloy:~/linuxshell/CH09$ echo $BASH
/bin/bash
alloy@alloy:~/linuxshell/CH09$ echo $SHELL
/bin/bash
alloy@alloy:~/linuxshell/CH09$
```

2. BASH_SUBSHELL

BASH_SUBSHELL 记录了子 Shell 的层次，这个变量在 bash 版本 3 之后才出现的，关于子 Shell 的相关内容，将在第 12 章介绍，在此不举例介绍该变量的用法。

3. BASH_VERSINFO

BASH_VERSINFO 是一个数组，包含 6 个元素，这 6 个元素用于表示 bash 的版本信息，例 9-2 新建名为 bashver.sh 的脚本，用于输出 BASH_VERSINFO 数组的值，脚本内容如下：

```
#例 9-2：bashver.sh 输出 BASH_VERSINFO 数组的值
#!/bin/bash

for n in 0 1 2 3 4 5          #for 循环执行 6 次
do
  echo "BASH_VERSINFO[$n]=${BASH_VERSINFO[$n]}"
done
```

下面给出例 9-2 中 bashver.sh 脚本的执行结果，BASH_VERSINFO[0]表示 bash Shell 的主版本号，BASH_VERSINFO[1]表示 bash Shell 的次版本号，BASH_VERSINFO[2]表示 bash Shell 的补丁级别，BASH_VERSINFO[3]表示 bash Shell 的编译版本，BASH_VERSINFO[4]表示 bash Shell 的发行状态，BASH_VERSINFO[5]表示 bash Shell 的硬件架构。上述结果是在 Ubuntu 12.04 操作系统下得到的。

```
#例 9-2 bashver.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH09$ chmod u+x bashver.sh
alloy@alloy:~/linuxshell/CH09$ ./bashver.sh
BASH_VERSINFO[0]=4
BASH_VERSINFO[1]=2
BASH_VERSINFO[2]=25
BASH_VERSINFO[3]=1
BASH_VERSINFO[4]=release
```

```
BASH_VERSINFO[5]=x86_64-pc-linux-gnu
```

4. BASH_VERSION

Linux 系统的 bash Shell 版本包含主次版本、补丁级别、编译版本和发行状态，即 BASH_VERSINFO 数组从 0 到 4 的值。例 9-3 显示了 BASH_VERSION 的值。

```
#例 9-3：BASH_VERSION 变量值
alloy@alloy:~/linuxshell/CH09$ echo $BASH_VERSION
4.2.25(1)-release
alloy@alloy:~/linuxshell/CH09$
```

5. DIRSTACK

它显示目录栈的栈顶值，栈是一种线性数据结构，遵循后进先出（Last In First Out，LIFO）的原则，即最后压入栈的元素位于栈顶，它将最早被弹出。栈结构就像是一个子弹夹，最后压入弹夹的子弹将第一个被射出。栈结构一般有两个动作；入栈（Pushing）将元素放入栈的顶端，出栈（Poping）将顶端元素取出。

Linux 目录栈用于存放工作目录，便于程序员手动控制目录的切换，bash Shell 定义了两个系统命令 pushd 和 popd 来维护目录栈，pushd 命令用于将某目录压入目录栈，同时将当前工作目录切换到入栈的目录，popd 命令将栈顶目录弹出，栈顶元素变为下一个元素，同时将当前工作目录切换到栈弹出的目录。两个命令的格式如下：

```
pushd 目录名
popd
```

DIRSTACK 记录了栈顶目录值，初值为空。另外，Linux 还有一个命令 dirs 用于显示目录栈的所有内容，pushd 和 popd 命令执行成功将自动激活 dirs 命令以显示当前目录栈的内容。请看下面的例 9-4 给出的四条命令，连续运行这四条命令时，变量$DIRSTACK 保存最新进入目录栈的目录名称，即$DIRSTACK=/usr/local，目录栈自顶到底为：/usr/local、/usr、/home/globus、/home。

```
#例 9-4：四条 pushd 命令后的目录栈结果
pushd /home
pushd globus    #注意：此处以相对目录的形式，将/home/globus 压入目录栈
pushd /usr
pushd local
```

例 9-5 演示了目录栈和$DIRSTACK 变量，其中第 1 条和第 2 条命令分别将/home、/home/alloy 压入目录栈，两条命令执行后都显示了目录栈的所有元素，我们可以看到/home/alloy 进栈后，目录栈从顶到底为：/home/alloy、/home、~ /home ~/linuxshell/CH09，/home/alloy 和/home 是由 pushd 命令压入目录栈的元素，~ /home ~/linuxshell/CH09 是当前的工作目录，因此，bash Shell 将当前工作目录自动压入目录栈。例 9-5 中的第 3 条命令打印 DIRSTACK，显示为目录栈顶的值，第 4 条命令 popd 将栈顶元素/home/alloy 出栈，第 5 条命令打印 DIRSTACK，显示为新的目录栈顶值：/home。同时，由例 9-5 也可看出，每次执行 pushd 命令后，工作目录也随之切换到入栈的目录。

```
#例 9-5：演示目录栈和$DIRSTACK 变量
alloy@alloy:~/linuxshell/CH09$ pushd /home  #第 1 条命令：将/home 入栈
/home ~/linuxshell/CH09
alloy@alloy:/home$ pushd alloy/             #第 2 条命令：将/home/alloy 入栈
~ /home ~/linuxshell/CH09
alloy@alloy:~$ echo $DIRSTACK               #第 3 条命令：打印$DIRSTACK 变量
```

```
~
alloy@alloy:~$ popd                                    #第 4 条命令：使栈顶元素出栈
/home ~/linuxshell/CH09
alloy@alloy:/home$ echo $DIRSTACK                      #第 5 条命令：打印$DIRSTACK 变量
/home
alloy@alloy:/home$
```

6. GLOBIGNORE

GLOBIGNORE 是由冒号分隔的模式列表，表示通配（Globbing）时忽略的文件名集合。如 3.3 节所述，通配是把一个包含通配符的非具体文件名扩展到存储在计算机、服务器或者网络上的一批具体文件名的过程，一旦 GLOBIGNORE 非空，Shell 会将通配得到的结果中符合 GLOBIGNORE 模式中的目录去掉。下面通过一个例子说明 GLOBIGNORE 的意义，例 9-6 中第 1 条命令 ls *a 的作用是列出当前目录以 a 开头的所有文件，一共有 14 个，其中以 ar 开头的文件有 9 个；然后，第 2 条命令将 GLOBIGNORE 变量赋为 ar*，表示以 ar 开头的模式。第 3 条命令再次执行 lsa*，列出以 a 开头的所有文件时，发现 9 个以 ar 开头的文件已经被剔除，这是因为 append.sed 文件符合 GLOBIGNORE 所定义的模式，在通配时被忽略。

```
#例 9-6：演示$GLOBIGNORE 变量的用法
#第 1 条命令：列出字母 a 开头的所有文件，一共有 14 个
alloy@alloy:~/linuxshell/CH09$ ls a*
alias.sh      andlist2.sh     append.sed  array_      array_eval1.sh  array_eval4.sh
array_print2.sh
andlist1.sh  anotherres.sh  argv.awk     array.awk  array_eval3.sh  array_print1.sh
arrivedcity.sh
#第 2 条命令：将 GLOBIGNORE 赋为以 ar 开头的模式
alloy@alloy:~/linuxshell/CH09$ GLOBIGNORE="ar*"
alloy@alloy:~/linuxshell/CH09$ echo $GLOBIGNORE
ar*
#第 3 条命令：再次列出以字母 a 开头的文件时，所有以 ar 开头的文件已经被剔除
alloy@alloy:~/linuxshell/CH09$ ls a*
alias.sh  andlist1.sh  andlist2.sh  anotherres.sh  append.sed
alloy@alloy:~/linuxshell/CH09$
```

7. GROUPS

GROUPS 记录了当前用户所属的群组，Linux 的一个用户可同时包含在多个组内，因此，GROUPS 是一个数组，数组记录了当前用户所属的所有群组号。Linux 管理用户组的文件是/etc/group，每个群组对应该文件中的一行，并用冒号分成四个域，/etc/group 每一行的格式如下：

```
群组名:加密后的组口令:群组号:组成员列表
```

下面摘取出 Ubuntu 12.04 操作系统/etc/group 文件的前面几行：

```
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:alloy
```

可以看出，root 用户同时属于 root、bin、daemon、sys 等群组，接着，我们看一看 root 用户的 GROUPS 变量的值，如例 9-7 所示，$GROUPS=${GROUPS[0]}=0，是/etc/group 第 1 行 root 群组的号码；${GROUPS[1]}=1，是/etc/group 第 2 行 bin 群组的号码。下面的例 9-7 中 GROUPS 数组依次记录了 root 用户所属的所有群组。

```
#例 9-7：root 用户的 GROUPS 变量值
alloy@alloy:~/linuxshell/CH09$ echo $GROUPS
0
alloy@alloy:~/linuxshell/CH09$ echo ${GROUPS[0]}
0
alloy@alloy:~/linuxshell/CH09$ echo ${GROUPS[1]}
1
alloy@alloy:~/linuxshell/CH09$ echo ${GROUPS[2]}
2
alloy@alloy:~/linuxshell/CH09$
```

8. HOSTNAME

HOSTNAME 记录了主机名，Linux 主机名是网络配置时必须要设置的参数，我们一般在 /etc/sysconfig/network 文件中设置主机名。/etc/hosts 文件用于设定 IP 地址和主机名之间的对应关系，以利于快速从主机名查找到 IP 地址。

9. HOSTTYPE 和 MACHTYPE

HOSTTYPE 和 MACHTYPE 一样，都用于记录系统的硬件架构，实际上，它们与 BASH_VERSINFO[5]也是等值的，例 9-8 显示了 HOSTTYPE 和 MACHTYPE 的值，可以看到，该机器为 i386 架构的。

```
#例 9-8：显示 HOSTTYPE 和 MACHTYPE 变量的值
alloy@alloy:~/linuxshell/CH09$ echo $HOSTTYPE
x86_64
alloy@alloy:~/linuxshell/CH09$ echo $MACHTYPE
x86_64-pc-linux-gnu
alloy@alloy:~/linuxshell/CH09$
```

10. OSTYPE

OSTYPE 记录了操作系统类型，Linux 系统中，$OSTYPE=linux。

11. REPLY

REPLY 变量与 read 和 select 命令有关。因此，需要先介绍这两个命令。

read 命令用于读取标准输入（stdin）的变量值，stdin 将在第 10 章 I/O 重定向中介绍，在此，读者可以将 read 命令简单理解为接受用户的键盘输入，交互式 Shell 脚本经常用到 read 命令。read 命令的一般格式为：

```
read variable
```

variable 是变量名，read 将读到的标准输入存储到 variable 变量中。read 命令也可以不带任何变量名，此时，read 就将读到的标准输入存储到 REPLY 变量中。下面的例 9-9 说明了 REPLY 变量在 read 命令中的用法，新建 readreply.sh 脚本，内容如下：

```
#例 9-9：readreply.sh 演示 REPLY 变量在 read 命令中的用法
#!/bin/bash

#第一部分
echo -n "What is your name?"
read                                    #read 不带变量名
echo "Your name is $REPLY"              #打印$REPLY

#第二部分
echo -n "What is the name of your father?"
read fname
echo "Your father's name is $fname"
```

```
echo "But the \$REPLY is $REPLY"
```

readreply.sh 脚本分为两部分，第一部分用不带变量名的 read 命令接受用户输入，然后打印 REPLY 变量值。下面给出 readreply.sh 脚本的执行结果，用户输入 Jack，$REPLY=Jack，这说明 read 命令读到的值确实存储到了 REPLY 变量中；第二部分 read 命令后跟 fname 变量，然后分别打印 fname 和 REPLY 的值，从 readreply.sh 脚本的执行结果可知，$fname=Tom，这说明此时 read 命令读到的值存储到了 fname 变量中，而 REPLY 的值保持不变，仍为 Jack。

```
#例 9-9 readreply.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH09$ chmod u+x readreply.sh
alloy@alloy:~/linuxshell/CH09$ ./readreply.sh
What is your name?Jack
Your name is Jack    #REPLY 变量的值，read 已将从标准输入读取的值存储到 REPLY 了
What is the name of your father?Tom
Your father's name is Tom                    #fname 变量的值
But the $REPLY is Jack                       #REPLY 变量值保持不变
alloy@alloy:~/linuxshell/CH09$
```

bash Shell 的 select 命令源于 Korn Shell，是建立菜单的一种工具，它提供一组字符串供用户选择，用户不必完整地输入字符串，只需输入相应的序号进行选择即可，select 命令的格式如下：

```
select variable in list
do
  Shell 命令 1
  Shell 命令 2
  Shell 命令 3
  ……
break
done
```

上述 select 命令格式中的 list 是字符串列表，select 自动将 list 形成有编号的菜单，用户输入序号以后，将该序号所对应 list 中的字符串赋给 variable 变量，而序号值则保存到 REPLY 变量中。select 命令的 do break 语句段中可以添加 Shell 命令，对 variable 或 REPLY 进行调用。

下面的例 9-10 阐述 select 命令的用法，以及 REPLY 变量在 select 命令中的含义。新建一个名为 selectreply.sh 的脚本，内容如下：

```
#例 9-10：selectreply.sh 脚本演示 REPLY 变量在 select 命令中的用法
#!/bin/bash

echo "Pls. choose your profession?"
select var in "Worker" "Doctor" "Teacher" "Student" "Other"
do
   echo "The \$REPLY is $REPLY."
   echo "Your preofession is $var."
break
done
```

selectreply.sh 脚本中的 select 命令将 list 设置为 5 个字符串，每个字符串用双引号引起，中间用空格分隔，select 命令的变量名为 var。do break 语句段打印 REPLY 和 var 的值。下面给出 selectreply.sh 脚本的执行结果，可以看到，select 自动将 list 中字符串建成菜单，并用数字对每个字符串标号，用户选择序号 3，结果显示 REPLY 变量值为 3，var 变量为序号 3 所对应的字符串 Teacher。

```
#例 9-10 selectreply.sh 脚本的执行结果
```

```
alloy@alloy:~/linuxshell/CH09$ chmod u+x selectreply.sh
alloy@alloy:~/linuxshell/CH09$ ./selectreply.sh
Pls. choose your profession?
1) Worker
2) Doctor
3) Teacher
4) Student
5) Other
#? 3                                          #用户选择序号 3
The $REPLY is 3.                              #REPLY 变量值为 3
Your preofession is Teacher.                  #var 变量是所选择序号所对应的字符串
alloy@alloy:~/linuxshell/CH09$
```

在上述 selectreply.sh 脚本的执行结果中，我们注意到输入 select 命令后，Shell 提示符变为“#?”，该提示符由 Shell 提示符变量 PS3 进行设置，“#?”是其默认值。下面的例 9-11 重新定义了 PS3 变量，运行 selectreply.sh 脚本后，Shell 提示符变成 PS3 的值。

```
#例 9-11：修改 PS3 变量，影响 select 命令的提示符
alloy@alloy:~/linuxshell/CH09$ PS3="Pls. Enter:"
alloy@alloy:~/linuxshell/CH09$ export PS3
alloy@alloy:~/linuxshell/CH09$ ./selectreply.sh
Pls. choose your profession?
1) Worker
2) Doctor
3) Teacher
4) Student
5) Other
Pls. Enter:4                                  #提示符由原来的“#?”变为 PS3 变量的值
The $REPLY is 4.
Your preofession is Student.
alloy@alloy:~/linuxshell/CH09$
```

12. SECONDS

SECONDS 记录脚本从开始执行到结束所耗费的时间，以秒为单位。下面的例 9-12 说明了 SECONDS 变量的用法，新建一个名为 runsec.sh 的脚本，内容如下：

```
#例 9-12：runsec.sh 脚本演示 SECONDS 变量的用法
#!/bin/bash

#定义两个变量：count 记录循环次数、MAX 为 while 循环条件
count=1
MAX=5

while [ "$SECONDS" -le "$MAX" ]          #当 SECONDS 小于或等于 MAX 时，执行循环体
do
  echo "This is the $count time to sleep."
  let count=$count+1
  sleep 2                                #运行该脚本进程休眠 2 秒
done

echo "The running time of this script is $SECONDS"
```

runsec.sh 脚本的主体是一个 while 循环，while 循环体的执行条件是该脚本的执行时间小于或等于 MAX 变量的值，循环体语句首先打印进入循环体的次数，然后调用 sleep 命令使运行该脚本进程休眠 2 秒。下面给出了 runsec.sh 脚本的执行结果，该脚本共休眠 3 次，执行时

间为 6 秒。

```
#例 9-12 runsec.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH09$ chmod u+x runsec.sh
alloy@alloy:~/linuxshell/CH09$ ./runsec.sh
This is the 1 time to sleep.
This is the 2 time to sleep.
This is the 3 time to sleep.
The running time of this script is 6
alloy@alloy:~/linuxshell/CH09$
```

13. SHELLOPTS

SHELLOPTS 记录了处于“开”状态的 Shell 选项（Options）列表，它是一个只读变量。Shell 选项用于改变 Shell 的行为，一个 Shell 选项有“开”和“关”两种状态。set 命令用于打开或关闭选项，最基本的两种格式如下：

```
set -o optionname                    #打开名为 optionname 选项
set +o optionname                    #关闭名为 optionname 选项
```

一个 set 命令可以同时设置多个选项，只要在每个选项名之前加上-o 或+o 即可。每个 Shell 选项名有一个简写，如 interactive 选项的简写是-i，因此，可以使用以下两条等价的命令打开 interactive 选项：

```
set -o interactive
set -i
```

关闭 interactive 仅需将上述两条命令中的“–”符号改为“+”符号，用完整的选项名打开或关闭选项，选项名前需加–o 或+o 符号，但是，用简写打开或关闭选项时，直接用“–”或“+”简写符号就可以了。例 9-12 演示 set 命令和 SHELLOPTS 的用法，例中命令首先开启 noclobber 选项，打开选项用的是–o 加选项名的命令格式，然后显示 SHELLOPTS 变量，该变量用冒号分隔记录 Shell 选项，最后一个选项就是刚添加的 noclobber 选项。noclobber 选项的简写是 C，因此，set +C 命令用于关闭 noclobber 选项，noclobber 选项也随之从 SHELLOPTS 变量中消失。

```
alloy@alloy:~/linuxshell/CH09$ set -o noclobber          #开启 noclobber 选项
alloy@alloy:~/linuxshell/CH09$ echo $SHELLOPTS           #该变量的最后一项是 noclobber
braceexpand:emacs:hashall:histexpand:history:interactive-comments:monitor:noclobber
alloy@alloy:~/linuxshell/CH09$ set +C                    #关闭 noclobber 选项
alloy@alloy:~/linuxshell/CH09$ echo $SHELLOPTS
braceexpand:emacs:hashall:histexpand:history:interactive-comments:monitor
alloy@alloy:~/linuxshell/CH09$
```

Shell 选项有很多，限于篇幅，我们不一一举例介绍，仅将常用的 Shell 选项名、简写、意义列于表 9-1 中，供读者查阅。

表 9-1　bash Shell 选项、简写及其意义

选项名称	简　　写	意　　义
noclobber	C	防止重定向时覆盖文件
allexport	a	export 所有已定义的变量
norify	b	后台作业运行结束时，发送通知
errexit	e	当脚本发生第一个错误时，退出脚本
noglob	f	禁止文件名扩展，即禁用通配（globbing）

续表

选项名称	简　　写	意　　义
interactive	i	使脚本以交互模式运行
noexec	n	读取脚本中的命令，进行语法检查，但不执行这些命令
POSIX	o posix	修改 bash 及其调用脚本的行为，使其符合 POSIX 标准
privileged	p	以 suid 身份运行脚本
restricted	r	以受限模式运行脚本
stdin	s	从标准输入（stdin）中读取命令
nounset	u	当使用未定义变量时，输出错误信息，并强制退出
verbose	v	在执行每个命令之前，将每个命令打印到标准输出（stdout）
xtrace	x	与 verbose 相似，但是打印完整命令
无	D	列出双引号内以$为前缀的字符串，但不执行脚本中的命令
无	c …	从…中读取命令
无	t	第一条命令执行结束就退出
无	-	选项结束标志，后面跟上位置参数（positional parameter）

14. SHLVL

SHLVL 记录了 bash Shell 嵌套的层次，一般来说，我们启动第一个 Shell 时，$SHLVL=1，如果在这个 Shell 中执行脚本，脚本中的 SHLVL 为 2，如果脚本再执行子脚本，子脚本中的 SHLVL 就变为 3。

15. TMOUT

TMOUT 变量用于设置 Shell 的过期时间，当 TMOUT 不为 0 时，Shell 在 TMOUT 秒后将自动注销。TMOUT 放在脚本中，可以规定脚本的执行时间，下面的例 9-13 为 timedread.sh 脚本给出一个 TMOUT 变量用法的例子。

```
#例 9-13: timedread.s 脚本等待用户输入 3 秒
#!/bin/bash

TMOUT=3                                    #脚本执行时间是 3 秒
echo "What is your name?"
read fname

if [ -z "$fname" ]                         #如果 fname 为空
then
   fname="(no answer)"
fi

echo "Your name is $fname"
```

timedread.sh 脚本向用户询问姓名，如果用户有输入，则脚本立即结束，如果用户没有输入，等待 TMOUT 秒后，脚本运行结束。下面给出 timedread.sh 脚本的执行结果。

```
#例 9-13 timedread.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH09$ chmod u+x timedread.sh
alloy@alloy:~/linuxshell/CH09$ ./timedread.sh
What is your name?                         #在此不输入任何字符，3 秒钟后出现如下结果
```

```
Your name is (no answer)
alloy@alloy:~/linuxshell/CH09$
```

9.2 字符串处理

bash Shell 提供了多种字符串处理的命令，归结起来有两种，第一种方法是第 4 章介绍的 awk 命令，第二种方法是本节即将要介绍的 expr 命令。种类如此多的字符串处理命令一方面使得用户的选择空间增大，具有较大的灵活性，另一方面，这导致命令语法不一致，引起不必要的冗余。

expr 是 Linux 中一个功能十分强大的命令，它引出通用求值表达式，可以实现算术操作、比较操作、字符串操作和逻辑操作等功能，本节着重介绍 expr 的字符串操作功能。

1. ${#...}和 expr length

bash Shell 提供多种计算字符串长度的方法，如 4.4.7 节介绍的 awk 的 length(s)函数，在此，再介绍两种方法计算字符串长度，假设字符串名为 string，两种方法的命令如下：

```
${#string}
expr length $string
```

例 9-14 演示两种计算字符串长度的方法，第 1 条命令使用 echo 显示${#string}的值，这说明 Shell 将{#string}当作变量来处理，其中存储的是 string 字符串的长度，而第 2 条命令用 expr length 得出$string 的长度。

```
#例 9-14：演示计算字符串长度的两种方法
alloy@alloy:~/linuxshell/CH09$ string="Speeding up small jobs in Hadoop"
#第 1 条命令：${#string}是变量，echo 显示其值
alloy@alloy:~/linuxshell/CH09$ echo ${#string}
32
#第 2 条命令：$string 上的双引号必不可少
alloy@alloy:~/linuxshell/CH09$ expr length "$string"
32
alloy@alloy:~/linuxshell/CH09$
```

注意，例 9-14 第 2 条命令中$string 上的双引号必不可少，因为 string 中包含了空格，如果不用双引号将$string 引起，Shell 认为 expr length 后面是带了多个参数，而 expr length 后面只能跟一个参数。因此，Shell 将报 expr length 命令语法错误，如例 9-15 所示。例 9-15 中定义 sstr 变量，其赋值不包含空格，因此，$sstr 可以不用引号引起。

```
#例 9-15：字符串是否用引号引起的区别
alloy@alloy:~/linuxshell/CH09$ string="Speeding up small jobs in Hadoop"
#string 包含空格，不用引号引起，报语法错误
alloy@alloy:~/linuxshell/CH09$ expr length $string
expr: 语法错误
alloy@alloy:~/linuxshell/CH09$ sstr="Speedingup"
#sstr 不包含空格，可以不用引号引起
alloy@alloy:~/linuxshell/CH09$ expr length $sstr
10
alloy@alloy:~/linuxshell/CH09$
```

2. expr index

expr 的索引命令格式为：

```
expr index $string $substring
```

expr 索引命令的功能在字符串$string 上匹配$substring 中字符第一次出现的位置，若在$string 上匹配不到$substring 中的任何字符，expr index 返回 0。

例 9-16 演示 expr index 命令的用法，例中先将 string 的值赋为：Speeding up small jobs in Hadoop，第 1 条命令 substring 为 job，job 在 string 中出现过，第一个出现的字符是 j，位置是 19；第 2 条命令 substring 为 hello，string 中并无 hello 字符串，但是 e、l 和 o 字符都出现过，第一个出现字符是 e，位置是 3；第 3 条命令 substring 为 dp，d 和 p 字符都在 string 中出现过，第一个出现字符是 p，尽管 d 字符在 substring 中是排在 p 前面的，但是 expr index 不管 substring 中的字符次序，仍然按照字符在 string 中的出现位置返回结果，因此，该命令返回 p 字符出现的位置 2；第 4 条命令 substring 为 hh，string 中没有 h 字符，因此，expr index 返回 0，表示没有找到匹配字符。

```
#例 9-16：expr index 命令的用法
alloy@alloy:~/linuxshell/CH09$ string="Speeding up small jobs in Hadoop"
#第 1 条命令：job 都在 string 中出现，返回第 1 个匹配字符 j 的位置 19
alloy@alloy:~/linuxshell/CH09$ expr index "$string" job
19
#第 2 条命令：hello 没有都在 string 中出现，返回第 1 个匹配字符 e 的位置 3
alloy@alloy:~/linuxshell/CH09$ expr index "$string" hello
3
#第 3 条命令：dp 都在 string 中出现，尽管 d 字符在前，但是仍然返回 p 字符出现的位置 2
alloy@alloy:~/linuxshell/CH09$ expr index "$string" dp
2
#第 4 条命令：hh 没有在 string 中出现，返回 0
alloy@alloy:~/linuxshell/CH09$ expr index "$string" hh
0
alloy@alloy:~/linuxshell/CH09$
```

3. expr match

expr match 命令的格式为：

```
expr match $string $substring
```

expr match 命令在 string 的开头匹配 substring 字符串，返回匹配到的 substring 字符串的长度，若 string 开头匹配不到 substring，则返回 0。substring 可以是字符串，也可以是正则表达式。

例 9-17 演示 expr match 命令的用法，string 的值仍旧赋为：Speeding up small jobs in Hadoop，第 1 条命令 substring 为正则表达式 S.*，S.*表示以 S 开头且后面跟任意字符的字符串，它能匹配整个 string 字符串，因此，返回值为 32，等于 string 字符串的长度；第 2 条命令 substring 为普通字符串 Spe，匹配 string 的前 3 个字符，返回值为 3；第 3 条命令 substring 为 small，尽管 small 在 string 中出现，但是未出现在 string 的开头处，因此，expr match 命令视为未曾找到匹配，返回 0。

```
#例 9-17：expr match 命令的用法
alloy@alloy:~/linuxshell/CH09$ string="Speeding up small jobs in Hadoop"
#第 1 条命令：S.*表示以 S 开头且后面跟任意字符的字符串，匹配 string 整个字符串
alloy@alloy:~/linuxshell/CH09$ expr match "$string" S.*
32
#第 2 条命令：Spe 字符串匹配长度是 3
alloy@alloy:~/linuxshell/CH09$ expr match "$string" Spe
```

```
3
#第 3 条命令：尽管 small 在 string 中出现过，但是不在 string 开头处，仍返回 0
alloy@alloy:~/linuxshell/CH09$ expr match "$string" small
0
alloy@alloy:~/linuxshell/CH09$
```

4．抽取子串

与计算字符串长度一样，bash Shell 依然提供两种命令#{...}和 expr 实现抽取子串功能，#{...}命令有两种格式，为：

```
(1) #{string:position}
(2) #{string:position:length}
```

第（1）种格式的命令从名称为$string 的字符串的第$position 个位置开始抽取子串，第（2）种格式命令在第（1）种格式命令的基础上添加了$length 变量，表示从名称为$string 的字符串的第$position 个位置开始抽取长度为$length 的子串。需要注意的是，#{...}格式的命令从 0 开始对名称为$string 的字符串进行标号。例 9-18 演示如何使用${string:position}命令抽取子串，例中第 1 条命令 position 为 0，由于 string 以 0 开始标号，所以，该命令将 string 的整串抽取出来作为子串；第 2 条命令 position 为 10，即从标号为 10 开始抽取子串，标号为 10 实际上是第 11 个字符，为 p。

```
#例 9-18：演示使用${string:position}命令抽取子串
alloy@alloy:~/linuxshell/CH09$ string="Speeding up small jobs in Hadoop"
#第 1 条命令：以 0 标号开始，抽取整个字符串
alloy@alloy:~/linuxshell/CH09$ echo ${string:0}
Speeding up small jobs in Hadoop
#第 2 条命令：从标号 10 开始抽取子串
alloy@alloy:~/linuxshell/CH09$ echo ${string:10}
p small jobs in Hadoop
alloy@alloy:~/linuxshell/CH09$
```

例 9-19 演示使用#{string:position:length}命令抽取字串，例中的命令 position 为 9、length 为 8，即从标号为 9 的字符开始抽取长度为 8 的子串。

```
#例 9-19：演示使用${string:position:length}命令抽取子串
alloy@alloy:~/linuxshell/CH09$ string="Speeding up small jobs in Hadoop"
alloy@alloy:~/linuxshell/CH09$ echo ${string:9:8}
up small
alloy@alloy:~/linuxshell/CH09$
```

${string:position}和#{string:position:length}两条命令都是从 string 的左边开始计数抽取子串，#{...}命令还提供了从 string 右边开始计数抽取子串的功能，有以下两种格式：

```
(1) #{string: -position}                    #冒号和短横线符号之间有一个空格符
(2) #{string:(position)}                    #冒号和左括号之间不必有空格符
```

下面的例 9-20 演示了上述两种格式的命令，例中第 1 条命令冒号和短横线符之间无空格，结果将 string 的整个字符串抽取出来作为子串，与我们预想的结果不符；第 2 条命令用圆括号将-6 括起，结果从右边开始抽取出长度为 6 的字符串作为子串；第 3 条命令与第 1 条命令的区别仅在于冒号和短横线符之间多了空格，结果与第 2 条命令一样，从右边开始抽取出长度为 6 的字符串作为子串。从例 9-20 的例子可以看出，如果使用第（1）种格式的命令，冒号和短横线符之间一定要有空格，否则将抽取整个字符串，若用第（2）种格式的命令，则仅需用圆括号将 position 括起就可从 string 右边开始计数抽取子串。

```
#例 9-20：演示使用从 string 的右边抽取子串
```

```
alloy@alloy:~/linuxshell/CH09$ string="Speeding up small jobs in Hadoop"
#第 1 条命令：冒号和短横线之间无空格，表示抽取整个字符串
alloy@alloy:~/linuxshell/CH09$ echo ${string:-6}
Speeding up small jobs in Hadoop
#第 2 条命令：用圆括号将-6 括起，从右边开始抽取长度为 6 的字符串
alloy@alloy:~/linuxshell/CH09$ echo ${string:(-6)}
Hadoop
#第 3 条命令：冒号和短横线之间有空格，从右边开始抽取长度为 6 的字符串
alloy@alloy:~/linuxshell/CH09$ echo ${string: -6}
Hadoop
alloy@alloy:~/linuxshell/CH09$
```

除了上面所说的#{...}命令之外，expr substr 也能够实现抽取子串功能，该命令的格式为：

```
expr substr $string $position $length
```

该命令表示从名称为$string 的字符串的第$position 个位置开始抽取长度为$length 的子串，expr substr 命令与#{...}命令最大的不同之处在于 expr substr 命令是从 1 开始对名称为$string 的字符串进行标号的。另外，expr substr 命令中的$length 是必不可少的，如果缺少$length 参数，Shell 将报 expr 语法错误，如例 9-21 所示。

```
#例 9-21：缺少$length 参数的 expr substr 命令
alloy@alloy:~/linuxshell/CH09$ string="Speeding up small jobs in Hadoop"
alloy@alloy:~/linuxshell/CH09$ expr substr "$string" 1   #缺少$length 参数，报语法错误
expr: 语法错误
alloy@alloy:~/linuxshell/CH09$
```

下面的例 9-22 给出${string:position:length}和 expr substr $string $position $length 命令在同样的 position 和 length 参数值下的执行结果，可以看到，${string:position:length}以 0 开始标号，第 8 个字符是 p，而 expr substr $string $position $length 以 1 开始标号，第 8 个字符为 S。

```
#例 9-22：${…}和 expr substr 命令的比较结果
alloy@alloy:~/linuxshell/CH09$ string="Speeding up small jobs in Hadoop"
#第 1 条命令：以 0 开始标号，第 8 个字符是 p
alloy@alloy:~/linuxshell/CH09$ echo ${string:1:8}
peeding
#第 2 条命令：以 1 开始标号，第 8 个字符是 S
alloy@alloy:~/linuxshell/CH09$ expr substr "$string" 1 8
Speeding
alloy@alloy:~/linuxshell/CH09$
```

接下来，介绍使用正则表达式抽取子串的命令，使用正则表达式只能抽取 string 开头处或结尾处的子串，抽取 string 开头处子串的命令有如下两种格式：

```
(1) expr match $string '\($substring\)'
(2) expr $string  :  '\($substring\)'                    #冒号前后都有一个空格
```

例 9-23 演示如何抽取字符串开头处的子串，该例子中 another_string 的值赋为：20091114 Reading Hadoop，它的开头处是一串数字，例中 4 条命令都将 substring 赋值为：[0-9]*，该正则表达式意义为 0～9 之间任意重复的数字。例 9-23 中第 1 条命令使用第（1）种格式，即 expr match 格式，结果抽取出 another_string 开头的一串数字作为子串；第 2 条命令使用第（2）种格式，并在冒号前后都加上了空格，结果得到正确结果；第 3、4 条命令演示了冒号前后缺少空格所引起的错误，可以看到，无论是冒号之前还是之后缺少空格，Shell 都将报语法错误。

```
#例 9-23：演示如何抽取 another_string 开头处的子串
alloy@alloy:~/linuxshell/CH09$ another_string="20091114 Reading Hadoop"
```

```
#第 1 条命令：第（1）种格式
alloy@alloy:~/linuxshell/CH09$ expr match "$another_string" '\([0-9]*\)'
20091114
#第 2 条命令：第（2）种格式，注意：冒号前后都有一个空格
alloy@alloy:~/linuxshell/CH09$ expr "$another_string" : '\([0-9]*\)'
20091114
#第 3 条命令：冒号前缺少空格
alloy@alloy:~/linuxshell/CH09$ expr "$another_string": '\([0-9]*\)'
expr: 语法错误
#第 4 条命令：冒号后缺少空格
alloy@alloy:~/linuxshell/CH09$ expr "$another_string" :'\([0-9]*\)'
expr: 语法错误
alloy@alloy:~/linuxshell/CH09$
```

抽取 string 结尾处子串的命令同样有两种格式，如下：

```
(1) expr match $string '.*\($substring\)'
(2) expr $string  :  '.*\($substring\)'              #冒号前后都有一个空格
```

相比于抽取 string 开头处的子串，抽取 string 结尾处子串的命令在\(之前多了.*符号，我们可以将.*\($substring\)理解成正则表达式，.*表示任意字符的任意重复，$substring 为正则表达式，因此，.*\($substring\)的意义为在任意字符的任意重复后（即字符串结尾处）抽取满足 $substring 的子串。

例 9-24 演示了如何抽取字符串结尾处的子串，$substring 为 6 个句点，表示 6 个任意字符，因此，例中命令的意义都为抽取 string 结尾处的任意 6 个字符作为子串。

```
#例 9-24：演示如何抽取 another_string 结尾处的子串
alloy@alloy:~/linuxshell/CH09$ another_string="20091114 Reading Hadoop"
alloy@alloy:~/linuxshell/CH09$ expr match "$another_string" '.*\(......\)'
Hadoop
alloy@alloy:~/linuxshell/CH09$ expr "$another_string" : '.*\(......\)'
Hadoop
alloy@alloy:~/linuxshell/CH09$
```

5. 删除子串

与抽取子串相反，删除子串是指将原字符串中符合条件的子串删除，删除子串命令只有 ${...}格式的，但是，删除子串命令可以分为从开头处删除和从结尾处删除两种不同的命令。首先，介绍从开头处删除子串的命令，有如下两种格式：

```
(1) ${string#substring}                #删除 string 开头处与 substring 匹配的最短子串
(2) ${string##substring}               #删除 string 开头处与 substring 匹配的最长子串
```

上述两种格式命令的功能是不同的，第（1）种格式命令的功能是删除开头处与 substring 匹配的最短子串，即 string 开头处一旦与 substring 匹配就立即删除；第（2）种格式命令的功能是删除开头处与 substring 匹配的最长子串，即 string 开头处直到无法再与 substring 匹配时才删除。

例 9-25 演示上述两种格式命令的用法，第 1 条命令删除 string 开头字符 2 和 1 之间的最短匹配，结果匹配到 1114 的第一个 1；第 2 条命令删除 string 开头字符 2 和 1 之间的最长匹配，结果匹配到 1114 的最后一个 1。两条命令中的*字符表示了任意字符，需要注意的是，删除子串命令的 substring 并非是正则表达式，仅表示从起始字符到终止字符，*字符表示起始字符和终止字符之间的任意字符，这也正是*字符在此环境下的特殊用法。

```
#例 9-25：删除 another_string 开头处与 substring 匹配的最短子串和最长子串
alloy@alloy:~/linuxshell/CH09$ another_string="20091114 Reading Hadoop"
```

```
#第 1 条命令：删除 another_string 开头处 2 和 1 之间的最短匹配
alloy@alloy:~/linuxshell/CH09$ echo ${another_string#2*1}
114 Reading Hadoop
#第 2 条命令：删除 another_string 开头处 2 和 1 之间的最长匹配
alloy@alloy:~/linuxshell/CH09$ echo ${another_string##2*1}
4 Reading Hadoop
alloy@alloy:~/linuxshell/CH09$
```

删除子串还可以从 string 的结尾处开始删除，有与上述开头处删除子串命令格式相对应的两种命令格式：

```
(1) ${string%substring}          #删除 string 结尾处与 substring 匹配的最短子串
(2) ${string%%substring}         #删除 string 结尾处与 substring 匹配的最长子串
```

由上面两条命令可以看出，从 string 开头处或结尾处删除子串的不同仅在于#和%两个符号不同。例 9-26 演示上述两种格式命令的用法，第 1 条命令删除 string 结尾处字符 a 和 p 之间的最短匹配，结果删除了 adoop 子串；第 2 条命令删除 string 结尾处字符 a 和 p 之间的最长匹配，结果删除了 ading Hadoop 子串。

```
#例 9-26：删除 another_string 结尾处与 substring 匹配的最短子串和最长子串
alloy@alloy:~/linuxshell/CH09$ another_string="20091114 Reading Hadoop"
#第 1 条命令：删除 another_string 结尾处 a 和 p 之间的最短子串
alloy@alloy:~/linuxshell/CH09$ echo ${another_string%a*p}
20091114 Reading H
#第 2 条命令：删除 another_string 结尾处 a 和 p 之间的最长子串
alloy@alloy:~/linuxshell/CH09$ echo ${another_string%%a*p}
20091114 Re
alloy@alloy:~/linuxshell/CH09$
```

6. 替换子串

替换子串命令都是${...}格式，可以在任意处（包括开头处和结尾处）替换满足条件的子串。首先介绍在任意处替换子串的命令，如下两条格式的命令：

```
(1) ${string/substring/replacement}        #仅替换第一次与 substring 相匹配的子串
(2) ${string//substring/replacement}       #替换所有与 substring 相匹配的子串
```

第（1）种格式的命令仅替换第一次与 substring 相匹配的子串，替换内容为 repacement 子串，第（2）种格式在 string 和 substring 之间比第（1）种格式多出一条斜杠，表示替换所有与 substring 相匹配的子串，替换内容同样为 repacement 子串。

例 9-27 演示上述两种格式命令的用法，第 1 条命令为第（1）种格式，仅将第一个 111 子串用 zzzz 子串进行替换，由例中结果也可看出，第二个 111 子串未被替换，第 1 条命令为第（2）种格式，它将所有的 111 子串替换为 zzzz 子串。

```
#例 9-27：替换与 substring 匹配的子串
alloy@alloy:~/linuxshell/CH09$ string="20091115sunday20091116tomorrow"
#第 1 条命令：仅替换第 1 次出现 111 的地方，第 2 次出现 111 的地方不做修改
alloy@alloy:~/linuxshell/CH09$ echo ${string/111/zzzz}
2009zzzz5sunday20091116tomorrow
#第 2 条命令：替换所有出现 111 的地方
alloy@alloy:~/linuxshell/CH09$ echo ${string//111/zzzz}
2009zzzz5sunday2009zzzz6tomorrow
alloy@alloy:~/linuxshell/CH09$
```

与删除子串一样，替换子串中的 substring 不是正则表达式，但是，*字符具有特殊的意义，表示起始字符和终止字符之间的任意字符，例 9-28 演示了*字符在替换命令中的用法，substring 为 0*y，表示从 0 字符开始到 y 字符结束的字符串，结果将 0091115sunday 替换为

ZZZZ。

```
#例 9-28：*字符在替换子串命令中的用法
alloy@alloy:~/linuxshell/CH09$ string="20091115sunday20091116tomorrow"
alloy@alloy:~/linuxshell/CH09$ echo ${string/0*y/zzzz}
2zzzz20091116tomorrow
alloy@alloy:~/linuxshell/CH09$
```

替换子串命令还有两种格式，分别在 string 开头处和结尾处替换与 substring 相匹配子串，命令格式为：

```
(1) ${string/#substring/replacement}          #替换 string 开头处与 substring 相匹配的子串
(2) ${string/%substring/replacement}          #替换 string 结尾处与 substring 相匹配的子串
```

上述两种格式命令的区别仅在于第（1）种格式斜杠后为#符号，而第（2）种格式斜杠后为%符号，例 9-29 演示替换开头处和结尾处子串命令的用法，例中第 1 条命令利用第（1）种格式命令替换开头处子串，将 2009 替换为 YEAR；例中第 2 条命令利用第（2）种格式命令替换结尾处子串，用 t*w 表示从 t 字符开头到 w 字符结束的字符串，结果将结尾处的 tomorrow 替换为 YESTADAY。

```
#例 9-29：替换开头处和结尾处子串的命令
alloy@alloy:~/linuxshell/CH09$ string="20091115sunday20091116tomorrow"
#第 1 条命令：替换开头处的子串
alloy@alloy:~/linuxshell/CH09$ echo ${string/#2009/YEAR}
YEAR1115sunday20091116tomorrow
#第 2 条命令：替换结尾处的子串
alloy@alloy:~/linuxshell/CH09$ echo ${string/%t*w/YESTADAY}
20091115sunday20091116YESTADAY
alloy@alloy:~/linuxshell/CH09$
```

9.3 有类型变量

Shell 变量一般是无类型的，但是 bash Shell 提供了 declare 和 typeset 两个命令用于指定变量的类型，两个命令是完全等价的，declare 命令在 bash 2.0 版本后被引入，typeset 命令也适用于 Korn Shell。由于这两个命令完全等价，因此，本节仅以 declare 命令为例来解释这两个命令的用法。declare 命令的格式为：

```
declare [选项] 变量名
```

declare 命令有 6 个选项，表 9-2 给出了 declare 命令的选项及其意义。

表 9-2 declare 命令的选项及其意义

选 项 名	意 义
-r	将变量设置为只读属性
-i	将变量定义为整型数
-a	将变量定义为数组
-f	显示此脚本前定义过的所有函数名及其内容
-F	仅显示此脚本前定义过的所有函数名
-x	将变量声明为环境变量

declare 命令的-r 选项将变量设置为只读属性，其功能与 6.1.1 节介绍的 readonly 命令完全一样，变量被设置为只读后，变量值不允许再被修改。

declare 命令的-i 选项将变量定义为整型数，一旦变量被定义为整型数，Shell 不再按照字符串形式来处理该变量，允许使用该变量进行算术运算。下面的例 9-30 说明了 declare 命令的-i 选项，同时介绍了 let 命令的意义。新建名为 vartype.sh 的脚本，内容如下：

```
#例 9-30：vartype.sh 脚本说明 declare -i 命令和 let 命令
#!/bin/bash

variable1=2009
variable2=$variable1+1                    #以字符型处理 variable2
echo  "variable2=$variable2"

let variable3=$variable1+1                #let 命令以整型数处理 variable3
echo  "variable3=$variable3"

declare -i variable4                      #将 variable4 定义为整型
variable4=$variable1+1
echo  "variable4=$variable4"
```

下面给出 vartype.sh 脚本的执行结果，vartype.sh 脚本首先定义 variable1，赋值为 2009，然后定义 variable2，用表达式$variable1+1 赋值。从结果可以看到，variable2=2009+1，这说明 Shell 将 variable2 当作字符串来处理，并没有进行加法运算。接着，vartype.sh 脚本定义 variable3，利用 let 命令将其赋为$variable1+1，从结果可以看到，variable3=2010，这说明此时 Shell 将 variable3 当作整型来处理，执行了加法运算。最后，vartype.sh 脚本用 declare -i 命令将 variable4 定义为整型数，然后用表达式$variable1+1 赋值，结果显示，variable4=2010，这说明 Shell 同样将 variable4 当作整型数处理，执行加法运算。

```
#例 9-30 vartype.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH09$ chmod u+x vartype.sh
alloy@alloy:~/linuxshell/CH09$ ./vartype.sh
variable2=2009+1
variable3=2010
variable4=2010
alloy@alloy:~/linuxshell/CH09$
```

由 vartype.sh 脚本可以看出，declare -i 命令可以将变量定义为整型，在默认情况下就进行算术运算。如果不使用 declare -i 命令，在算术运算时需要使用 let 命令，顺便提及，expr 命令可以替换 let 命令，使后面的表达式进行算术运算。

在此我们再介绍一种使变量执行算术运算的方法——双圆括号方法，即((...))格式。例 9-31 的 doubleparenthese.sh 脚本演示了双圆括号的用法，doubleparenthese.sh 脚本的内容如下：

```
#例 9-31：doubleparenthese.sh 脚本演示双圆括号的用法
#!/bin/bash

variable1=12
variable2=5

result=$((variable1*variable2))      #用双圆括号括起使变量进行算术运算
echo "result=$result"
```

doubleparenthese.sh 脚本首先定义 variable1 和 variable2，然后用 result=$((variable1*

variable2))语句为 result 变量赋值，Shell 将双圆括号内的表达式解析为算术运算，下面给出了例 9-31 中 doubleparenthese.sh 脚本的执行结果，我们可以看到，result=60，这表明 result 的值确实是 variable1 和 variable2 执行乘法运算后的结果。

```
#例 9-31 doubleparenthese.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH09$ chmod u+x doubleparenthese.sh
alloy@alloy:~/linuxshell/CH09$ ./doubleparenthese.sh
result=60
alloy@alloy:~/linuxshell/CH09$
```

双圆括号方法的另一功能是使 bash Shell 实现了 C 语言风格的变量操作，例 9-32 说明了这点，新建名为 cstyle.sh 的脚本，内容如下：

```
#例 9-32：cstyle.sh 脚本利用双圆括号方法实现 C 语言风格的变量操作
#!/bin/bash

((a = 2009))                              #等于符号两端各有一个空格
echo "The initial value of a is:$a"

#自加、自减是 Shell 算术运算符中未曾定义过的，C 语言中有
((a++))
echo "After a++,the value of a is:$a"
((++a))
echo "After ++a,the value of a is:$a"
((a--))
echo "After a--,the value of a is:$a"
((--a))
echo "After --a,the value of a is:$a"
```

cstyle.sh 脚本在双圆括号内实现 5 种 C 语言风格的运算，首先是 C 语言风格的赋值，C 语言允许赋值号两端存在空格，但是，这在 Shell 中是不允许的。其次，变量 a 可以使用 C 语言中定义的自加和自减符号，这两个符号在 Shell 的算术运算符中是未曾定义过的。下面给出 cstyle.sh 脚本的执行结果，可以看到，a 确实被成功地赋值，并成功实现自加和自减运算。

```
#例 9-32 cstyle.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH09$ chmod u+x cstyle.sh
alloy@alloy:~/linuxshell/CH09$ ./cstyle.sh
The initial value of a is:2009
After a++,the value of a is:2010
After ++a,the value of a is:2011
After a--,the value of a is:2010
After --a,the value of a is:2009
alloy@alloy:~/linuxshell/CH09$
```

当然，双圆括号方法还可以实现更加复杂的 C 语言风格的运算，如逻辑判断、三元操作等，读者可以执行本章上机提议第 9 题给出的脚本。

declare 命令的-a 选项将变量声明为数组类型，Shell 将按照数组来处理该变量，关于数组我们将在第 14 章详细介绍，在此不再展开论述。

declare 命令的-f 选项有两种格式，第（1）种格式不带任何参数，第（2）种格式带函数名，具体格式如下：

```
(1) declare -f
(2) declare -f function-name
```

declare -f 命令的第（1）种格式显示此脚本前定义过的所有函数名及内容，而第（2）种格式仅显示 function-name 函数名及其内容。

declare 命令的-F 选项只显示函数名，不显示函数的内容。因此，declare -F 命令后不再需要添加任何参数。

Shell 编程中的函数将在第 13 章详细介绍，在此仅介绍 declare 的-f 和-F 选项的意义，并不对函数展开介绍。

declare 命令的-x 选项将变量声明为环境变量，相当于 export 命令，但是，declare -x 允许在声明变量为环境变量的同时给变量赋值，而 export 命令不支持此功能。即如下格式的 declare 命令是成立的。

```
declare -x variable-name=value
```

9.4 间接变量引用

如果第一个变量的值是第二个变量的名字，从第一个变量引用第二个变量的值就称为间接变量引用。设有如下两个表达式：

```
variable1=variable2
variable2=value
```

variable1 的值是 variable2，而 variable2 又是变量名，variable2 的值为 value，间接变量引用是指通过 variable1 获得变量值 value 的行为，bash Shell 提供了两种格式实现间接变量引用：

```
(1) eval tempvar=\$$variable1
(2) tempvar=${!variable1}
```

第（1）种格式中 eval 是关键字，用\$$形式得到 variable1 的间接引用，保存在 tempvar 变量中，第（2）种格式用${!...}得到 variable1 的间接引用，并赋给 tempvar。下面的例 9-33 说明了以上两种格式命令，新建名为 indirect.sh 的脚本文件，内容如下：

```
#例 9-33：indirect.sh 脚本演示间接变量引用
#!/bin/bash

variable1=variable2                    #定义 variable1 变量，它的值为 variable2
variable2=Hadoop                       #variable2 又是一变量名，值为 Hadoop

echo "varialbe1=$variable1"            #直接引用 variable1

eval tempvar=\$$variable1              #用第（1）种格式命令间接引用 variable1
echo "tempvar=$tempvar"

#用第（2）种格式命令间接引用 variable1
echo "Indirect ref variable1 is :${!variable1}"
```

indirect.sh 脚本首先定义 variable1 变量，其值为 variable2，而 variable2 又是一个变量名，其值为 Hadoop。indirect.sh 脚本分别输出直接引用 variable1、用第（1）种格式命令间接引用 variable1、用第（2）种格式命令间接引用 variable1 的结果。下面给出 indirect.sh 脚本的执行结果。

```
#例 9-33 indirect.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH09$ chmod u+x indirect.sh
alloy@alloy:~/linuxshell/CH09$ ./indirect.sh
varialbe1=variable2                          #直接引用 variable1 的结果
tempvar=Hadoop                               #间接引用 variable1 的结果
Indirect ref variable1 is :Hadoop            #间接引用 variable1 的结果
alloy@alloy:~/linuxshell/CH09$
```

从例 9-33 的结果可以看出，直接引用 variable1 的结果为 variable2，间接引用 variable1 的结果为 Hadoop。

我们再查看 indirect.sh 脚本中两种间接引用命令的使用方法，第（1）种格式中，由于 eval 关键字的存在，必须引入临时变量 tempvar 保存间接引用结果，而第（2）种格式则可以直接使用${!variable1}得到 variable1 的间接引用。因此，第（2）种格式的间接引用比第（1）种格式直观且更加便于使用。

接下来，我们举一个间接变量引用的例子，该例利用间接变量引用实现数据库表格的查找，新建名为 indirectapp1.sh 的脚本，内容如下：

```
#例 9-34：indirectapp1.sh 脚本说明间接变量引用的应用
#!/bin/bash

############################数据库表格数据#####################################
S01_name="Li Hao"
S01_dept=Computer
S01_phone=025-83481010
S01_rank=5

S02_name="Zhang Ju"
S02_dept=English
S02_phone=025-83466524
S02_rank=8

S03_name="Zhu Lin"
S03_dept=Physics
S03_phone=025-83680010
S03_rank=3
############################数据库表格数据#####################################

#设置 3 级 Shell 提示符变量，改变 select 命令的提示符
PS3='Pls. select the number of student:'

#用 select 建立选择菜单，供用户选择学号
select stunum in "S01" "S02" "S03"
do
  #将输入的学号组合成名字、系名、电话和排名的变量名
name=${stunum}_name
  dept=${stunum}_dept
  phone=${stunum}_phone
  rank=${stunum}_rank

  #通过间接变量引用得到学生的信息
  echo "BASIC INFORMATION OF NO.$stunum STUDENT:"
  echo "NAME:${!name}"
  echo "DEPARTMENT:${!dept}"
```

```
    echo "PHONE:${!phone}"
    echo "RANK:${!rank}"
    break
  done
```

indirectapp1.sh 脚本首先将数据库中的表格数据赋给一些变量，这些变量以数据库主键学生的学号开头，下画线后面跟该学生具体信息的名称，如 S01_name 表示学号为 S01 的学生的姓名、S02_phone 表示学号为 S02 的学生的电话号码。然后，indirectapp1.sh 脚本用 select 命令新建菜单，供用户选择需要查询的学号，stunum 变量保存 S01、S02 或 S03，再通过 stunum 与_name、_dept、_phone 和_rank 字符串组合成 S01_name、S01_phone 和 S02_phone 等变量，显然，name 变量的值为 S01_name、S02_name 或 S03_name。最后，indirectapp1.sh 脚本间接引用 name、dept、phone 和 rank 变量，得到用户所选择的那个学生的相应信息。下面给出 indirectapp1.sh 脚本的执行结果。

```
#例 9-34 indirectapp1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH09$ chmod u+x indirectapp1.sh
alloy@alloy:~/linuxshell/CH09$ ./indirectapp1.sh
1) S01
2) S02
3) S03                                          #以上是 select 命令自动生成的菜单
Pls. select the number of student:2             #用户输入，本例中为 2
#以下是由间接变量信息反馈与用户输入相对应的信息
BASIC INFORMATION OF NO.S02 STUDENT:
NAME:Zhang Ju
DEPARTMENT:English
PHONE:025-83466524
RANK:8
alloy@alloy:~/linuxshell/CH09$
```

从例 9-34 的结果可以看出，运行 indirectapp1.sh 脚本后，Shell 弹出由 select 命令自动生成的菜单，等待用户在由 PS3 所设置的 3 级 Shell 提示符“Pls. select the number of student:”后输入选择，此时输入的是 2，Shell 反馈出 S02 学生的信息。此时，stunum 变量值为 S02、name 变量值为 S02_name、dept 变量值为 S02_dept、phone 变量值为 S02_phone、rank 变量值为 S02_rank，indirectapp1.sh 脚本通过间接引用变量 name、dept、phone 和 rank，分别得到 Zhang Ju、English、025-83466524 和 8。

从另一个角度理解，在间接变量引用中，第一个变量中存放的是最终值的地址，因此，间接变量引用使得 Shell 编程实现了类似于高级程序语言中指针的功能，在很多场合能够精简 Shell 脚本。

9.5 bash 数学运算

在编写程序的过程中经常会涉及数学运算，而 9.3 节指出 bash Shell 的变量只有整型和字符型之分，没有浮点型。那么在 Shell 脚本编程中如何实现浮点型运算呢？本节将要解决这一问题。

9.5.1 expr 命令

expr 命令对读者来说已经不陌生，它在整型数运算和字符串处理中都曾经出现，尽管如此，我们依然有必要总结一下 expr 命令的用法，表 9-3 列出了 expr 操作符的名称及其意义。

表 9-3 expr 命令的操作符及其意义

操 作 符	意 义
ARG1 | ARG2	当 ARG1 不为空且非零时，返回 ARG1；否则返回 ARG2
ARG1 & ARG2	当 ARG1 不为空且非零时，返回 ARG1；否则返回 0
ARG1 < ARG2	当 ARG1 小于 ARG2 时，返回 1；否则返回 0
ARG1 <= ARG2	当 ARG1 小于或等于 ARG2 时，返回 1；否则返回 0
ARG1 = ARG2	当 ARG1 等于 ARG2 时，返回 1；否则返回 0
ARG1 != ARG2	当 ARG1 不等于 ARG2 时，返回 1；否则返回 0
ARG1 >= ARG2	当 ARG1 大于或等于 ARG2 时，返回 1；否则返回 0
ARG1 > ARG2	当 ARG1 大于 ARG2 时，返回 1；否则返回 0
ARG1 + ARG2	返回 ARG1 和 ARG2 之和
ARG1 - ARG2	返回 ARG1 和 ARG2 之差
ARG1 * ARG2	返回 ARG1 和 ARG2 之积
ARG1 / ARG2	返回 ARG1 除以 ARG2 的商数
ARG1 % ARG2	返回 ARG1 除以 ARG2 的余数

当 expr 命令的操作符是普通字符时，根据表 9-3 中的格式进行运算就能得出结果。但是，当 expr 命令的操作符是元字符，即该操作符还有其他含义时，需要用转义符将操作符的特殊含义屏蔽，方能使 expr 成功执行数学运算。下面的例 9-35 演示了 expr 操作符是元字符的情况。

```
#例 9-35：演示 expr 操作符是元字符
#第 1 条命令：*字符是元字符，不用转义符屏蔽*的特殊含义将报语法错误
alloy@alloy:~/linuxshell/CH09$ expr 2010 * 2
expr: 语法错误
#第 2 条命令
alloy@alloy:~/linuxshell/CH09$ expr 2010 \* 2
4020
alloy@alloy:~/linuxshell/CH09$
```

例 9-35 中，expr 操作符是*符号，由于*字符是元字符，第 1 条命令不用转义符屏蔽*的特殊含义时，Shell 报语法错误；第 2 条命令在*字符前加上转义符，成功得到运算结果。

需要提醒读者注意的一点是：expr 操作符两端必须有空格，否则将不执行数学运算，请看下面的例 9-36。

```
#例 9-36：expr 操作符两端必须有空格
#第 1 条命令：-号两端无空格时，没有执行数学运算
alloy@alloy:~/linuxshell/CH09$ expr 2010-2009
2010-2009
#第 2 条命令：-号两端加上空格，得到运算结果
alloy@alloy:~/linuxshell/CH09$ expr 2010 - 2009
1
alloy@alloy:~/linuxshell/CH09$
```

例 9-36 的第 1 条命令：-号两端无空格时，没有执行数学运算；第 2 条命令在“-”号两端加上空格，得到运算结果。

9.5.2 bc 运算器

bc 是一种内建的运算器，是 bash Shell 中最常用的浮点数运算工具，bc 能包含下面的一些元素：

- 整型数和浮点数。
- 简单变量和数组变量。
- C 语言风格的注释（/*...*/格式）。
- 表达式。
- 复杂程序结构，如 if-then 结构等。
- 函数。

首先，举一个简单的例子说明 bc 在命令行中的用法，如例 9-37 所示。

```
#例 9-37：演示 bc 在命令行中的用法
alloy@alloy:~/linuxshell/CH09$ bc                    #输入 bc 命令，启动 bc 运算器
bc 1.06.95
Copyright 1991-1994, 1997, 1998, 2000, 2004, 2006 Free Software Foundation, Inc.
This is free software with ABSOLUTELY NO WARRANTY.
For details type `warranty'.
#以上是 bc 版本信息，从下面开始输入运算表达式
3.1415 * 3 * 3 /*Compute the area of a circle*/
28.2735
r=3
3.1415 * r * r
28.2735
13 / ( 3 + 4)
1
quit
alloy@alloy:~/linuxshell/CH09$
```

例 9-37 首先输入 bc 命令，启动 bc 运算器，Shell 上显示 bc 版本信息，bash 4.0 的是 bc 1.06 版本。在版本信息下方，可以输入运算表达式，例 9-37 计算一个半径为 3 的圆面积，输入“3.1415 * 3 * 3 /*Compute the area of a circle*/”，“/**/”内是注释，bc 返回浮点数结果 28.2735；接着，例 9-37 又以变量的方式计算圆面积，将 r 变量赋为 3，可以调用 r 进行计算；最后，输入“quit”命令可以退出 bc 运算器。

bc 运算器支持的数学运算符号如表 9-4 所示。

表 9-4 bc 运算器的操作符及其意义

操 作 符	意 义
ARG1 + ARG2	返回 ARG1 和 ARG2 之和
ARG1 - ARG2	返回 ARG1 和 ARG2 之差
ARG1 * ARG2	返回 ARG1 和 ARG2 之积
ARG1 / ARG2	返回 ARG1 除以 ARG2 的商数（按 scale 变量设定结果的小数位数）
ARG1 % ARG2	返回 ARG1 除以 ARG2 的余数

续表

操 作 符	意 义
ARG1 ^ ARG2	返回 ARG1 的 ARG2 次方，指数运算
++ ARG 或 ARG ++	自加操作
-- ARG 或 ARG --	自减操作

bc 运算器定义了内建变量 scale 用于设定除法运算的精度，默认情况下，scale 变量等于 0，因而，除法运算的结果将自动取整。下面的例 9-38 演示了 scale 变量的作用：

```
#例 9-38：演示 bc 运算器 scale 变量的作用
alloy@alloy:~/linuxshell/CH09$ bc -q      #-q 选项使得 bc 运算器不输出版本信息
13 / (3 + 4)
1                                         #未设置 scale 之前，除法运算结果自动取整
scale=4                                   #将 scale 设为 4
13 / (3 + 4)
1.8571                                    #除法结果小数点后保留 4 位
quit
alloy@alloy:~/linuxshell/CH09$
```

例 9-38 未设置 scale 之前，除法运算结果自动取整；当将 scale 设为 4 后，除法结果小数点后保留 4 位。bc 命令后加上-q 选项使得 bc 运算器不输出版本信息，这使得 bc 运算器更加简洁。

例 9-37 和例 9-38 解释清楚了如何在命令行中使用 bc 运算器。实际上，我们经常需要在 Shell 脚本中进行浮点数运算，那么如何在脚本中使用 bc 运算器呢？幸运的是，命令替换可以帮助我们方便地将 bc 运算器的结果赋给一个 Shell 变量，在脚本中调用 bc 运算器的一般格式为：

```
variable=`echo "options; expression" | bc`
```

上述命令中，variable 是变量名，echo 命令后 options 部分一般用于设置 scale 变量，expression 是数学运算表达式，echo 的结果通过管道符传输给 bc 命令，这样就将 expression 的结果赋给了 variable 变量。下面的例 9-39 说明了 Shell 脚本调用 bc 运算器，新建 calare.sh 脚本，内容如下：

```
#例 9-39：calare.sh 脚本用 bc 运算器计算圆面积
#!/bin/bash

var1=20
var2=3.14159
var3=`echo "scale=5; $var1 ^ 2 " | bc`          #计算半径的平方
var4=`echo "scale=5; $var3 * $var2" | bc`       #计算圆面积
echo "The area of this circle is:$var4"
```

例 9-39 的 calare.sh 脚本用 bc 运算器计算圆面积，首先利用 bc 的指数运算，计算出 var1 变量的平方，赋给 var3；接着，将 var3 和 var2 相乘得出圆面积；calare.sh 脚本将 scale 变量设为 5，即输出结果小数点后精确到第 5 位。下面给出 calare.sh 脚本的执行结果：

```
#例 9-39 calare.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH09$ chmod u+x calare.sh
alloy@alloy:~/linuxshell/CH09$ ./calare.sh
The area of this circle is:1256.63600           #结果的小数点后精确到第 5 位
alloy@alloy:~/linuxshell/CH09$
```

calare.sh 脚本确实实现了浮点数运算，输出结果的小数点后精确到第 5 位。由此可见，将 bc 运算器和命令替换结合可以方便地实现脚本的复杂数学运算。

9.6 本章小结

在第 6 章的基础上，本章探讨了 Linux Shell 变量的高级用法。首先，在第 6 章介绍的几个简单环境变量的基础上全面介绍了 bash Shell 中的内部变量及其应用；其次，介绍了 Shell 处理字符串的命令和方法，它与 awk 编程一起构成了 Shell 编程中字符串处理的全部方法；再次，介绍了 declare 和 typeset 命令，它们将原本无类型的 Shell 变量声明为特定类型；接着还讨论了间接变量引用的方法及其应用；最后，总结了 bash 进行数学运算的方法，整型数运算可以用 expr 命令，但是，浮点运算等复杂运算需要使用 bc 运算器。

9.7 上机提议

1．连续执行以下命令，给出目录栈中元素入栈和出栈的序列，在每条命令后查看 DIRSTACK 的值。

```
pushd /etc
pushd sysconfig
popd
pushd /usr/share
popd
popd
```

2．利用 grep 命令查找某目录下所有以 b 开头的文件中的“#”字符，但是，如果这些以 b 开头的文件以.sh 结尾，则忽略对其进行查找。（提示：结合使用 GLOBIGNORE 变量实现）。

3．将 9.1 节中 selectreply.sh 脚本的 break 关键字去掉，观察执行该脚本后将出现什么情况。然后，将 selectreply.sh 脚本中 select 命令后的 var 去掉，观察执行该脚本后又将出现什么情况。

4．挑选 1～3 个 Shell 选项，分别用选项名及其简写开关该选项，并查看 SHELLOPTS 的值。

5．定义变量 string="Rolling lessons learned from Hadoop into an open source Hadoop successor"，分别实现以下操作：（1）打印 string 的长度；（2）给出 Hadoop 在 string 中的索引；（3）将第 1 个 Hadoop 替换为 MapReduce；（4）将所有的 Hadoop 替换为 MapReduce。

6．用 typeset 命令定义 integervar 为环境变量，并赋值为 1101，然后对 integervar 执行加法操作，并打印结果。

7．执行下面的脚本，比较 result1 和 result2 的值，体会 typeset 命令的作用。

```
#!/bin/bash
var1=12
var2=5
```

```
result1=val1*var2
echo "result1=$result1"

typeset -i var3=15 var4=8
typeset result2=var3*var4
echo " result2=$result2"
```

8．定义 variable1=911、variable2=1101，使用三种方法实现 variable1*variable2 操作。（提示：三种方法为 declare 或 typeset 将 variable1 和 variable2 声明为整型、let 命令、用双圆括号方法。）

9．下列脚本用双圆括号方法实现了更复杂的 C 语言风格的算术运算，执行下面的脚本，观察和分析结果。

```
#!/bin/bash

a=1
let --a && echo " TRUE" || echo " FALSE"
b=1
let b-- && echo " TRUE" || echo " FALSE"

((c = a<4?9:11))
echo "c=$c"
```

10．执行下列脚本，验证间接变量引用的方法。

```
#!/bin/bash
t=cell
var=9999
cell=$var
tempvar=\$$t

echo "Direct refer t:$t"
echo "tempvar=$tempvar"
echo "Indirect refer t:${!t}"
```

11．使用 REPLY 改写 9.4 节的 indirectapp1.sh 脚本，并执行之，巩固 select 命令、REPLY 变量、PS3 变量和间接变量引用等知识。

12．脚本中调用 bc 运算器还可以使用<<delimiter 的方法，格式如下：

```
variable=`bc << EOF
options
expresstons
EOF`
```

请用这种方法改写例 9-39 的 calare.sh 脚本，实现圆面积的计算。

第 10 章

I/O 重定向

I/O 重定向是 Shell 编程中的一个重要议题，用于捕捉一个文件、命令、程序或脚本，甚至代码块的输出，然后把捕捉到的输出作为输入发送给另外一个文件、命令、程序或脚本等。本章首先介绍 I/O 重定向最常用的方法：管道，并结合 sed 和 awk 等命令说明管道的用法；其次介绍面向文件的 I/O 重定向，内容涉及文件标识符、I/O 重定向符号及其用法、exec 命令和代码块重定向等。在此基础上，本章上升到 Shell 设计者的高度介绍 Shell 如何对一个命令行进行处理，首先结合实例详细介绍 Shell 处理命令行的流程，其次举例介绍 Shell 中的一个高级命令：eval 命令。

10.1 管道

管道是 Linux 编程中最常用的技术之一，管道符“|”在前面章节曾经出现过，本节将详细介绍管道技术的概念、用法，以及与其他命令的结合使用等内容。

10.1.1 管道简介

管道技术是 Linux 一种基本的进程间通信技术，它利用先进先出（First In First Out，FIFO）排队模型来指挥进程间的通信。对于管道，我们可以形象地把它们当作是连接两个实体的一个单向连接器。

Linux 管道可用于应用程序之间、Linux 命令之间，以及应用程序和 Linux 命令之间的通信，Shell 编程主要是利用管道进行 Linux 命令之间的通信，本章介绍的重点也在于此。

Shell 编程中管道符号是竖杠符号“|”，命令之间利用管道进行通信的一般格式为：

```
command1 | command2 | command3 | … |commandn
```

command1 到 commandn 表示 Linux 的 n 个命令，这 n 个命令利用管道进行通信。command1 执行完后，如果没有管道，command1 的输出结果将直接显示在 Shell 上，当 Shell 遇到管道符“|”后，就将 command1 的输出发送给 command2，作为 command2 的输入。

例 10-1 演示了管道的基本用法，例中命令的功能是列出/etc 目录的文件列表，在文件列表中用 grep 命令查找与字符串 vi 匹配的行，从结果可以看到，列出的文件中都包含 vi 字符串。

```
#例 10-1：演示管道的基本用法
alloy@alloy:~/linuxshell/CH10$ cd /etc
alloy@alloy:/etc$ ls -l | grep vi    #列出/etc 目录的文件列表，并在其中查找与 vi 匹配的行
-rw-r--r--  1 root root      79  2 月  4 19:58 environment
-rw-r--r--  1 root root   19281  2 月 14  2012 services
drwxr-xr-x  2 root root    4096  3 月 15 13:16 vim
alloy@alloy:/etc$
```

管道也可用于多个命令进行通信，例 10-2 演示了三个命令使用管道进行通信，第 2 个管道符将例 10-1 的结果发送给 wc -l 命令，wc -l 命令用于统计结果的行数。例 10-1 中与 vi 匹配的行有 6 行，因此，例 10-2 的结果为 6。

```
#例 10-2：三个命令用管道进行通信
#列出/etc 目录的文件列表，并在其中查找与 vi 匹配的行，最后对行计数
alloy@alloy:/etc$ ls -l | grep vi | wc -l
3
alloy@alloy:/etc$
```

例 10-2 中命令的执行过程可以用图 10-1 来描述。

图 10-1　Linux 命令的管道通信示意图

如图 10-1 所示，ls -l | grep vi | wc -l 命令实际上就是在三个命令之间建立了两根管道，第一个命令 ls -l 执行后产生的输出作为第二个命令 grep vi 的输入，第二个命令在管道输入下执行后产生的输出又作为第三个命令 wc -l 的输入，最后第三个命令 wc -l 在管道输入下执行命令，并将产生的结果输出到 Shell，图 10-1 清晰地表示出了三条命令之间的传输内容。显然，这是一个半双工通信，因为通信是单向的，两个命令之间连接的具体工作是由 Linux 内核来完成的。

10.1.2 cat 和 more 命令

cat 和 more 命令都用来显示文件的内容，它们的基本格式也比较类似，具体如下：

```
cat [option] fileName
more [option] [+linenum] fileNames
```

上述两条命令中，option 都指选项，cat 命令选项及其含义如表 10-1 所示；more 命令的选项如表 10-2 所示；fileName 都指文件名；more 命令中的+linenum 可以指定显示文件的起始行，即从 linenum 这行开始显示整个文件。

表 10-1 cat 命令的选项名称及其意义

选　项	意　义
-a	显示文件的所有内容
-b	由 1 开始对所有输出的行数编号，但对空白行不编号
-E	每一行末显示“$”
-n	由 1 开始对所有输出的行数编号
-s	当遇到有连续两行以上的空白行时，就替换为一行空白行
-v	不可显示的字符（制表符、新行符和换页符除外）以可见的形式显示

表 10-2 more 命令的选项名称及其意义

选　项	意　义
-num	一次显示的行数
-d	提示使用者，在画面下方显示“Press space to continue, q to quit.”，如果使用者按错键，则会显示“Press h for instructions.”
-l	取消遇见特殊字元 ^L（送纸字元）时会暂停的功能
-f	计算行数时，是实际的行数，而非自动换行后的行数（有些单行字数太长的会被扩展为两行或两行以上）
-p	不以卷动的方式显示每一页，而是先清除屏幕后再显示内容
-c	跟 -p 相似，不同的是先显示内容，再清除其他旧的资料
-s	当遇到有连续两行以上的空白行时，就代换为一行空白行
-u	禁止显示强调符，即不显示下画线（根据环境变量 TERM 指定的 terminal 而有所不同）
+/	在每个文件显示前搜寻该字符串（pattern），然后从该字符串之后开始显示
+num	从第 num 行开始显示

两个命令最大的区别在于：cat 命令在显示文件时不提供分页功能，而 more 命令在显示

超过一页的文件时提供了分页功能。下面举一个例子来说明分页功能所产生的作用，例中使用 more 命令显示/etc/vim/vimrc 文件，输入命令后产生如下输出，其中没有显示/etc/vim/vimrc 文件的全部内容，而只显示一页内容，实际上是一屏内容，屏幕大则显示内容多，最后一行提示--More(69%)--表示已经显示了该文件内容的 69%，用户可以按回车键浏览该文件的剩余内容。

```
alloy@alloy:/etc$ more /etc/vim/vimrc
" All system-wide defaults are set in $VIMRUNTIME/debian.vim (usually just
" /usr/share/vim/vimcurrent/debian.vim) and sourced by the call to :runtime
" you can find below.  If you wish to change any of those settings, you should
" do it in this file (/etc/vim/vimrc), since debian.vim will be overwritten
" everytime an upgrade of the vim packages is performed.  It is recommended to
" make changes after sourcing debian.vim since it alters the value of the
" 'compatible' option.

" This line should not be removed as it ensures that various options are
" properly set to work with the Vim-related packages available in Debian.
runtime! debian.vim

" Uncomment the next line to make Vim more Vi-compatible
" NOTE: debian.vim sets 'nocompatible'.  Setting 'compatible' changes numerous
" options, so any other options should be set AFTER setting 'compatible'.
"set compatible

" Vim5 and later versions support syntax highlighting. Uncommenting the next
" line enables syntax highlighting by default.
if has("syntax")
  syntax on
endif

" If using a dark background within the editing area and syntax highlighting
" turn on this option as well
"set background=dark

" Uncomment the following to have Vim jump to the last position when
" reopening a file
"if has("autocmd")
"  au BufReadPost * if line("'\"") > 1 && line("'\"") <= line("$") | exe "normal! g'\""
| endif
"endif

" Uncomment the following to have Vim load indentation rules and plugins
" according to the detected filetype.
"if has("autocmd")
"  filetype plugin indent on
"endif

--更多--(69%)
```

当我们用 cat 命令显示/etc/vim/vimrc 文件时，其输出如下，一次性全部显示该文件的内容，用户看到的仅是该文件最后一屏的内容。

```
" Vim5 and later versions support syntax highlighting. Uncommenting the next
" line enables syntax highlighting by default.
if has("syntax")
```

```
  syntax on
endif

" If using a dark background within the editing area and syntax highlighting
" turn on this option as well
"set background=dark

" Uncomment the following to have Vim jump to the last position when
" reopening a file
"if has("autocmd")
"  au BufReadPost * if line("'\"") > 1 && line("'\"") <= line("$") | exe "normal! g'\""
| endif
"endif

" Uncomment the following to have Vim load indentation rules and plugins
" according to the detected filetype.
"if has("autocmd")
"  filetype plugin indent on
"endif

" The following are commented out as they cause vim to behave a lot
" differently from regular Vi. They are highly recommended though.
"set showcmd          " Show (partial) command in status line.
set showmatch         " Show matching brackets.
set nu
"set ignorecase       " Do case insensitive matching
"set smartcase        " Do smart case matching
"set incsearch        " Incremental search
"set autowrite        " Automatically save before commands like :next and :make
"set hidden           " Hide buffers when they are abandoned
"set mouse=a          " Enable mouse usage (all modes)

" Source a global configuration file if available
if filereadable("/etc/vim/vimrc.local")
  source /etc/vim/vimrc.local
endif

alloy@alloy:/etc$
```

值得一提的是，pg 命令是一个与 more 命令等价的命令，但是，pg 命令是 Linux 扩展命令，很多 Shell 都未曾安装该命令，more 命令是 Linux 核心命令，任何 Shell 都可以使用该命令。

用 more 命令显示文件显然便于用户阅读，尤其是对那些一页不够显示的结果。利用管道功能可以方便地将多页的结果进行分页显示，比如，/etc 目录下有很多文件，列出这些文件时，需要多页显示，我们可以将 ls 的结果通过管道传给 more 命令，more 命令将多页结果进行分页显示，便于用户查看。例 10-3 演示了用分页效果查看/etc 目录下的文件：

```
#例 10-3：ls -l | more 命令的执行结果
alloy@alloy:/etc$ ls -l | more
总用量 1200    #结果共有 1200 个
drwxr-xr-x  3 root root     4096  2 月  4 20:01 acpi
-rw-r--r--  1 root root     2981  2 月  4 19:58 adduser.conf
-rw-r--r--  1 root root       10  3 月 15 10:17 adjtime
drwxr-xr-x  2 root root    12288  4 月 10 14:57 alternatives
-rw-r--r--  1 root root      395  6 月 20  2010 anacrontab
```

```
-rw-r--r--  1 root root     112  6月 22  2007 apg.conf
drwxr-xr-x  6 root root    4096  2月  4 19:59 apm
drwxr-xr-x  3 root root    4096  2月  4 20:00 apparmor
drwxr-xr-x  8 root root    4096  4月 27 11:23 apparmor.d
drwxr-xr-x  5 root root    4096  2月  4 20:01 apport
drwxr-xr-x  6 root root    4096  4月 27 23:04 apt
-rw-r-----  1 root daemon   144 10月 25  2011 at.deny
drwxr-xr-x  2 root root    4096  2月  4 20:01 at-spi2
drwxr-xr-x  3 root root    4096  2月  4 20:01 avahi
-rw-r--r--  1 root root    2076  4月  3  2012 bash.bashrc
-rw-r--r--  1 root root   58753  5月 11  2013 bash_completion
drwxr-xr-x  3 root root    4096  3月 28 14:32 bash_completion.d
-rw-r--r--  1 root root     356  4月 20  2012 bindresvport.blacklist
-rw-r--r--  1 root root     321  3月 30  2012 blkid.conf
lrwxrwxrwx  1 root root      15  3月 15 10:13 blkid.tab -> /dev/.blkid.tab
drwxr-xr-x  2 root root    4096  4月 13 10:07 bluetooth
drwxr-xr-x  2 root root    4096  3月 28 14:25 bonobo-activation
-rw-r--r--  1 root root      33  2月  4 20:01 brlapi.key
drwxr-xr-x  2 root root   12288  2月  4 20:01 brltty
-rw-r--r--  1 root root   19925  4月 11  2012 brltty.conf
drwxr-xr-x  3 root root    4096  2月  4 19:59 ca-certificates
-rw-r--r--  1 root root    7869  3月 28 10:34 ca-certificates.conf
-rw-r--r--  1 root root    6961  2月  4 20:00 ca-certificates.conf.dpkg-old
drwxr-xr-x  2 root root    4096  2月  4 20:01 calendar
drwxr-s---  2 root dip     4096  2月  4 20:01 chatscripts
drwxr-xr-x  2 root root    4096  2月  4 20:01 checkbox.d
-rw-r--r--  1 root root     699 10月 24  2012 colord.conf
drwxr-xr-x  3 root root    4096  2月  4 20:01 compizconfig
drwxr-xr-x  5 root root    4096  2月  4 19:59 ConsoleKit
drwxr-xr-x  2 root root    4096  2月  4 19:58 console-setup
drwxr-xr-x  2 root root    4096  2月  4 20:01 cron.d
drwxr-xr-x  2 root root    4096  2月  4 20:01 cron.daily
drwxr-xr-x  2 root root    4096  2月  4 19:58 cron.hourly
--更多--    #回车可以查看下一页
```

cat 命令还可以同时显示多个文件，命令格式如下：

```
cat file1 file2 file3 … filen
```

该命令将逐个显示从 file1 到 filen 的文件内容，在此处我们可能看不出该命令的作用，但是一旦与文件重定向结合，就可以将多个文件合并为一个文件。

cat 命令还有一个很重要的选项——-v 选项，它可以显示文件中的控制字符，例 10-4 演示了-v 选项的用法。

```
#例 10-4：演示 cat 命令-v 选项的用法
#显示 msdfmap.ini 包括控制字符在内的所有字符
alloy@alloy:~/linuxshell/CH10$ cat -v msdfmap.ini
;[connect name] will modify the connection if ADC.connect="name"^M
;[connect default] will modify the connection if name is not found^M
;[sql name] will modify the Sql if ADC.sql="name(args)"^M
;[sql default] will modify the Sql if name is not found^M
;Override strings: Connect, UserId, Password, Sql.^M
;Only the Sql strings support parameters using "?"^M
;The override strings must not equal "" or they are ignored^M
;A Sql entry must exist in each sql section or the section is ignored^M
;An Access entry must exist in each connect section or the section is ignored^M
```

```
;Access=NoAccess^M
… #没行结束时的^M 是控制字符，表示 CTRL+M
```

在例 10-4 中，用 cat -v 显示 msdfmap.ini 文件，就能够显示出 msdfmap.ini 文件中包含控制字符在内的所有字符，由结果可以看出，msdfmap.ini 文件中每行结束时的^M 就是控制字符，表示“Ctrl+M”组合键。

10.1.3 sed 命令与管道

第 4 章所介绍的 sed 和 awk 命令都是从文件中读取输入数据，事实上，sed 和 awk 命令都支持从管道获得输入数据，sed 和 awk 命令与管道结合的用法也是 Shell 编程中的一个重要方面，本节通过一组例子介绍 sed 命令与管道结合的用法。

我们知道，sed 命令的标准格式为：

```
sed [选项] 'sed 命令' 输入文件
```

在上述格式中，sed 是对输入文件进行处理，当 sed 从管道读取输入数据时，命令中就没必要出现输入文件了，sed 命令的格式就变为：

```
| sed [选项] 'sed 命令'
```

sed 在管道符“|”之后出现，这表示 sed 命令对管道输入的数据进行处理。例 10-5 演示了 sed 用于处理 Shell 命令的输出，例中命令利用 sed 处理 ls -l 命令的输出结果，即 ls -l 命令的输出结果通过管道传给 sed 命令作为输入，sed -n '1,5p'表示打印 ls -l 命令结果的第 1～5 行。

```
#例 10-5：用 sed 处理 ls -l 命令的结果
#打印 ls -l 命令结果的第 1～5 行
alloy@alloy:/etc$ ls -l | sed -n '1,5p'
总用量 1200
drwxr-xr-x  3 root root     4096  2 月  4 20:01 acpi
-rw-r--r--  1 root root     2981  2 月  4 19:58 adduser.conf
-rw-r--r--  1 root root       10  3 月 15 10:17 adjtime
drwxr-xr-x  2 root root    12288  4 月 10 14:57 alternatives
alloy@alloy:/etc$
```

利用管道符同样可以对文件用 sed 命令进行编辑，请看下面的例 10-6，例中第 1 条命令先用 cat 显示/etc/passwd 文件，通过管道传给 sed 命令。实际上，sed 从管道接收到的输入数据就是/etc/passwd 文件的全部内容，sed 查找并打印输入数据中包含 root 关键字的所有行，结果显示两条包含 root 的行。第 2 条命令在第 1 条命令的基础上添加一个管道，对第 1 条命令所产生的结果继续用 sed 命令处理，查找并打印第 1 条命令结果中包含 login 关键字的行，结果变成一行，该行既包含 root 关键字，又包含 login 关键字。

```
#例 10-6：sed 和 cat 结合对文件进行处理
#第 1 条命令：显示/etc/passwd 文件中与 root 的匹配行
alloy@alloy:/etc$ cat passwd | sed -n '/root/p'
root:x:0:0:root:/root:/bin/bash
#第 2 条命令：显示/etc/passwd 文件中与 root 和 login 的匹配行，结果是没有找到
alloy@alloy:/etc$ cat passwd | sed -n '/root/p' | sed -n '/login/p'
alloy@alloy:/etc$
```

由例 10-6 可以看出，sed 和 cat 命令结合同样可以对文件进行处理，例 10-6 中的第 1 条命令等价于下面的命令：

```
sed -n '/root/p' /etc/passwd
```

另外，例 10-6 中的第 2 条命令用到了两个管道。当然，管道可以无限扩展下去，如有必

要，可以继续添加管道，将结果传递给下面的命令。

我们再举一个 sed、管道和变量赋值相结合的例子，如例 10-7 所示，假设变量 variable1="Yahoo develops MapReduce Framework"，我们需要将 varialbe1 中的 Yahoo 改成 Google，并赋给 variable2，即 variable2="Google develops MapReduce Framework"。 例 10-7 给出了利用 sed 和管道实现变量赋值，首先，对 variable1 和 replace 变量赋值，例 10-7 中最重要的命令是对 variable2 进行赋值的命令：variable2 是一个命令替换，即 variable2 是反引号（`）引起命令的执行结果，反引号中有两个命令，中间用管道符相连接，echo 打印 variable1 的值作为 sed 命令的输入，sed 命令 s 表示替换，即将字符串 Yahoo 替换成 replace 变量的值，而 $replace=Google，这样 variable2 的值就变成了“Google develops MapReduce Framework”。

```
#例 10-7：sed、管道和变量赋值相结合的示例
alloy@alloy:/etc$ variable1="Yahoo develops MapReduce Framework"
alloy@alloy:/etc$ replace=Google
# variable2 值是命令替换的结果，命令替换用 sed 和管道结合将 Yahoo 改成 replace 变量的值
alloy@alloy:/etc$ variable2=`echo $variable1 | sed "s/Yahoo/$replace/g"`
alloy@alloy:/etc$ echo $variable2
Google develops MapReduce Framework
alloy@alloy:/etc$
```

例 10-7 虽然简单，却结合了命令替换、管道、sed、变量赋值、echo 等知识，读者应当仔细分析和体会例 10-7 中每一条语句、每一个符号的用法，比如，前面几乎所有的 sed 命令都用的是单引号，为何例 10-7 中 sed 命令后却用了双引号呢？

10.1.4 awk 命令与管道

awk 也可以将管道输入作为输入数据，并对其进行处理，当 awk 将管道作为输入时，awk 命令同样要将输入文件去掉，命令格式变为：

```
| awk [-F 域分隔符] 'awk 程序段'
```

有了管道这一概念后，就可以利用 awk 方便地处理字符串，可以说，几乎所有可以用 expr 命令的字符串处理功能都可以用 awk 代替实现。

例 10-8 演示了 awk 和管道结合处理字符串。首先，将 string 的值赋为：Speeding up small jobs in Hadoop，第 1 条命令利用 awk 内置字符串函数 length 计算 string 的字符串长度，该命令的 length 中是$0，因为 string 从管道赋给 awk 时，string 字符串是 awk 的输入数据，string 此时就相当于原 awk 命令中的输入文件，因此，$0 表示输入数据的全域，即整个 string 字符串。第 2 条命令仅将第 1 条命令的$0 改为$1，表示计算 string 第 1 域的长度，由于 awk 默认以空格为域分隔符，所以，string 的第 1 域为 Speeding，长度为 8。第 3 条命令利用 awk 内置字符串函数 substr 抽取 string 字符串中第 1～8 个字符作为子串输出，相当于 expr substr 命令，该命令仍然以全域$0 代表整个 string 字符串。第 4 条命令将抽取 string 字符串中第 25 个字符开始到最后的子串，并将$0 改成$string，从结果可以看出，awk 同样可以解析 string 变量名，实现 substr 抽取子串功能。

```
#例 10-8：awk 和管道结合处理字符串
alloy@alloy:/etc$ string="Speeding up small jobs in Hadoop"
#第 1 条命令：计算 string 字符串的长度
alloy@alloy:/etc$ echo $string | awk '{print length($0)}'
32
```

```
#第 2 条命令：计算 string 字符串第 1 域的长度
alloy@alloy:/etc$ echo $string | awk '{print length($1)}'
8
#第 3 条命令：抽取 string 字符串中第 1～8 个字符
alloy@alloy:/etc$ echo $string | awk '{print substr($0,1,8)}'
Speeding
#第 4 条命令：awk 可以解析 string 变量名
alloy@alloy:/etc$ echo $string | awk '{print substr($string,25)}'
n Hadoop
alloy@alloy:/etc$
```

由例 10-8 可以看出，当用管道将字符串作为 awk 的输入数据时，awk 将管道输入当作输入文件，因而就可对管道输入分域，awk 也就可以根据$0 和$1 等域号对字符串进行处理。另外，awk 也能够解析变量名，并进行相应的处理，但是，这必须要在该变量从管道输入的情况下，例 10-9 给出了当字符串变量不从管道输入时，awk 试图解析变量名出现的后果。

```
#例 10-9：awk 直接引用变量名的结果
alloy@alloy:/etc$ string="Speeding up small jobs in Hadoop"
alloy@alloy:/etc$ awk '{print substr($string,25)}'
光标位置，等待用户继续输入，完善 awk 命令
```

例 10-9 定义 string 变量后，使用 awk 命令的 substr 抽取子串，substr 引用 string 变量，由结果可以看出，awk 无法执行下去，在下一行出现光标。这说明 Shell 将 awk 解析为不完整的命令，等待用户继续输入。由此可知，当变量不从管道输入 awk 命令时，awk 无法引用已定义的变量名。

下面再举几个 awk 命令中使用管道的例子，先看例 10-10，该例将/etc/passwd 的第 1 域按字母排序后打印，awk 命令先将域分隔符指定为冒号，然后打印第 1 域，将打印结果通过管道传输给 sort 命令，sort 命令对第 1 域结果进行排序，然后输出，结果将/etc/passwd 的第 1 域（即用户名）按字母排序后打印。限于篇幅，例 10-10 仅摘选部分结果。

```
#例 10-10：将/etc/passwd 的第 1 域按字母排序后打印
#注意：sort 上必须加双引号
alloy@alloy:/etc$ awk -F ':' '{print $1 | "sort"}' passwd

alloy
avahi
avahi-autoipd
backup
bin
colord
daemon
games
gnats
hplip
irc
kernoops
libuuid
lightdm
list
lp
mail
man
messagebus
```

```
news
nobody
proxy
pulse
root
rtkit
saned
speech-dispatcher
sshd
sync
sys
syslog
usbmux
uucp
whoopsie
www-data
xrdp
alloy@alloy:/etc$
```

特别注意的是，awk 中调用 Linux 命令时需要用双引号将这些命令引起来，比如例 10-10 用双引号将 sort 命令引起。

例 10-10 是在 awk 命令里用 Shell 处理 awk 的域，反过来，当我们要用 awk 处理 Shell 命令的输出时，需要引入 getline 函数将 Shell 命令的输出保存到变量中，awk 再对该变量进行处理。例 10-11 演示了在 awk 中处理 Shell 命令输出，例中命令的作用是将 ls /usr 命令的输出传给 awk，awk 再将 awk 打印出来。我们在 awk 的 BEGIN 字段中使用了 while 循环，循环内将 ls /usr 命令的结果逐一通过管道传给 geline d 命令，打印 d 变量，直到 ls /usr 命令的结果全部处理结束。

```
#例 10-11：用 awk 打印 ls /usr 命令的结果
#用 while 循环将 ls /usr 命令的输出结果保存到变量 d 中，然后打印
alloy@alloy:/etc$ awk 'BEGIN{while (("ls /usr" | getline d)>0) print d}'
bin
games
include
lib
local
sbin
share
src
alloy@alloy:/etc$
```

在 awk 中，如果要对 Shell 命令结果进行处理，必须将结果通过管道传给 getline 函数，如果结果较多，则需要使用循环。我们也可以不将 Shell 命令放在 awk 的内部，awk 同样可以对 Shell 命令进行处理。若我们要查看文件系统剩余的空间容量，输出可用空间大于一个触发值的文件系统，我们就可以将 df -k 命令的结果通过管道传给 awk，awk 通过判断第 4 域是否大于这一触发值而决定是否输出该文件空间，df -k 命令可以列出 Linux 系统文件空间的详细信息，第 4 域是剩余空间量。下面的例 10-12 中的命令是输出可用空间大于 1GB 的文件系统，结果显示只有文件系统/dev/sda3 满足条件，挂载点是 Linux 的/目录。

```
#例 10-12：用 awk 查看文件系统的可用空间
#输出可用空间大于 1GB 的文件空间
alloy@alloy:/etc$ df -k | awk '$4>1000000'
```

```
文件系统        1K-块       已用       可用        已用% 挂载点
/dev/sda1       16381864    5479664    10047008    36% /
udev            2010556     4          2010552     1% /dev
none            2020124     152        2019972     1% /run/shm
alloy@alloy:/etc$
```

10.2 I/O 重定向

I/O 重定向是 Shell 编程中的一个重要内容，何谓 I/O 重定向呢？简言之，I/O 重定向是一个过程，这个过程捕捉一个文件、命令、程序或脚本，甚至代码块（code block）的输出，然后把捕捉到的输出作为输入发送给另外一个文件、命令、程序或脚本。

本节首先介绍 I/O 重定向中的重要概念——文件标识符，同时介绍标准输入（stdin）、标准输出（stdout）、标准错误输出（stderr）和 tee 命令等内容；其次，完整介绍 I/O 重定向的所有符号及用法；然后，介绍利用 exec 实现 I/O 重定向；最后介绍代码块重定向。在所有的内容介绍完后，我们再介绍几个 I/O 重定向的应用。

10.2.1 文件标识符

文件标识符（File Descriptor，FD）是 I/O 重定向的重要概念，本节将介绍文件标识符、标准输入（stdin）、标准输出（stdout）和标准错误输出（stderr）。

Linux 使用文件标识符（FD）来标识一个进程正在访问的特定文件，当打开一个文件或者创建一个文件时，Linux 将返回一个文件标识符供其他操作引用。通常来说，文件标识符是一个小的非负整数，它是对应进程的，每一个文件标识符都对应一个特定的文件，而每一个特定的文件可能在对应该进程而言存在多个不同的文件标识符。当 Linux 系统启动一个进程（该进程可能用于执行 Shell 命令）时，将自动为该进程打开三个文件：标准输入、标准输出和标准错误输出，分别由文件标识符 0、1、2 标识，该进程如果要打开其他的输入或输出文件，则从整数 3 开始标识。标准输入、标准输出和标准错误输出是软件设计中最基本的概念，软件设计认为程序应该有一个数据来源、一个数据出口，以及产生错误报告的地方。它们分别是标准输入、标准输出和标准错误输出。默认情况下，标准输入与键盘输入相关联，标准输出和标准错误输出与显示器相关联。

图 10-2 描述了 stdin、stdout、stderr 与 Shell 命令的关系，Shell 命令从标准输入读取输入数据，将输出送到标准输出，如果该命令在执行过程中发生错误，则将错误信息输出到标准错误输出。

图 10-2　stdin、stdout 与 stderr

如果需要将 Shell 命令的输出从标准输出复制一份到某个文件中，可以使用 tee 命令。例 10-13 演示了 tee 命令的基本用法，例中命令 who | tee output 先将 who 命令的执行结果从管道传给 tee 命令，tee 命令将 who 命令的标准输出复制到 output 文件，因此，结果仍然输出 who 命令的标准输出，查看 output 文件内容后，证实 output 文件确实保存了 who 命令的标准输出。

```
#例 10-13：演示 tee 命令的基本用法
#将 who 命令的标准输出复制到 output 文件
alloy@alloy:~/linuxshell/CH10$ who | tee output
alloy    tty7         2014-04-27 23:44
alloy    pts/2        2014-04-29 17:09 (192.168.0.102)
alloy    pts/1        2014-04-29 17:08 (:0)
#查看 output 文件，与 who 标准输出一样
alloy@alloy:~/linuxshell/CH10$ cat output
alloy    tty7         2014-04-27 23:44
alloy    pts/2        2014-04-29 17:09 (192.168.0.102)
alloy    pts/1        2014-04-29 17:08 (:0)
alloy@alloy:~/linuxshell/CH10$
```

例 10-13 中 who | tee output 命令的数据流向可以用图 10-3 来表示，tee 命令产生的数据流向很像英文字母 T，将一个输出分为两个支流，一个到标准输出，另一个到某输出文件。

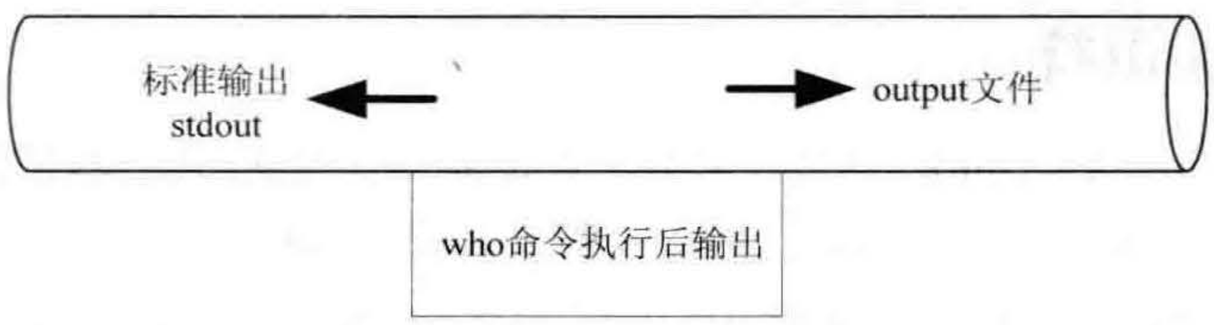


图 10-3　who | tee output 命令的数据流向

tee 命令还有一个选项-a，表示将 Shell 命令的输出追加到某个文件的末尾，例 10-14 说明了 tee -a 命令的用法，命令 ps | tee -a output 将 ps 命令的标准输出追加到 output 文件末尾，从结果中可以看到，output 首行是由 who 命令产生的输出，后面四行是 ps 命令产生的输出。

```
#例 10-14：tee -a 命令的用法
#将 ps 命令的标准输出复制并追加到 output 文件末尾
alloy@alloy:~/linuxshell/CH10$ ps | tee -a output
  PID TTY          TIME CMD
 5728 pts/2    00:00:00 bash
 5937 pts/2    00:00:00 ps
 5938 pts/2    00:00:00 tee
alloy@alloy:~/linuxshell/CH10$ cat output
alloy    tty7         2014-04-27 23:44
alloy    pts/2        2014-04-29 17:09 (192.168.0.102)
alloy    pts/1        2014-04-29 17:08 (:0)
  PID TTY          TIME CMD  #从此开始是 ps 命令的输出
 5728 pts/2    00:00:00 bash
 5937 pts/2    00:00:00 ps
 5938 pts/2    00:00:00 tee
alloy@alloy:~/linuxshell/CH10$
```

通过例 10-13 和例 10-14 不难掌握 tee 命令的基本用法，tee 命令可用于 Shell 脚本的调试，尤其是管道的调试，这将在第 17 章中详细论述。

10.2.2 I/O 重定向符号及其用法

I/O 重定向符号有很多种，为了便于读者学习，我们将 I/O 重定向符号分为两类：基本 I/O 重定向符号和 I/O 重定向符号。表 10-3 了列出基本 I/O 重定向符号及其意义。

表 10-3 基本 I/O 重定向符号及其意义

符　　号	意　　义
cmd1 | cmd2	管道符，将 cmd1 的标准输出作为 cmd2 的标准输入
> filename	将标准输出写到文件 filename 中
< filename	将文件 filename 的内容读入标准输入中
>> filename	将标准输出写到文件 filename 中，若 filename 文件已存在，则将标准输出追加到 filename 已有的内容之后
>| filename	即使 noclobber 选项已开启，仍然强制将标准输出写到文件 filename 中，即将 filename 文件覆盖
n>| filename	即使 noclobber 选项已开启，仍然强制将 FD 为 n 的输出写到文件 filename 中，即将 filename 文件覆盖
n> filename	将 FD 为 n 的输出写到文件 filename 中
n< filename	将文件 filename 的内容读入到 FD n 中
n>> filename	将 FD 为 n 的输出写到文件 filename 中，若 filename 文件已存在，则将 FD 为 n 的输出追加到 filename 已有的内容之后
<<delimiter	此处文档（Here-document）

管道符也是一种 I/O 重定向符号，由于 10.1 节已经详细探讨了管道，所以，在此忽略对管道符的论述，下面举一些例子说明其他 I/O 重定向符号及其用法。首先，请看例 10-15，该例子将 cat 和>符号结合成为简易文本编辑器，当 cat 命令后不加任何参数时，cat 命令的输入是标准输入，即键盘输入，然后利用 I/O 重定向符号“>”将键盘输入写入文件，因此，输入 cat > newfile 后，就可输入需要写到 newfile 的内容，最后按 Ctrl+D 组合键结束对 newfile 的编辑。在用 cat newfile 命令查看 newfile 文件内容时，发现 newfile 中确实是从键盘输入的文本，由于 newfile 最后一行后面无换行符，因此，显示完 newfile 内容后，紧接着就是 Shell 提示符。

```
#例 10-15：cat 和>符号结合成为简易文本编辑器
alloy@alloy:~/linuxshell/CH10$ cat > newfile
deadlock and livelock avoidance protocol
Proposed by Parkalloy@alloy:~/linuxshell/CH10$ cat newfile
                                                    #按 Ctrl+D 组合键结束文件编辑
deadlock and livelock avoidance protocol
Proposed by Parkalloy@alloy:~/linuxshell/CH10$
```

newfile 被创建后，我们再用“>>”符号在 newfile 后追加一些文本，如例 10-16 所示，例中的命令将/etc 目录中包含 rc 字符的文件名追加到 newfile 中，命令先列出/etc 的所有文件名，将结果传给 grep 命令，grep 查找与 rc 字符串匹配的文件名，将输出结果追加到 newfile。观察 newfile 中的文本内容，前面两行是原来的文本，Park 字符串后是追加的文本。

```
#例 10-16：利用>>符号向 newfile 追加文本
alloy@alloy:~/linuxshell/CH10$ ls /etc | grep "rc" >> newfile
alloy@alloy:~/linuxshell/CH10$ more newfile
```

```
bash.bashrc
dirirc
inputrc
nanorc
ODBCDataSources
rc0.d
rc1.d
rc2.d
rc3.d
rc4.d
rc5.d
rc6.d
rc.local
rcS.d
wgetrc
alloy@alloy:~/linuxshell/CH10$
```

我们将例 10-16 命令中的>>符号改为>符号，执行它并观察 newfile 的文本内容，如例 10-17 所示，从结果可以看到，newfile 中原来的文本被覆盖，换成 ls /etc | grep "rc"命令的输出结果。由此可以总结出，>>符号是将标准输出追加到已有文件的内容之后，而>符号则将文件的原有文本覆盖，然后写入标准输出的内容。

```
#例 10-17：说明>符号与>>符号的区别
#与例 10-16 相比，仅将>>符号改成了>符号

alloy@alloy:~/linuxshell/CH10$ ls /etc | grep "rc" > newfile
alloy@alloy:~/linuxshell/CH10$ more newfile  #newfile 中原来的内容被新内容覆盖
bash.bashrc
dirirc
inputrc
nanorc
ODBCDataSources
rc0.d
rc1.d
rc2.d
rc3.d
rc4.d
rc5.d
rc6.d
rc.local
rcS.d
wgetrc
alloy@alloy:~/linuxshell/CH10$
```

>|符号是强制覆盖文件的符号，它与 Shell 的 noclobber 选项有关系，如果 noclobber 选项开启，表示不允许覆盖任何文件，而>|符号则可以不管 noclobber 选项的作用，强制将文件覆盖。下面的例 10-18 演示了>|符号的基本用法，其中第 1 条命令用 set-o 命令开启 noclobber 选项，第 2 条命令用>符号将 ls /etc | grep "rc.d"命令的输出结果写入 newfile，由于>符号是需要覆盖文件的，因此，Shell 报错“不能覆盖已存在的文件”，这说明 noclobber 选项确实起作用了。第 3 条命令将第 2 条命令的>符号改为>|符号，执行成功，这说明>|符号强制将 newfile 覆盖，第 4 条命令查看 newfile 文件内容，表明 newfile 文件确实已被覆盖。

```
#例 10-18：演示>|符号的基本用法
alloy@alloy:~/linuxshell/CH10$ set -o noclobber
```

```
#第 1 条命令：开启 noclobber 选项
#第 2 条命令：试图覆盖 newfile 时，报语法错误，noclobber 选项起作用了
alloy@alloy:~/linuxshell/CH10$ ls /etc | grep "rc.d" > newfile
-bash: newfile: 无法覆盖已存在的文件
#第 3 条命令：用>|符号强制将 newfile 覆盖，而忽略 noclobber 选项的作用
alloy@alloy:~/linuxshell/CH10$ ls /etc | grep "rc.d" >| newfile
#第 4 条命令：查看 newfile 文件内容，证实 newfile 确实已被覆盖
alloy@alloy:~/linuxshell/CH10$ cat newfile
alloy@alloy:~/linuxshell/CH10$
```

下面再举一个标准错误输出的例子，如例 10-19 所示，例中第 1 条命令显示该目录下以 z 开头的文件，由于该目录中没有这类文件，因此，Shell 输出错误信息“ls: 无法访问 z*: 没有那个文件或目录”；第 2 条命令将 ls z*的输出重定向到 newfile 文件，但是，错误信息仍在 Shell 上显示，这说明错误信息并非是该命令的输出结果；第 3 条命令查看 newfile 内容，结果为空，说明 ls z*命令的输出是空；第 4 条命令用 n>filename 符号将 FD 为 2 的文件重定向到 newfile 文件，FD 为 2 的文件即为标准错误输出，然后用第 5 条命令查看 newfile 内容时，发现 newfile 中保存了 ls z*命令的错误信息。值得注意的是，第 4 条命令中 2 和>之间不能有空格。

```
#例 10-19：标准错误输出的用法
#第 1 条命令：显示该目录下以 z 开头的文件
alloy@alloy:~/linuxshell/CH10$ ls z*
ls: 无法访问 z*: 没有那个文件或目录
#第 2 条命令：将输出重定向到 newfile，但是结果仍然跟第 1 条命令一样
alloy@alloy:~/linuxshell/CH10$ ls z* > newfile
ls: 无法访问 z*: 没有那个文件或目录
#第 3 条命令：查看 newfile 内容，为空
alloy@alloy:~/linuxshell/CH10$ cat newfile
#第 4 条命令：将 stderr 重定向到 newfile，不再输出错误信息
alloy@alloy:~/linuxshell/CH10$ ls z* 2> newfile
#第 5 条命令：查看 newfile 内容，保存了错误信息
alloy@alloy:~/linuxshell/CH10$ cat newfile
ls: 无法访问 z*: 没有那个文件或目录
alloy@alloy:~/linuxshell/CH10$
```

n>>filename、n>|filename 与 n>filename 都是将 FD 为 n 的文件重定向到 filename 文件中，三者的区别实际上是>>、>|和>这三个符号的区别，例 10-17 和例 10-18 已经介绍得十分清楚了，在此就不再举例介绍 n>>filename 和 n>|filename 两种符号的用法。

<是 I/O 重定向的输入符号，它可将文件内容写到标准输入之中。下面举一个例子说明<符号，如例 10-20 所示，其中第 1 条命令打印 newfile 的内容。第 2 条命令和第 3 条命令相似，形式上仅相差一个<符号，下面分析这两条相似命令的执行过程：第 2 条命令中 wc 开始执行时，将-l 和 newfile 作为两个输入参数，-l 参数表示对行进行计数，newfile 指明了需要行计数的文件名，因此，wc 打开 newfile 文件，统计行数，并在 Shell 上打印出统计的行数 1 和相应的文件名；第 3 条命令开始执行时，Shell 识别出输入重定向符号<，并判断<符号后面的字符串表示重定向数据源文件的名称，Shell 从命令行“吞”掉<newfile，启动 wc 命令，将它的标准输入重定向到 newfile 文件，并给 wc 命令传递一个参数-l，因此，wc 命令并不知道 newfile 的存在，因为 wc 命令只管从标准输入中读取数据，所以，结果只打印行数 1，并不打印文件名称。

```
#例 10-20：说明<符号的用法
alloy@alloy:~/linuxshell/CH10$ cat newfile
#第 1 条命令：显示 newfile 文件内容
ls：无法访问 z*：没有那个文件或目录
alloy@alloy:~/linuxshell/CH10$ wc -l newfile
#第 2 条命令：-l 和 newfile 作为参数输入
1 newfile
#第 3 条命令：从 stdin 中读数据，不知道 newfile 的存在
alloy@alloy:~/linuxshell/CH10$ wc -l <newfile
1                              #结果未列出文件名 newfile
alloy@alloy:~/linuxshell/CH10$
```

<<delimiter 符号称为此处文档（Here-document），delimiter 称为分界符，该符号表明：Shell 将分界符 delimiter 之后直至下一个 delimiter 之前的所有内容作为输入。可能对此处文档的解释不容易理解，在此举一个例子帮助读者理解，如例 10-21 所示，例中命令 cat >> hfile 类似于例 10-15 中的命令 cat > newfile，cat 作为文本编辑器，我们在 cat >> hfile 后加上了 <<CLOUD 表示将 CLOUD 作为分界符，输入到下一个 CLOUD 为止。例 10-21 中，我们在 cat >> hfile <<CLOUD 后输入两行文本，第 3 行输入 CLOUD 后按 Enter 键就结束编辑，查看 hfile 发现确实为以上所输入的两行文本。例 10-15 中，我们是按 CTRL+D 结束编辑的，加上 <<CLOUD 后，下一个 CLOUD 就相当于 Ctrl+D 组合键。

```
#例 10-21：演示<<delimiter 符号的用法
alloy@alloy:~/linuxshell/CH10$cat >> hfile <<CLOUD
> Hadoop is developed by Yahoo
> Google also develops GFS and MapReduce
>CLOUD                          #两个 delimiter 之间的内容作为输入
alloy@alloy:~/linuxshell/CH10$ cat hfile
Hadoop is developed by Yahoo
Google also develops GFS and MapReduce
alloy@alloy:~/linuxshell/CH10$
```

<<delimiter 符号还有一种形式：-<<delimiter，即在<<符号加上一个负号，这样输入文本行所有开头的 Tab 键都会被删除，但是，开头的空格键却不会被删除，例 10-22 演示了 -<<delimiter 符号的用法，例中命令 cat > hfile -<<CLOUD 表示以将 CLOUD 作为分界符，到下一个 CLOUD 为止的输入写入 hfile 文件。我们输入的第 1 行以空格开头，第 2 行输入几个 Tab 键，然后输入 Google also develops GFS and MapReduce 这段文字，用 cat 命令查看 hfile 发现第 1 行的空格未被删除，第 2 行开头的 Tab 键却被删除了。

```
#例 10-22：-<<delimiter 符号的用法
alloy@alloy:~/linuxshell/CH10$ cat > hfile -<< CLOUD
>  Hadoop is developed by Yahoo                #以空格开头
>          #在新的一行输出一些 Tab 键，出现当前目录的所有文件
anotherres.sh    example/        loggg1          parttotal       subsenv.sh
> Google also develops GFS and MapReduce
> CLOUD
alloy@alloy:~/linuxshell/CH10$ cat hfile
 Hadoop is developed by Yahoo                #空格未被删除
Google also develops GFS and MapReduce       #Tab 键被删除了
alloy@alloy:~/linuxshell/CH10$
```

基本 I/O 重定向符号就介绍到这里，下面介绍稍微复杂一些的 I/O 重定向符号，我们称之为高级 I/O 重定向符号。由于高级 I/O 重定向符号与 exec 命令有关，所以，我们将高级 I/O

重定向符号的内容放到下一节介绍。

基本 I/O 重定向符号和 10.1.1 节中介绍的管道区别说明如下：

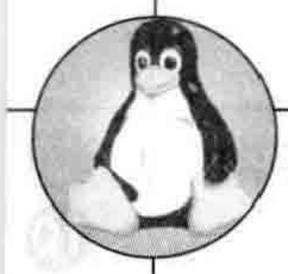
- 管道“|”其实也是 I/O 重定向的一个操作部分。
- 管道左边是一个可以送到标准输出（stdout）的命令，右边是一个从标准输入接收（stdin）的命令；如果 I/O 重定向符左边是有输出的命令，右边只能是文件，如果 I/O 重定向符左边是能接收输入的命令，右边只能是文件。
- 在管道执行的时候启动了两个进程，该进程分别执行左右两边的命令；而 I/O 重定向符值则在同一个进程中完成。

10.2.3 exec 命令的用法

exec 命令可以通过文件标识符打开或关闭文件，也可将文件重定向到标准输入或将标准输出重定向到文件。下面首先举一个例子说明 exec 将标准输入重定向到文件，新建名为 execin.sh 的脚本，内容如下：

```
#例 10-23: execin.sh 脚本使用 exec 将 stdin 重定向到文件
#!/bin/bash

#FD 8 是 FD 0（即标准输入）的副本，用于恢复 FD 0
exec 8<&0

exec < hfile                    #将标准输入重定向到 hfile
read a                          #读取 hfile 文件的第 1 行
read b                          #读取 hfile 文件的第 2 行

echo "--------------------------"
echo $a
echo $b

echo "Close FD 8:"
# 0<&8: 将 FD 8 复制到 FD 0，FD 8 是原来的标准输入，FD 0 从 FD 8 中恢复原状
#8<&-: 关闭 FD 8，其他进程可以重复使用 FD 8
exec 0<&8 8<&-

#在执行 read 命令时，read 命令从键盘输入读取数据
echo -n "Pls. Enter Data:"
read c                          #标准输入恢复成键盘输入
echo $c
```

execin.sh 脚本演示如何将标准输入重定向到文件，以及如何通过其他 FD 的文件恢复标准输入。execin.sh 脚本首先用 exec 8<&0 将 FD 0 复制到 FD 8，此时，exec 命令同时打开了 FD 8 文件。然后，execin.sh 脚本用 exec <hfile 将 hfile 复制到标准输入，事实上，<符号是等价于 0<符号的。read 命令本来是用于接收用户输入的，即 read 命令原本从标准输入读取数据。当 execin.sh 脚本执行 read 命令时，每次读入 hfile 的一行，因此，a 和 b 变量分别保存了 hfile 的第 1 行和第 2 行，从下面 execin.sh 脚本的执行结果打印了 a 和 b 变量，确实是 hfile 的第 1 行和第 2 行。接着，execin.sh 脚本将 FD 8 文件复制到 FD 0，由于 FD 8 保存了原来的 FD 0，因此，将 FD 8 复制给 FD 0 后，FD 0 恢复原状了，并且 execin.sh 脚本关闭 FD 8 文件，便于其他进程可以重复使用 FD 8。最后，execin.sh 脚本再次执行 read 命令以测试标准输入是



否恢复原状，从下面 execin.sh 脚本的执行结果可以看出，执行 read 命令后，Shell 等待用户输入，并且回显用户所输入的变量，这说明 read 命令从键盘输入读取数据，说明 FD 0 恢复原状。

```
#例 10-23 execin.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH10$ chmod u+x execin.sh
alloy@alloy:~/linuxshell/CH10$ ./execin.sh
---------------------------
Hadoop is developed by Yahoo
Google also develops GFS and MapReduce      #read 命令从 hfile 中读取的结果
Close FD 8:                                 #将 FD 8 复制给 FD 0，使 FD 0 恢复从键盘输入，再关闭 FD 8
Pls. Enter Data:CAICAI                      #read 命令从键盘读取数据，说明 FD 0 已恢复到键盘输入
CAICAI
alloy@alloy:~/linuxshell/CH10$
```

下面再举一个例子说明 exec 将标准输出重定向到文件，新建名为 execout.sh 的脚本，内容如下：

```
#例 10-24：execout.sh 脚本将 stdout 重定向到文件
#!/bin/bash

#FD 8 是 FD 1（即标准输出）的副本，用于恢复 FD 1
exec 8>&1                                   #符号与 execin.out 中略有不同

exec > loggg                                #将标准输出重定向到 loggg

#执行 date 和 df 命令，测试输出是否写入 loggg 文件
echo "Output of date command"
date
echo "Output of df command"
df

# 0<&8：将 FD 8 复制到 FD 0，FD 8 是原来的标准输出，FD 0 从 FD 8 中恢复原状
#8<&-：关闭 FD 8，其他进程可以重复使用 FD 8
#这条命令与 execin.out 中的原理相同，只是将<&改为了>&
exec 1>&8 8>&-

#再次执行 date 和 df 命令，测试标准输出是否恢复原状
echo "--------------------------------"
echo "Output of date command"
date
echo "Output of df command"
df
```

execout.sh 脚本的思路类似于 execin.sh 脚本，先将标准输出 FD 1 复制到 FD 8，命令与复制 FD 0 时类似，只是符号变成了>&。execout.sh 脚本再将标准输出重定向到 loggg 文件，>符号等价于 1>符号，然后执行 date 和 df 命令，从下面 execout.sh 脚本的执行结果可以看出，此时 date 和 df 命令不在屏幕上打印任何内容，而将输出的结果写入了 loggg 文件。接着，execout.sh 脚本将 FD 8 复制给 FD 1，使 FD 1 恢复原状，并关闭 FD 8 文件，然后，再次执行 date 和 df 命令，从下面 execout.sh 脚本的执行结果可以看出，此时，上述两条命令的结果输出到屏幕，这说明标准输出恢复原状了。

```
#例 10-24 execout.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH10$ chmod u+x execout.sh
```

```
alloy@alloy:~/linuxshell/CH10$ ./execout.sh
--------------------------------                    #下面是将标准输出回复原状后的输出
Output of date command
2014年 04月 29日 星期二 21:21:01 CST
Output of df command
文件系统        1K-块        已用      可用        已用% 挂载点
/dev/sda1     16381864     5479672  10047000      36% /
udev          2010556      4        2010552      1% /dev
tmpfs         404028       804      403224       1% /run
none          5120         0        5120         0% /run/lock
none          2020124      152      2019972      1% /run/shm
alloy@alloy:~/linuxshell/CH10$ cat loggg          #将标准输出重定向到loggg后的输出
Output of date command
2014年 04月 29日 星期二 21:21:01 CST
Output of df command
文件系统        1K-块        已用      可用        已用% 挂载点
/dev/sda1     16381864     5479672  10047000      36% /
udev          2010556      4        2010552      1% /dev
tmpfs         404028       804      403224       1% /run
none          5120         0        5120         0% /run/lock
none          2020124      152      2019972      1% /run/shm
alloy@alloy:~/linuxshell/CH10$
```

例 10-23 和例 10-24 的两个例子涉及不同 FD 文件间的复制、关闭 FD、将标准输入和标准输出重定向到文件等操作，这些操作的符号就是前面未曾讲述的高级 I/O 重定向符号，我们将所有的高级 I/O 重定向符号及其意义列于表 10-4 中。

表 10-4　高级 I/O 重定向符号及其意义

符　　号	意　　义
n>&m	将 FD 为 m 的输出复制到 FD 为 n 的文件
n<&m	将 FD 为 m 的输入复制到 FD 为 n 的文件
n>&-	关闭 FD 为 n 的输出
n<&-	关闭 FD 为 n 的输入
&>file	将标准输出和标准错误输出重定向到文件

&>file 可以同时将标准输出和标准错误输出重定向到文件，下面的例 10-25 说明了&>file 的用法，新建名为 execerr.sh 的脚本，内容如下：

```
#例 10-25：execerr.sh 脚本将 stdout 和 stderr 重定向到文件
#!/bin/bash

#将 FD 1（即标准输出）复制到 FD 8，FD 2（即标准错误输出）复制到 FD 9
exec 8>&1 9>&2

#将标准输出和标准错误输出重定向到 loggg 文件
exec &> loggg

#执行 ls z*和 date 命令，测试输出和错误输出是否写入 loggg 文件
ls z*                          #出现错误输出
date

#恢复标准输出和标准错误输出，并关闭 FD 8 和 FD 9
```

```
exec 1>&8 2>&9 8<&- 9<&-

#再次执行 ls z*和 date 命令，测试输出和错误输出是否恢复原状
echo "-----------------"
echo "Close FD 8 and 9:"
ls z*
date
```

execerr.sh 脚本开启两个 FD，分别用于保存 FD 1 和 FD 2，即保存标准输出和标准错误输出。execerr.sh 脚本用&>符号同时将标准输出和标准错误输出重定向到 loggg 文件，然后执行 ls z*和 date 命令，测试输出和错误输出是否写入 loggg 文件，由于当前目录下没有以 z 开头的文件，因此，ls z*将产生错误，而 date 命令产生的输出，从 execerr.sh 脚本的执行结果可以看出，loggg 文件既能够保存由 ls z*命令输出的错误信息，又能够保存由 date 命令输出的正常输出，可见，>&确实可以同时重定向标准输出和标准错误输出。接着，execerr.sh 脚本利用 FD 8 和 FD 9 恢复 FD 1 和 FD 2，然后再次执行 ls z*和 date 命令，测试标准输出和标准错误输出是否恢复原状。从 execerr.sh 脚本也可看出，exec 命令可以对多个 I/O 重定向符号进行操作，如 exec 1>&8 2>&9 8<&- 9<&-命令同时实现了 4 个 FD 操作。

```
#例 10-25 execerr.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH10$ chmod u+x execerr.sh
alloy@alloy:~/linuxshell/CH10$ ./execerr.sh
-----------------
Close FD 8 and 9:
ls: 无法访问 z*: 没有那个文件或目录
2014 年 03 月 23 日 星期日 13:24:10 CST
alloy@alloy:~/linuxshell/CH10$ cat loggg     #loggg 文件既保存 stderr，又保存 stdin
ls: 无法访问 z*: 没有那个文件或目录
2014 年 03 月 23 日 星期日 13:24:10 CST
alloy@alloy:~/linuxshell/CH10$
```

在 Linux 中有以下两个经常被用作 I/O 重定向操作重要的特殊文件（其他更多的内容可以参考 15.5 节）。

- /dev/null：这是一个空设备文件，被送到其中的所有数据都会被丢弃，可以将标准输出和错误输出全部重定向到该文件中，此时即可将错误信息全部丢弃。其标准写法是>/dev/null 2>&，其中，>/dev/null 表示丢弃文件；2>&1 的 2 和 1 分别表示标准输出 stdout 和标准出错 stderr，>&表示等同于，也就是将标准出错输出 2 重定向于标准输出 1，由于标准输出已经被重定位到了/dev/null，所以标准出错的输出也被扔掉了。该写法也可以是 1>/dev/null 2>/dev/null，区别在这种情况下标准输出和标准出错都被送到/dev/null，这个文件被打开了两次；而>/dev/null 2 >& 1 则是将标准输出送到/dev/null，标准出错则是从管道送到/dev/null，/dev/null 只被打开了一次，其中/dev/null 可以换成其他任何文件。
- /dev/tty：当该文件被打开的时候 Shell 会将其自动重定向到一个终端（包括显示控制台、远程登录伪中断或者串口）。

10.2.4 代码块重定向

代码块重定向是指在代码块内将标准输入或标准输出重定向到文件，而在代码块之外还

是保留默认状态。换句话说，代码块重定向是指对标准输入或标准输出的重定向只在代码块内有效。可以重定向的代码块可以是 while、until、for 等循环结构，也可以是 if/then 测试结构，甚至可以是函数。代码块输入重定向符号是<，输出重定向符号是>。本节举例说明上述两个符号在代码块重定向中的用法。

首先，介绍 while 循环的重定向，新建名为 rewhile.sh 的脚本，内容如下：

```
#例 10-26：rewhile.sh 脚本演示 while 循环的重定向
#!/bin/bash

#将 ls /etc 的结果写到 loggg 文件中
ls /etc > loggg

#搜索 loggg 文件中与 rc.d 所匹配的行，输出匹配行的行数
while [ "$filename" != "rc.d" ]          #当不匹配时，执行 while 循环体
do
 #按行读取 loggg 内容
read filename

let "count +=1"
done < loggg                             #将标准输入重定向到 loggg 文件

echo "$count times read"

#测试循环体外面的标准输入是否被重定向
echo -n "-----Pls. Input Data:-----"
read test
echo $test
```

rewhile.sh 脚本首先将 ls /etc 的结果写到 loggg 文件中，loggg 文件中按行保存了/etc 目录下的所有文件名，rewhile.sh 脚本需要查找 loggg 文件中与 rc.d 所匹配的行，并输出匹配行的行数，为此，rewhile.sh 脚本使用了 while 循环，该循环在结尾处 done 关键字的后面利用<符号将标准输入重定向到 loggg 文件，然后用 read 命令按行读取 loggg 内容，保存在 filename 变量中，while 循环测试条件判断 filename 变量的值是否等于关键字 rc.d，若该测试条件不满足，则继续读取下一行，计数器 count 增加 1，一旦测试条件满足，则跳出 while 循环，输出 count 变量值。rewhile.sh 脚本还对 while 循环体外面的标准输入进行了测试。下面给出 execerr.sh 脚本的执行结果：count 变量值等于 188，这说明 loggg 文件的第 188 行与 rc.d 匹配，循环体外面的标准输入依然是默认的状态，即从键盘读取输入数据。

```
#例 10-26 rewhile.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH10$ chmod u+x rewhile.sh
alloy@alloy:~/linuxshell/CH10$ ./rewhile.sh
188 times read                           #loggg 文件的第 188 行与 rc.d 匹配
-----Pls. Input Data:-----caicai         #循环体外面的 stdin 依然从键盘读取数据
caicai
alloy@alloy:~/linuxshell/CH10$
```

用 until 循环也可以实现与 rewhile.sh 脚本同样的功能，限于篇幅，我们在本章上机提议第 6 题给出 reuntil.sh 脚本，建议读者将 rewhile.sh 脚本和 reuntil.sh 脚本仔细进行比较，并运行 reuntil.sh 脚本。

下面的例 10-27 利用 for 循环的重定向实现对 loggg 文件 rc.d 关键字的查找，新建名为

refor.sh 的脚本，内容如下：

```
#例 10-27：refor.sh 脚本演示 for 循环的重定向
#!/bin/bash

#将 ls /etc 的结果写到 loggg 文件中
ls /etc > loggg

#计算 loggg 文件的最大行数，并赋给 maxline 变量
#这是与 while 和 until 循环最大的区别
#灵活运用了输入重定向符号<，类似用法可以参见例 10-26 的例子
maxline=$(wc -l < loggg)

#搜索 loggg 文件中与 rc.d 匹配的行，输出匹配行的行数
for filename in `seq $maxline`                    #利用 seq 命令产生循环参数
do
 read filename                                    #按行读取 loggg 中的数据

#for 循环中需要有 if 语句指定跳出循环的条件
 if [ "$filename" = "rc.d" ]
 then
   break
 else
   let "count +=1"
 fi
done < loggg                                      #将标准输入重定向到 loggg 文件

echo "$count times read"

#测试循环体外面的标准输入是否被重定向
echo -n "-----Pls. Input Data:-----"
read test
echo $test
```

refor.sh 脚本同样首先将 ls /etc 的结果写到 loggg 文件中，loggg 作为搜索的目标文件。为了限定 for 循环的最大次数，需要计算 loggg 文件的最大行数，并赋给 maxline 变量，计算方法灵活运用了输入重定向符号<，loggg 作为 wc -l 命令的输入，将计算所得的行数赋给 maxline。for 循环使用命令替换，反引号中是 seq 命令产生数字序列作为 for 循环的次数，实际上，for filename in `seq $maxline`相当于 for filename in 1,2,3…,maxline，使用命令替换后使得 for 循环显得十分简洁。for 循环在结尾处 done 关键字的后面利用<符号将标准输入重定向到 loggg 文件，然后用 read 命令按行读取 loggg 内容，保存在 filename 变量中，循环体中 if/then 结构判断 filename 变量的值是否等于关键字 rc.d，若匹配，则跳出 for 循环，否则计数器 count 增加 1。refor.sh 脚本最后同样对 for 循环体外面的标准输入进行了测试。下面给出 refor.sh 脚本的执行结果，与 rewhile.sh 脚本的执行结果完全一样。

```
#例 10-27 refor.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH10$ chmod u+x refor.sh
alloy@alloy:~/linuxshell/CH10$ ./refor.sh   #得到与 rewhile.sh 一样的结果
188 times read
-----Pls. Input Data:-----caicai
caicai
alloy@alloy:~/linuxshell/CH10$
```

if/then 结构也可将标准输入重定向到文件，它的命令格式如下（注意：重定向符号<是放在 fi 关键字后面的）：

```
if [condition]
 then
   …
 else
   …
fi < filename
```

代码块的输出重定向与输入重定向类似，它只是将该代码块内的输出写入文件中，符号改成输出重定向符号>。下面以 if/then 结构的输出重定向为例，来说明代码块输出重定向的用法，新建名为 reif.sh 的脚本，内容如下：

```
#例 10-28：reif.sh 脚本演示 if/then 结构的输出重定向
#!/bin/bash

#if/then 结构的输出重定向到 loggg 文件
if [ -z "$1" ]                                    #如果位置参数$1 为空
then
 echo "Positional Parameter is NULL"
fi > loggg                                        #将标准输出重定向到 loggg 文件

#测试 if/then 结构之外的标准输出是否被重定向
echo "--------Normal Stdout ------------"
```

reif.sh 脚本将 if/then 结构的输出重定向到 loggg 文件，if/then 结构的测试条件是位置参数$1 是否为空，如果$1 为空，打印“Positional Parameter is NULL”，由于 if/then 结构的输出将被重定向到 loggg 文件，因此，这句话将写入 loggg 文件。reif.sh 脚本还对 if/then 结构之外的标准输出是否被重定向进行了测试。下面给出 reif.sh 脚本的执行结果，执行 reif.sh，且不带任何参数。因此，位置参数$1 为空，if/then 结构内的输出写入了 loggg 文件，这说明输出重定向确实起了作用。而 if/then 结构之外的测试语句则正常输出到屏幕，这说明输出重定向只在代码块内有效。

```
#例 10-28 reif.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH10$ chmod u+x reif.sh
alloy@alloy:~/linuxshell/CH10$ ./reif.sh     #if/then 结构外面，标准输出为屏幕
--------Normal Stdout ------------
alloy@alloy:~/linuxshell/CH10$ cat loggg
Positional Parameter is NULL                 #if/then 结构内输出到 loggg 文件
alloy@alloy:~/linuxshell/CH10$
```

代码块重定向在一定程度上增强了 Shell 脚本处理文本文件的灵活性，它可以让一段代码很方便地处理一个文件（只要将该文件输入重定向到该代码块），本书第 17 章将介绍一个结合代码块重定向实现文本处理的一个实例。

10.3 命令行处理

前面介绍了大量的 Shell 命令，我们也看到了 Shell 如何读取输入数据，如何处理单引号、双引号、反引号等符号，以及如何根据环境变量 IFS 将命令行分割成字符，所有的这些都是

由 Shell 自动完成的，我们称之为命令行处理。命令行处理贯穿于 Shell 编程的全部，本节从 Shell 程序设计人员的角度详细介绍命令行处理的内在机制。

10.3.1 命令行处理流程

Shell 从标准输入或脚本读取的每一行称为管道（pipeline），每一行包含一个或多个命令，这些命令用管道符隔开，Shell 对每一个读取的管道都按照图 10-4 给出的流程进行处理。

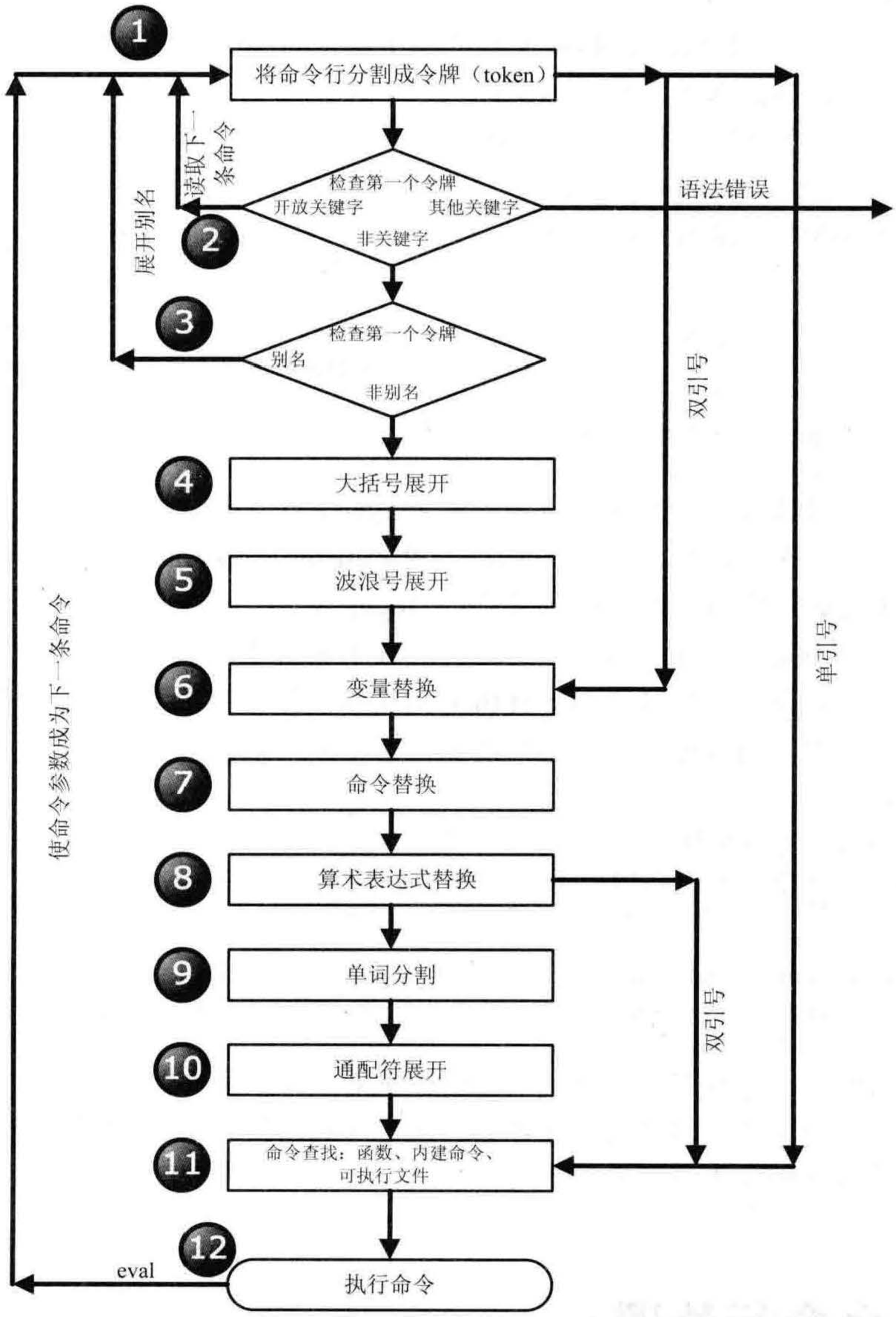


图 10-4 命令行处理的流程

命令行处理流程共有 12 个步骤，图 10-4 已将这 12 个步骤标出，下面依次解释这 12 个步骤。

1）将命令分割成令牌（token），令牌之间以元字符分隔，Shell 的元字符集合是固定不变的，包括空格、Tab 键、换行字符、分号（;）、圆括号、输入重定向符（<）、输出重定向符（>）、管道符（|）和&符号，令牌可以是单词（word）、关键字，也可以是 I/O 重定向器和分号。

2）检查命令行的第一个令牌是否为不带引号或反斜杠的关键字，如果此令牌是开放关键字，开放关键字指 if、while、for 或其他控制结构中的开始符号，Shell 就认为此命令是复合命令，并为该复合命令进行内部设置，读取下一条命令，再次启动进程。如果此令牌不是复合命令的开始符号，如该令牌是 then、else、do、fi、done 等符号，这说明该令牌不应该处在命令行的首位，因此，Shell 提示语法错误信息。

3）检查命令行的第一个令牌是否为某命令的别名，这需要将此令牌与别名（alias）列表逐个比较，如果匹配，说明该令牌是别名，则将该令牌替换掉，返回步骤 1），否则进入步骤 4）。这种机制允许别名递归，也允许定义关键字别名，比如可以用下面命令定义 while 关键字的别名 when。

```
alias when=while
```

4）执行花括号展开，比如 h{a,i}t 展开为 hat 或 hit。

5）将单词开头处的波浪号（~）替换成用户的根目录$HOME。

6）将任何开头为$符号的表达式执行变量替换。

7）将反引号内的表达式，执行命令替换。

8）将$((string))的表达式进行算术运算。

9）从变量、命令和算术替换的结果中取出命令行，再次进行单词切分，与步骤 1）不同的是，此时不再用元字符分隔单词，而是使用$IFS 分隔单词。

10）对于*、?、[...]等符号，执行通配符展开，生成文件名。

11）将第一个单词作为命令，它可以是函数、内建命令和可执行文件。

12）在完成 I/O 重定向与其他类似事项后，执行命令。

命令行处理步骤是由 Shell 自动完成的，用户不能清晰地看到 Shell 完成的每一个步骤，但是，对于一个需要深入理解 Shell 程序的人来说，是有必要知道每个步骤中命令行是如何被转换的。命令行处理步骤看起来有点复杂，为了便于读者理解命令行处理步骤，下面举例说明 Shell 是如何处理一个命令行的，假设我们在/root 目录下输入下面的命令行：

```
echo ~/i* $PWD `echo Yahoo Hadop` $((21*20)) > output
```

Shell 处理此命令行的步骤如下：

1）Shell 首先将命令行分割成令牌，分割成的令牌如下，我们在命令行下方用数字标出各个令牌：

```
echo ~/i* $PWD `echo Yahoo Hadop` $((21*20))
|--1-||--2-| |--3---| |---------4-------------| |-----5-----|
```

需要注意的是，重定向>output 虽已被识别，但是它不是令牌，Shell 将在后面对 I/O 重定向进行处理。

2）检查第一个单词 echo 是否为关键字，echo 显然不是开放的关键字，所以命令行继续下面的判断。

3）检查第一个单词 echo 是否为别名，echo 不是别名，命令行继续往下处理。

4）扫描命令行是否需要花括号展开，这条命令没有花括号，命令行继续往下处理。

5）扫描命令行是否需要波浪号展开，命令行中存在波浪号，令牌 2 将被修改，命令行变为如下形式：

```
echo /root/i* $PWD `echo Yahoo Hadop` $((21*20))
|--1-||----2---| |--3---| |---------4-------------| |-----5-----|
```

6）扫描命令行中是否存在变量，若存在变量，则进行变量替换，该命令行中存在环境变量 PWD。因此，令牌 3 将被修改，命令行变为如下形式：

```
echo /root/i* /root `echo Yahoo Hadop` $((21*20))
|--1-||----2---| |--3-| |---------4-------------| |-----5-----|
```

7）扫描命令行中是否存在反引号，若存在，则进行命令替换，该命令行存在命令替换。因此，令牌 4 将被修改，命令行变为如下形式：

```
echo /root/i* /root Yahoo Hadop $((21*20))
|--1-||----2---| |--3-| |-------4-------| |-----5-----|
```

8）执行命令行中的算术替换，令牌 5 将被修改，命令行变为如下形式：

```
echo /root/i* /root Yahoo Hadop 420
|--1-||----2---| |--3-| |-------4-------| |-5-|
```

9）Shell 将对前面所有展开后产生的结果进行再次扫描，依据$IFS 变量值对结果进行单词分割，形成如下形式的新命令行：

```
echo /root/i* /root Yahoo Hadop 420
|--1-||----2---| |--3-| |--4---| |---5--| |-6-|
```

由于$IFS 是空格，因此，命令行被分割为 6 个令牌，Yahoo Hadop 被分成两个令牌。

10）扫描命令行中的通配符，并展开，该命令行中存在通配符*，展开后，命令行变为如下形式：

```
echo /root/indirect.sh /root/install.log /root/install.log.syslog /root Yahoo Hadop 420
 |--1-||--------2---------| |--------3-------| |---------4----------------| |--5-|
|---6--| |---7--| |-8-|
```

i*展开为当前目录下所有以 i 开头的文件，该目录下有三个以 i 开头的文件：indirect.sh、install.log 和 install.log.syslog，因此，令牌 2 又被分为令牌 2、3 和 4。

11）此时，Shell 已经准备执行命令了，它寻找 echo，echo 是内建命令。

12）Shell 执行 echo 命令，此时执行>output 的 I/O 重定向，再调用 echo 命令，显示最后的参数，执行结果如例 10-29 所示。

```
#例 10-29：命令的执行结果
alloy@alloy:~/linuxshell/CH10$ echo ~/i* $PWD `echo Yahoo Hadoop` $((21*20)) > output
alloy@alloy:~/linuxshell/CH10$ cat output
/root/install.log /root/install.log.syslog /root Yahoo Hadoop 420
alloy@alloy:~/linuxshell/CH10$
```

我们再看图 10-4 右边的三个跳转箭头，从第 1 步跳转到第 11 步的箭头上标注的是单引号，从第 1 步跳转到第 6 步和第 8 步跳转到第 11 步的箭头上标注的是双引号，因此，从这个角度来看，引用是一种使 Shell 在上述 12 个步骤中跳转的方法。单引号内所有的字符都按照字面意思被解析，因此，Shell 无须再进行第 3～10 步的处理。而双引号内允许变量替换、命令替换和算术运算，所以，第 6～8 步双引号是无法忽略的。

10.3.2 eval 命令

图 10-4 的左侧跳转箭头从执行命令步骤跳转到初始步骤，这正是 eval 命令的作用。eval

命令将其参数作为命令行，让 Shell 重新执行该命令行，eval 的参数再次经过 Shell 命令行处理的 12 个步骤。

eval 在处理简单命令时，与直接执行该命令无区别，请看下面的例 10-30，用 eval 处理显示变量值的操作与直接用 echo 显示变量值的效果是相同的。

```
#例 10-30：eval 执行简单命令
alloy@alloy:~/linuxshell/CH10$ var1=BLACK
alloy@alloy:~/linuxshell/CH10$ eval echo $var1   #对于简单命令，是否用 eval 的效果是一样的
BLACK
alloy@alloy:~/linuxshell/CH10$ echo $var1
BLACK
alloy@alloy:~/linuxshell/CH10$
```

例 10-30 中是否用 eval 的效果一样，主要是因为例子中的变量很简单，如果我们将变量赋一个特殊值，那么 eval 的效果就会显现出来了。请看例 10-31，例中第 1 条命令将 pipe 变量赋为管道符，第 2 条命令预期的目的是将当前目录下的文件列出来，再通过管道发送给 wc -l 命令以统计行数，中间的管道符引用了 pipe 变量，但是，执行第 2 条命令出现语法错误，提示名为“|”和“wc”的文件或目录不存在，原因在于：Shell 在处理 ls $pipe wc -l 命令行时，第 1 步扫描没有发现有管道符，直到第 6 步变量替换之后，命令行才变成 ls | wc -l，第 9 步根据$IFS 变量将命令行重新分割成 4 个令牌，第 11 步将 ls 当作命令，后面的 3 个令牌|、wc 和-l 被解析为 ls 命令的参数，由于该目录下没有|和 wc 等文件或目录，因此，Shell 报语法错误。第 3 条命令在第 2 条命令之前使用 eval 命令，得到了预期的结果，Shell 对 eval ls $pipe wc -l 命令行第 1 轮的处理与 ls $pipe wc -l 一样，得到 ls | wc -l 命令行，由于 eval 命令的作用，ls | wc -l 命令行被重新提交到 Shell，Shell 在第 1 步以元字符对 ls | wc -l 命令行进行令牌分割，管道符属于元字符，Shell 成功地将此命令行解释为两个命令，且中间用管道符连接，从而用 wc -l 命令统计出 ls 命令结果的行数。

```
#例 10-31：eval 执行复杂命令
alloy@alloy:~/linuxshell/CH10$ pipe="|"
#第 1 条命令：piple 变量的值是管道符
#第 2 条命令：Shell 将$pipe 和 wc 看做 ls 命令的参数，出现语法错误
alloy@alloy:~/linuxshell/CH10$ ls $pipe wc -l
ls: 无法访问 |: 没有那个文件或目录
ls: 无法访问 wc: 没有那个文件或目录
#第 3 条命令：eval 使$pipe 被解析为管道符，命令行得以正确执行
alloy@alloy:~/linuxshell/CH10$ eval ls $pipe wc -l
12
alloy@alloy:~/linuxshell/CH10$
```

从例 10-31 中，读者应该知道 eval 命令所起的作用。事实上，如果变量中包含任何需要 Shell 直接在命令中看到的字符，就需要使用 eval 命令。命令结束符（;、|、&）、I/O 重定向符（<、>）和引号这些对 Shell 具有特殊意义的符号，必须直接出现在命令行中。

下面再举几个 eval 命令的例子，例 10-32 的脚本用于显示最后一个输入参数，从 6.1.4 节的介绍我们知道，脚本利用位置参数从命令行接受参数，$#符号表示输入参数的数目，显示最后一个输入参数实际上就是显示第$#个参数，新建名为 evalpos.sh 的脚本，内容如下：

```
#例 10-32：evalpos.sh 脚本显示最后一个输入参数
#!/bin/bash
```

```
echo "The number of arguments passed to this script:$#"
echo -n "The last argument is:"
eval echo \$$#                          #用 eval 显示最后一个输入参数
```

evalpos.sh 脚本的核心代码是最后一句：eval echo \$$#，Shell 在第 1 轮扫描该命令行时，反斜杠使紧跟其后的$被转义，即用$符号本身代替了\$，在第 6 步进行变量替换时，Shell 用特殊参数$#替换它的值，第 1 轮扫描后得到的命令行为：

```
echo $4                                 #假设执行 evalpos.sh 脚本时带了 4 个参数
```

eval 命令将第 1 轮扫描后得到的命令行重新提交到 Shell，Shell 同样在第 6 步将位置参数$4 替换成第 4 个输入参数。下面给出 evalpos.sh 脚本的执行结果，evalpos.sh 脚本带了 4 个输入参数，结果确实显示了最后一个输入参数：disk。

```
#例 10-32 evalpos.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH10$ chmod u+x evalpos.sh
alloy@alloy:~/linuxshell/CH10$ ./evalpos.sh army boot cap disk
The number of arguments passed to this script:4
The last argument is:disk      #显示第 4 个输入参数 disk
alloy@alloy:~/linuxshell/CH10$
```

例 10-33 试图将 evalsource 文件中每一行的第 1 列作为变量名，第 2 列作为相应的变量值，evalsource 文件的内容如下：

```
#例 10-33：显示 evalsource 文件内容
alloy@alloy:~/linuxshell/CH10$ cat evalsource
var1 APPLE
var2 BAIDU
var3 CAMEL
var4 DOT
var5 EMULE
alloy@alloy:~/linuxshell/CH10$
```

接着，新建名为 evalre.sh 的脚本，该脚本将 var1～var5 作为变量名，APPLE 到 EMULE 作为对应的变量值，evalre.sh 脚本的内容如下：

```
#例 10-34：evalre.sh 脚本将 evalsource 文件中每一行的第 1 列作为变量名，第 2 列作为变量值
#!/bin/bash

#NAME 变量保存每行的第 1 列，VALUE 变量保存每行的第 2 列
while read NAME VALUE
do
  eval "${NAME}=${VALUE}"      #eval 命令实现 NAME 变量的赋值
done <evalsource                  #灵活运用代码块重定向

#测试 var1～var5 变量的值
echo "var1=$var1"
echo "var2=$var2"
echo "var3=$var3"
echo "var4=$var4"
echo "var5=$var5"
```

evalre.sh 脚本将 while 循环的标准输入重定向到 evalsource 文件，然后使用 read 命令依次读入 evalsource 文件的每一行，将第 1 列保存在 NAME 变量中，将第 2 列保存在 VALUE 变量中，然后执行关键语句 eval "${NAME}=${VALUE}"。以 var1 APPLE 行为例，简单分析 Shell 处理该命令行的过程：第 1 轮进行变量替换，得到 var1=APPLE，eval 将 var1=APPLE 重新提交到 Shell，Shell 完成对 var1 变量的赋值操作。下面给出了 evalre.sh 脚本的执行结果，

可以看到，var1～var5 变量确实已经赋为 APPLE～EMULE 的值。

```
#例 10-34 evalre.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH10$ chmod u+x evalre.sh
alloy@alloy:~/linuxshell/CH10$ ./evalre.sh
var1=APPLE
var2=BAIDU
var3=CAMEL
var4=DOT
var5=EMULE
alloy@alloy:~/linuxshell/CH10$
```

10.4 本章小结

本章深入探讨了 Linux Shell 的变量和引用，首先从变量的替换和赋值的基本操作入手，讨论了 Shell 脚本变量的无类型性，并对 Linux Shell 环境变量和位置参数两种特殊的变量做了详尽分析。然后，介绍了 Linux Shell 中四种引用符号及其意义和用法，重点讨论转义符的用法及其一些特殊的用法。由于变量和引用无处不在，因此，本章是 Shell 编程的基础章节，扎实地掌握本章内容是学习本书后续章节的基础。

10.5 上机提议

1．首先分析下列命令的作用，然后执行它们，验证你的分析。

```
who | grep 'glo'
who | grep '^glo'
sed '/^$/d' text >text.out
sed 's/\([Ll]inux\)/\1(TM)/g' text >text.out
date | cut -c12-16
date | cut -c5-11,25- | sed 's/\([0-9]\{1,2\}\)/\1,/'
```

2．设系统中有两个日志文件：log1 和 log2，请实现将 log1 的全部和 log2 的最后 5 行写入一个新的文件 biglog。(提示：tail -n filename 可以将 filename 的最后 n 行输出到标准输出。)

3．写一个命令，去掉某文件中所有的空格符，并将结果仍然存储到该文本中。

4．写一个命令，输出某 Shell 变量中所包含字符的个数。在此基础上，再写一个命令，输出该变量中所包含字母的个数(空格不计算在内)。(提示：用 sed、wc 命令，以及管道符。)

5．将例 10-7 中 variable2 赋值命令中 sed 后的双引号改为单引号，观察 variable2 的值是否产生变化，分析其中的原因。

```
variable2=`echo $variable1 | sed 's/Yahoo/$replace/g'`
```

6．利用 until 循环实现与 10.2.4 节 rewhile.sh 脚本相同的功能，脚本名为 reuntil.sh，下面给出 reuntil.sh 循环体的代码，请读者据此扩展出 reuntil.sh 脚本，并执行它。

```
until  [ "$filename" = "rc.d" ]
do
read filename
 let "count +=1"
```

```
done < loggg
```

7．通过设置$IFS 变量，利用 read 命令读取/etc/passwd 文件中的数据，将/etc/passwd 文件中的各个域：注册名、口令、用户标识号、组标识号、用户名、用户主目录和命令解释程序，并打印出来。

8．编写一个脚本，该脚本需要有两个输入参数，第一个输入参数指定标准输入，第二个输入参数指定标准输出和标准错误输出，并用命令测试输入、输出和错误输出，脚本完成后需要将标准输入、标准输出和标准错误输出恢复到原状。（提示：可利用位置参数、exec 命令、文件标识符等知识实现该脚本。）

9．如下的 nonintervi.sh 脚本能够以非交互的方式来使用 vi 命令编辑一个文件，执行该脚本，并体会 here-document 的作用。

```
# nonintervi.sh 脚本：用非交互的方式来使用 vi 编辑一个文件
#!/bin/bash
.
if [ -z "$1" ]
then
 echo "Usage: `basename $0` filename"
 exit 1
fi

# 在文件中插入两行，然后保存
vi $1 <<CLOUD
 i
 This is line 1 of the example file.
 This is line 2 of the example file.
 ^[                          #  转义符，请输入 Ctrl+v <Esc>
ZZ
 CLOUD
 exit 0
```

10．写出 Shell 处理下面两条命令的步骤，每个步骤需要标出令牌号。

```
ll `type -path cc` ~globus/.*$(($$%1000))                                    (1)

touch log1 log2
var1=log
var2="Yahoo Hadop"
echo ~+/${var1}[12] $var2 `echo $var2` > output                              (2)
```

11．10.3.2 节的 evalpos.sh 脚本是否适用于输入参数大于 10 个的情况？如果不适用，请修改 evalpos.sh 脚本，使之能够在输入参数大于 10 个的情况下，打印最后一个输入参数。

12．执行下面的操作，分析产生结果的原因。

```
#新建 test 文件，内容如下：
Yahoo Hadop
Google GFS
#依次执行如下命令：
var="cat test"
echo $var
eval $var
```

第11章

Linux/UNIX Shell类型与区别

尽管本书介绍的是bash Shell编程技术，但是，了解Linux/UNIX Shell的起源与特性将有助于程序员基于所学的bash Shell编程技术在其他的Shell下编程，也有助于程序员将bash Shell的程序无缝迁移到其他类型的Shell下。Linux是一套类UNIX操作系统，从UNIX衍生而来，因而，要阐释清楚Linux Shell的种类，则需要从UNIX Shell的起源和发展说起。本章首先概述了Linux/UNIX Shell的起源和分类，然后选择三种具有代表性的Shell从Shell选项、内部变量和内建命令三个角度进行介绍，并着重于bash Shell的比较和分析，这三种代表性的Shell是：从Bourne Shell发展而来的dash，它代表了Bourne Shell的特性；C Shell中的典型代表tcsh，它继承了C Shell的所有风格与特性；兼容Bourne Shell和C Shell诸多特征的Korn Shell，重点介绍了流行的Korn Shell版本——ksh93。

11.1 Linux/UNIX Shell 起源与分类

Shell 是用户与内核进行交互操作的一种接口，Linux 的 Shell 有多种类型。由于 Linux 是源自 UNIX 的，因此，若要阐释清楚 Linux Shell 的种类，需要从 UNIX Shell 的起源和发展说起。

第一个重要的标准 UNIX Shell 是在 1970 年由美国贝尔实验室（Bell Lab）开发的 Bourne Shell，它以开发者 Stephen Bourne 的名字命名。Bourne Shell 是 UNIX 操作系统最初使用的 Shell，而且在每种 UNIX 系统上都可以使用，Bourne Shell 是基于 Algol 语言编写的，它在 Shell 编程方面性能优秀，但是，由于 Bourne Shell 不支持如历史、别名和作业控制等机制，因此，Bourne Shell 在处理交互式操作方面有所欠缺。

20 世纪 70 年代末，在加利福尼亚大学 Berkeley 分校研发的 C Shell 作为 BSD UNIX 的一部分发布，顾名思义，C Shell 是用 C 语言编写的，C Shell 提供了许多标准 Bourne Shell 里不提供的功能。C Shell 使用类似 C 语言的语法，提供交互使用的增强功能，如命令行历史、别名和作业控制。因为这种 Shell 是在大型机上设计的，而且加入了大量附加的特征，所以，C Shell 的运行速度较为缓慢。

由于 Bourne Shell 和 C Shell 都可使用，所以现在 UNIX 用户有了选择余地，同时也在选择 Shell 的问题上产生了困扰。来自 AT&T 的 David Korn 于 20 世纪 80 年代中期开发了 Korn Shell，发布于 1996 年，并且在 1998 年正式成为 UNIX 的 SVR4 分支的组成部分。Korn Shell 不仅能在 UNIX 系统上运行，而且能在 OS / 2、VMS 和 DOS 上运行。它提供与 Bourne Shell 的向上兼容性，增加了许多 C Shell 的受欢迎的特征，而且快捷有效。在 2000 年以前，Korn Shell 的版权一直被 AT&T 控制，目前，Korn Shell 也称为开源软件。Korn Shell 有两种独立的版本：ksh88 和 ksh93，大部分 Korn Shell 版本都是源自于 ksh93，如 IBM 的 AIX 系统。但是，Sun 公司的 Solaris 系统却除外，Solaris 的 Shell 是从 ksh88 的基础上修改而来的。

bash 是大多数 Linux 系统的默认 Shell，它是 Linux 操作系统上一款优秀的 Shell，bash 是 Bourne Again Shell 的简写，正如它的名字所暗示的，bash 是 Bourne Shell 的扩展。bash Shell 与 Bourne Shell 完全向后兼容，并且在 Bourne Shell 的基础上增加了很多特性，bash Shell 包含 C Shell 和 Korn Shell 的很多优点，这使得 bash Shell 具有灵活的编程接口，同时又有友好的用户界面。bash Shell 在 Bourne Shell 上扩展的特性包含以下几方面：

- 命令行编辑。
- 命令历史显示无大小限制。
- 作业控制命令。
- 函数和别名。
- 无大小限制的数组。

除了 bash Shell，我们再介绍几种 Linux 系统使用的 Shell。基于 UNIX 系统的 Bourne Shell，Kenneth Almquist 开发了 Bourne Shell 的小型版本，名为 Almquist Shell，简称 ash。ash 非常小巧，因而，速度非常快，但是，ash 不支持命令行编辑和历史命令查询等功能，因而 ash

不适用于交互式 Shell。ash 因其小巧的特征，适用于内存较小的机器，如嵌入式芯片等。

在 ash 的基础上，Debian Linux 创建了 Debian 专用的 Shell，称为 Debian ash，简称为 dash，Debian 完全由社区人员设计和完善，Debian 项目以创造一份自由操作系统为共同目标，Debian Linux 也是最古老的 Linux 发行版之一，目前流行的桌面版 Linux 系统 Ubuntu 就是从 Debian Linux 发展而来的，Ubuntu 也不例外地采用 dash 作为其默认 Shell。dash 在 ash 里增加了很多新功能，使得 ash 更接近于 Bourne Shell，dash 增加的新功能包括命令行编辑，以及 emacs 和 vi 等文本编辑命令。

tcsh 是 Linux 使用的一种 Shell，它源自 C Shell，同时也是 C Shell 一种流行的开源版本。随着 bash Shell 的流行，tcsh 只能作为 Linux 系统的一种替代型 Shell，tcsh 易于安装，它能很方便地将 UNIX 中 C Shell 风格的环境迁移到 Linux 系统下。

本书其他章节介绍的 Shell 编程是基于 bash Shell 的，Shell 编程的大部分技巧适用于其他类型的 Shell，换句话说，掌握了 bash Shell 的编程技巧就很容易学会在其他类型的 Shell 下编程。不同类型 Shell 的区别体现在 Shell 选项、内部变量和内建命令等方面，后面的内容以 dash、tcsh 和 Korn Shell 为例，介绍上述三种 Shell 的选项、内部变量和内建命令等方面的异同。

11.2 dash 简介

dash 类似于 Bourne Shell，因此，我们将 dash 作为 Bourne Shell 的代表向读者介绍。dash 的选项没有 bash 多，Debian Linux 为 dash 新添了两个 bash 所没有的选项：-E 和-V 选项，打开-E 选项就允许用户利用 emacs 编辑文本文件，而打开-V 选项则允许用户利用 vi 编辑文本文件，由于 emacs 和 vi 是 Debian Linux 新添到 dash 的编辑器，因此，Debian Linux 为 dash 扩展-E 和-V 两个选项不足为怪，我们将 dash 的选项列于表 11-1 中，读者可以将其与 bash 选项进行比较（本书第 9 章表 9-1 所示）。

表 11-1 dash 选项及其意义

简　写	意　义
a	将所有已定义的变量声明为环境变量
e	当脚本发生第一个错误时，退出脚本
f	禁止文件名扩展，即禁用通配（globbing）
i	使脚本以交互模式运行
l	在交互模式时，忽略 EOF 字符
m	转入作业控制（默认在交互模式下激活）
n	读取脚本中的命令，进行语法检查，但不执行这些命令
s	从标准输入（stdin）中读取命令
v	在执行每个命令之前，将每个命令打印到标准输出（stdout）
x	将完整命令写到 stderr
u	当扩展一个未初始化的变量时，向 stderr 写错误信息

续表

简　写	意　义
c …	从…中读取命令
E	允许利用 emacs 编辑文本文件
V	允许利用 vi 编辑文本文件

从表 11-1 可以看出，dash 的-I、-m、-u、-E 和-V 选项都是 bash 所没有的，但是，dash 打开或关闭选项的命令和 bash 是一样的。

dash 的内部变量和 bash 极为相似，这是由于 dash 和 bash 都扩展了 Bourne Shell 的特性，但是，dash 的目标是精简，因此，dash 的内部变量比 bash 要少得多，表 11-2 列出了 dash 的内部变量及其意义。

表 11-2　dash 的内部变量及其意义

变量名称	意　义
CDPATH	cd 命令的搜索路径
HISTSIZE	history 命令所显示历史命令的行数
HOME	用户的默认根目录
IFS	输入域分隔符，默认值是空格、Tab 和新行
MAIL	用户的邮箱文件名称
MAILCHECK	在邮箱文件中检查新邮件的间隔时间
MAILPATH	用逗号分隔的邮箱文件名列表，如果设置此变量， MAIL 变量的值将被覆盖掉
OLDPWD	前一个工作目录的值
PATH	默认的搜索可执行文件的路径
PPID	父 Shell 的进程 ID
PS1	用于设置主 Shell 命令提示符
PS2	用于设置二级 Shell 命令提示符
PWD	当前工作目录值
TERM	Shell 的默认终端

与 bash 类似，dash 有很多内建命令，同样由于 dash 是精简的 Bourne Shell，其内建命令比 bash 少得多，表 11-3 列出了 dash 的内建命令及其意义。

表 11-3　dash 的内建命令及其意义

命令名称	意　义
alias	设置命令或命令行的别名
bg	将作业置于后台运行
cd	切换当前工作目录
echo	打印参数